Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e Statistica

Laurea Triennale

Dipartimento di Informatica

Basi di Dati

Appunti a cura di
Lorenzo Ferraro

13 novembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Basi di Dati*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale `github.com/TonnoArcobaleno/uni-notes`, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1 Introduzione alle basi di dati **1**
1.1 Sistema informativo . . . 1
1.2 Gestione dei dati . . . 1
1.2.1 Informazioni condivise . . . 3
1.3 Sistemi informatici . . . 3
1.3.1 Sistemi informatici operativi . . . 4
1.3.2 Sistemi informatici di supporto alle decisioni . . . 4
1.3.3 OLTP vs OLAP . . . 4
1.3.4 Requisiti per l'analisi dei dati . . . 5
1.4 Big Data e Data Lake . . . 5
1.5 Modelli di DBMS . . . 6
1.5.1 Prima del modello relazionale . . . 6
1.5.2 Modello relazionale . . . 6
1.5.3 Linguaggio per gestire i DB . . . 7
1.5.4 Concorrenza . . . 9
1.6 Progettare una base di dati . . . 10
1.6.1 Problemi della progettazione di DB . . . 12
1.6.2 Modellare la realtà . . . 12
1.6.3 Associazioni . . . 16
1.6.4 Realizzare una vera base di dati . . . 17

2 Progettazione concettuale **18**
2.1 Il modello Entità-Relazione . . . 18
2.2 Cardinalità, identificatori e generalizzazione . . . 21
2.2.1 Cardinalità . . . 21
2.2.2 Identificatori . . . 23
2.2.3 Generalizzazione . . . 24
2.3 Esempio: Società di formazione - Progettazione Concettuale . . . 25

3 Progettazione logica **30**
3.1 Modello relazionale . . . 30
3.1.1 Chiavi . . . 32
3.1.2 Riferimenti per valore . . . 33
3.1.3 NULL . . . 33
3.1.4 Vincoli . . . 34
3.2 Passare dal modello concettuale a quello logico . . . 34
3.2.1 Ristrutturare lo schema ER . . . 34

3.2.2 Tradurre le associazioni . . . 37
3.3 Esempio: Società di formazione - Progettazione Logica . . . 39
3.3.1 Ristrutturare lo schema ER . . . 39
3.3.2 Passaggio allo schema logico . . . 41

4 SQL **44**
4.1 Cos'è SQL? . . . 44
4.2 DDL . . . 44
4.3 Algebra relazionale e DQL . . . 50
4.3.1 Introduzione all'algebra relazionale . . . 50
4.3.2 Algebra relazionale in SQL 1 . . . 55
4.3.3 Altri operatori dell'algebra relazionale . . . 57
4.3.4 Algebra relazionale in SQL 2 . . . 61
4.3.5 Problemi particolari nell'algebra relazionale . . . 63
4.3.6 Operatori avanzati in SQL . . . 68
4.4 DML . . . 74
4.5 DCL . . . 76
4.6 DTL . . . 77
4.7 DBMS attivi . . . 78
4.7.1 Relazione tra trigger e vincoli d'integrità . . . 79
4.7.2 Quando i trigger sono veramente utili . . . 80

5 Dipendenze funzionali **83**
5.1 Cosa sono le dipendenze funzionali? . . . 83
5.2 Implementazione in SQL . . . 84
5.3 Soddisfare una dipendenza funzionale . . . 85
5.4 Definizione di chiave . . . 87
5.5 Un altro insieme di dipendenze funzionali . . . 87
5.6 Calcolare la chiusura di un insieme di attributi . . . 92
5.7 Trovare le chiavi di uno schema . . . 94

Capitolo 1

Introduzione alle basi di dati

1.1 Sistema informativo

Definizione 1.1 – Sistema informativo

Un **sistema informativo** di un'organizzazione è una combinazione di risorse, umane e materiali, e di procedure organizzate per *raccogliere*, *archiviare*, *elaborare* e *scambiare* le informazioni necessarie alle attività **operative** (informazioni di servizio), di **programmazione e controllo** (informazioni di gestione) e di **pianificazione strategica** (informazioni di governo).

Questo sistema è quindi una componente dell'organizzazione che viene utilizzata per gestire le informazioni di interesse, supportando le altre componenti dell'organizzazione (come il sistema produttivo o decisionale, ad esempio).

Definizione 1.2 – Sistema informatico

Il **sistema informatico** è l'insieme delle tecnologie informatiche e della comunicazione (ICT) a supporto delle attività di un'organizzazione.

Definizione 1.3 – Sistema informativo automatizzato

Il **sistema informativo automatizzato** è quella parte del sistema informativo dove le informazioni sono raccolte, elaborate, archiviate e scambiate tramite l'ausilio di un sistema informatico.

Dato che ormai quasi tutti i sistemi informativi sono digitalizzati, i tre termini appena visti tendono spesso a sovrapporsi e a essere usati come sinonimi.

1.2 Gestione dei dati

Negli anni '60/'70 ogni applicazione aveva un proprio file privato dove venivano salvati i propri dati. Questo creava una serie di problemi:

- **ridondanza**, dato che se due applicazioni usavano gli stessi dati questi venivano salvati due volte.
- **inconsistenza**, dato che aggiornare un dato in un file non significava aggiornarlo automaticamente in tutti i file
- **dipendenza dei dati**, dato che ogni applicazione organizzava i propri dati tenendo conto solamente dell'uso che doveva farne

Proprio per questo motivo nascono le **basi di dati**.

Definizione 1.4 – Basi di Dati

Una **Base di Dati** (o Database, **DB**) è un **insieme di file connessi tra loro**. Gli insiemi di dati sono organizzati in *strutture di dati* che rendono più facile la creazione, l'accesso e l'aggiornamento di tali insiemi e ottimizzano la gestione delle risorse fisiche.

Definizione 1.5 – Dati

I **dati** sono dei fatti grezzi che bisogna interpretare e correlare per fornire una vera informazione.

Ad esempio, 3336669990 da solo è un semplice numero, ma se è la risposta alla domanda "Qual è il numero di telefono di X", allora ha tutto un altro significato, e diventa quindi una *informazione* vera e propria.

Quando si registrano delle informazioni, queste possono essere

- **dati strutturati**, ossia rappresentati da brevi stringhe di simboli e da numeri,
- o **dati non strutturati**, ossia testi scritti in linguaggio naturale.

La struttura dell'informazione dipende dal suo utilizzo e può essere modificata. Ad esempio, un tempo bastava il nome e il cognome di una persona per identificarla in modo efficace, mentre oggi utilizziamo anche la data di nascita, il luogo di nascita e, se necessario, il codice fiscale.

L'obiettivo di una base di dati non è solo il memorizzare i dati in modo compatto, ma soprattutto **facilitarne l'elaborazione** sulla base delle loro relazioni. Ad esempio, parlando dei dati strutturati, vi si può accedere singolarmente tramite delle "interrogazioni", dette *query*, e le relazioni tra i dati individuali sono salvate nella *struttura dei record*.

I dati contenuti nei DB sono suddivisi in due categorie:

- **dati**: rappresentazioni di fatti conformi alla definizione dello schema delle basi di dati. Questi hanno diverse caratteristiche:
  - Sono organizzati in insiemi strutturati e omogenei, con relazioni definite tra i vari insiemi.

  - Sono molti di più rispetto ai metadati, e non sono gestiti nella RAM (memoria temporanea).
  - Sono accessibili tramite transazioni, sequenze di istruzioni che devono tutte andare a buon fine per apportare vere modifiche.
  - Sono protetti sia dall'accesso di utenti non autorizzati, sia da malfunzionamento di hardware e/o software.
  - Sono utilizzabili contemporaneamente da più utenti.
- **metadati**: descrivono fatti sullo schema dei dati, utenti autorizzati, applicazioni e così via. Sono descritti da uno schema e sono interrogabili come i dati.

Definizione 1.6 – Sistemi di Gestione di Basi di Dati

I **Sistemi di Gestione di Basi di Dati** (o Database Management System, **DBMS**) sono **software per la gestione di molti dati** che si trovano nella memoria secondaria.

Questi offrono gli strumenti per **definire lo schema** di un DB, che va definito prima di inserire i dati, per **scegliere le strutture dati adeguate**, per **memorizzare i dati** rispettando i vincoli dello schema scelto, e per **recuperare/modificare** i dati in modo interattivo o attraverso dei programmi. Sono dunque il ponte tra gli utenti e il DB vero e proprio.

1.2.1 Informazioni condivise

In un'organizzazione, ogni componente è interessata a una porzione del Sistema Informativo. Quando queste porzioni si sovrappongono, la base di dati diventa fondamentale, facendo da *struttura condivisa* tra tutte le componenti, riducendo le ridondanze e le inconsistenze. Questa condivisione, però, non è sempre completa. Infatti, non tutte le componenti dell'organizzazione possono accedere a tutti i dati di questa: le informazioni più private devono essere salvaguardate. Inoltre, la condivisione comporta la necessità di gestire accessi in contemporanea sugli stessi dati: questo è il problema del **controllo della concorrenza**.

1.3 Sistemi informatici

I sistemi informatici si dividono in due macrogruppi:

- **sistemi informatici operativi**
- **sistemi informatici di supporto alle decisioni** (direzionali)

1.3.1 Sistemi informatici operativi

I **sistemi informatici operativi** (o Data processing, DP / Electronic Data processing, EDP / Transaction Processing Systems, TPS) sono organizzati in basi di dati. Qui le applicazioni sono usate per svolgere le attività strutturate e ripetitive di un'azienda nell'area amministrativa e finanziaria. Abbiamo quindi una base di dati, gestita da un DBMS, che lavora con più applicazioni in ambiti come la vendita, la contabilità, la finanza...

Definizione 1.7 – OLTP

L'**OLTP** (On-Line Transaction Processing) è il **paradigma architetturale** usato dai DBMS nei sistemi informatici operativi. Le sue operazioni sono predefinite e relativamente semplici, e ognuna di queste coinvolge pochi dati, che sono però dettagliati e aggiornati.

1.3.2 Sistemi informatici di supporto alle decisioni

I dati nei **sistemi informatici di supporto alle decisioni** (Decision Support Systems, DSS / Management Information Systems, MIS / Executive Information System, EIS) sono organizzati in **Data Warehouse** (DW) e gestiti da un sistema apposito. Le applicazioni, dette di *Business Intelligence*, supportano i processi di controllo delle prestazioni aziendali e di decisione manageriale. Abbiamo quindi una base di dati operazionale e dei possibili dati esterni che vanno dentro il DW che viene gestito da un DBMS dedicato, e infine il sistema OLAP (che ora vedremo) interroga il DW e fornisce le informazioni necessarie agli strumenti di Business Intelligence.

Definizione 1.8 – OLAP

L'**OLAP** (On-Line Analytical Processing) è il **paradigma** usato nei DSS per analizzare i dati. Le sue operazioni sono complesse e casuali, e ognuna coinvolge molti dati, che sono aggregati, storici e non sempre molto attuali.

1.3.3 OLTP vs OLAP

Vediamo le molte differenze tra OLTP e OLAP.

- L'OLTP supporta **l'operatività** dell'azienda, mentre l'OLAP le sue **decisioni**
- L'OLTP ha **molti utenti**, mentre l'OLAP è più **esclusivo**
- I dati dell'OLTP sono **analitici e relazionali**, mentre quelli dell'OLAP sono **sintetici** e **multidimensionali**
- Gli utilizzi dell'OLTP sono **noti a priori**, mentre quelli dell'OLAP **non sono molto prevedibili**.

- Per attività, la quantità di dati dell'OLTP è **bassa** (si parla di decine di dati), mentre quella dell'OLAP è **alta** (si parla di milioni di dati)
- L'OLTP è **orientato all'applicazione**, mentre l'OLAP al **soggetto**
- Nell'OLTP gli aggiornamenti sono **frequenti**, mentre nell'OLAP sono **rari**
- Nell'OLTP i dati sono **sovrascritti** (i dati vecchi non sono interessanti), mentre nell'OLAP sono **importanti anche i dati vecchi**
- L'OLTP è ottimizzato per le **transazioni**, mentre l'OLAP per l'**analisi dei dati**

1.3.4 Requisiti per l'analisi dei dati

I dati dell'OLAP sono presi per essere analizzati. Questi hanno bisogno di requisiti specifici:

- Devono essere **aggregati**: non ci interessa il singolo dato, ma un calcolo su più dati, come la somma, la media o la mediana.
- Le informazioni devono essere **incrociate**: bisogna spesso analizzare i dati da punti di vista diversi.
- Bisogna analizzare i dati su **diversi livelli di dettaglio**: ad esempio, se si scopre un calo nelle vendite in un periodo di una regione, bisogna analizzare quei dati in modo più approfondito per capirne le cause

I sistemi informatici operativi, essendo progettati per un'elaborazione rapida delle transazioni, sono dunque **perfetti per OLTP** ma non adatti per l'OLAP.

1.4 Big Data e Data Lake

Definizione 1.9 – Big Data

Big Data è un termine ampio che si riferisce a tutte le situazioni dove l'approccio *schema-first* tipico dei DB e delle DW è troppo lento o restrittivo. Si parla in questo caso delle 3 V:

- **Volume**
- **Varietà**
- **Velocità**

Sono questi 3 fattori a mandare in crisi i sistemi tradizionali (quantità di dati o eccessive, o troppo variegate, o dati che devono essere elaborati velocemente).

I Big Data sono associati solitamente ai sistemi NoSQL, al Machine Learning, e all'**approccio Data Lake**.

Definizione 1.10 – Data Lake

Un **Data Lake** è un **ambiente di data storage** a basso costo creato per gestire enormi quantità di dati non elaborati e di qualsiasi formato, inclusi dati strutturati, semi-strutturati e non strutturati.

Quasi tutti i Data Lake utilizzano lo storage a oggetti basato su cloud, come Google Cloud Storage. Questi sono nati per gestire il flusso di nuovi big data, molti non strutturati, creati da app e servizi connessi a Internet. A differenza dei DB e DW tradizionali, questi non richiedono uno schema di dati definito e possono anche memorizzare diversi tipi di dati in più formati tutti nello stesso posto. Questo li rende più adatti per i data scientist, grazie al loro schema *on-read*: i dati vengono strutturati solo quando vanno letti, al contrario dello schema *on-write* delle DW dove i dati sono strutturati fin da subito.

1.5 Modelli di DBMS

1.5.1 Prima del modello relazionale

Il modello relazionale è ad oggi lo standard per i DBMS. Lo analizzeremo tra poco, ma per ora diamo un'occhiata ai modelli che lo hanno preceduto.

Il primo modello di DB è il **modello gerarchico**, uscito a metà degli anni '60. Questo organizza i dati in una struttura ad albero dove ogni record figlio può avere solo un genitore. Questo modello è stato poi esteso dal modello **reticolare**, che ha trasformato l'albero (che è un grafo aciclico e connesso) in un grafo generico.

A metà degli anni '80 arrivano anche i primi sistemi **orientati a oggetti**. In questo modello troviamo gli *attributi* che descrivono lo stato di un oggetto, e i *metodi* che ne descrivono invece il comportamento. Al contrario del modello relazionale, però, non esiste ancora un modello universalmente riconosciuto.

1.5.2 Modello relazionale

Definizione 1.11 – Modello relazionale

Nel 1970 E.F. Codd, lavorando per IBM, idea il **modello relazionale**, che diventerà lo standard nel 1993. Qui **dati e relazioni sono rappresentati entrambi come valori**. Non ci sono puntatori espliciti come nei modelli reticolari e gerarchici, portando l'astrazione ad un livello più in alto.

I record sono gli **oggetti**, mentre le informazioni di interesse sono i **campi**. Lavoriamo con delle **tabelle** di oggetti omogenei, dove le colonne sono rappresentate dai campi mentre le righe dagli oggetti appunto. Vediamo un esempio di DB relazionale:

Studenti

Matricola	Nome	Cognome
123456	Mario	Rossi
654321	Andrea	Pirlo
456789	Francesco	Totti
987654	Cristiano	Ronaldo

Corsi

Codice	Titolo	Insegnante
01	Analisi	Aiello
02	Chimica	Sterbini
03	Fisica	Carlucci

Esami

Studente	Voto	Corso
123456	21	02
987654	30	03
456789	28	01

Notiamo come le relazioni tra i diversi oggetti siano rappresentate dai valori stessi appunto: nella tabella Esami, riusciamo a capire chi ha svolto quale esame e ha preso quale voto grazie ai valori nella tabella. Ad esempio, capiamo che Mario Rossi ha preso 21 al corso di Chimica del prof. Sterbini. È importante capire, però, che queste relazioni non dipendono dal nome del campo: bisogna sempre poter rinominare le colonne senza perdere le relazioni.

1.5.3 Linguaggio per gestire i DB

Ora che abbiamo definito cosa sono i DB, abbiamo bisogno di un **linguaggio** per gestirli. Per capire come definire questo linguaggio, però, dobbiamo prima capire appieno come sono strutturati i database. Possiamo distinguere 3 livelli distinti:

- **Vista Logica**, ossia le diverse viste degli utenti in base ai loro diritti di accesso. Non tutti possono accedere a tutti i dati del database, e per questo è necessario creare viste diverse.
- **Logico**, ossia la descrizione strutturale del database, come i dati vanno suddivisi, senza però preoccuparsi di come questi vengano salvati fisicamente.
- **Fisico**, ossia l'implementazione fisica dei dati sulle memorie di massa.

Per progettare un database, bisogna partire dal livello **logico**. Dobbiamo qui capire quali dati vogliamo salvare nel nostro DB, le loro relazioni, i loro vincoli e in che modo strutturare le tabelle. Riprendendo l'esempio visto prima con le tabelle Studenti, Corsi e Esami, avremmo scritto:

Studenti(Matricola char(8), Nome char(20), Cognome char(20))
Corsi(Codice char(2), Titolo char(20), Insegnante char(20))
Esami(Studente char(8), Voto int, Corso char(2))

Si passa poi al livello **fisico**, che descrive quali strutture dati ausiliarie bisogna utilizzare per facilitare l'uso del DB. Infine, vi sono le **viste logiche**, che permettono ad utenti con vari livelli di accesso di vedere solo determinate informazioni.

Proposizione 1.1 – Indipendenza fisica e logica

Tramite l'approccio con tre livelli, si garantisce l'**indipendenza fisica e logica** dei dati nel DBMS. Infatti, i programmi non devono essere modificati a seguito di modifiche sull'organizzazione fisica (**indipendenza fisica**), e nemmeno a seguito di modifiche allo schema logico (**indipendenza logica**).

Proprietà 1.1 – Modalità di accesso al DBMS

Dato che non tutti sono in grado di accedere a un DBMS come lo farebbe un programmatore, è necessario creare **diverse modalità di accesso** per soddisfare tutti gli utenti.

- **Interfaccia grafica**: è lo strumento visivo per gli utenti finali che fa interagire con il DBMS solo tramite bottoni e tabelle. Il più semplice di tutti.
- **Linguaggio di interrogazione per utenti non programmatori**: sono quelle interfacce usate dagli utenti per cercare qualcosa con dei filtri selezionati. L'esempio perfetto è quando si cerca su Amazon un oggetto con dei filtri selezionati (ad esempio prezzo < 50€).
- **Linguaggio di programmazione per sviluppatori di applicazioni**: sono quei linguaggi che fanno comunicare l'applicazione con il DBMS, come ad esempio Java o Python.
- **Linguaggio per lo sviluppo di interfacce di applicazioni**: sono quei linguaggi usati per progettare la grafica attraverso cui l'utente finale vede le tabelle e legge/inserisce i dati.

Proprietà 1.2 – Proprietà dei DB

Un DBMS deve garantire le seguenti proprietà di una base di dati:

- **Integrità**, ossia mantenere i dati coerenti con i vincoli prestabiliti.
- **Sicurezza**, ossia proteggere i dati da usi non autorizzati.
- **Affidabilità**, ossia proteggere i dati da malfunzionamenti hardware e software.

Riepiloghiamo quindi i vantaggi e gli svantaggi dei DBMS:

Tabella 1.1: Vantaggi e Svantaggi di un DBMS

Vantaggi	Svantaggi
Indipendenza dei dati	Necessario schema logico
Recupero efficiente dei dati	Possono essere costosi e complessi
Integrità e sicurezza dei dati.	Dati solo strutturati e omogenei
Dati protetti da malfunzionamenti	Non adatti per le applicazioni OLAP
Accessi concorrenti gestiti correttamente	
Amministrazione dei dati	
Riduzione dei tempi per lo sviluppo di applicazioni	
Ottimi per le applicazioni OLTP	

Definizione 1.12 – Transazioni

Una **transazione** è una **sequenza di azioni** di lettura e scrittura in memoria permanente e di elaborazioni di dati in RAM, che segue una serie di proprietà chiamate **ACID**.

Le proprietà ACID sono:

- **Atomicità**: le transazioni che per qualche motivo terminano prima di essere completate sono eliminate totalmente, come se non fossero mai iniziate. Dunque deve essere completata completamente (*committed*), oppure non deve essere eseguita affatto (*rolled back*).
- **Consistenza**: le transazioni devono lasciare i dati in uno stato coerente con i vincoli imposti.
- **Isolamento**: in caso di più transazioni, gli effetti di una non devono dipendere dalle altre.
- **Durabilità** (Persistenza): le modifiche fatte da una transazione terminata sono permanenti.

Un esempio di transazione è: "Trasferisci 1000€ dal conto c1 al conto c2". Il DBMS in questo caso cerca c1, modifica il suo `saldo` in `saldo - 1000`, cerca c2 e modifica il suo `saldo` in `saldo + 1000`.

1.5.4 Concorrenza

Quando due transazioni che lavorano sugli stessi dati lavorano insieme, possono verificarsi dei problemi. Facciamo un esempio.

Ipotizziamo che in un DBMS due utenti facciano partire due transazioni: la transazione 1 è "Aggiungi 1000€ sul conto c1", mentre la transazione 2 è "Aggiungi 500€ sul conto c1". Le transazioni partono insieme, e nel DBMS succede questo:

1° la transazione 1 cerca c1

2° la transazione 2 cerca c1

3° la transazione 1 modifica il `saldo` di c1 in `saldo + 1000`

4° la transazione 2 modifica il `saldo` di c1 in `saldo + 500`

Partendo con un valore `saldo = 0`, ci si aspetterebbe che il saldo finale sia `saldo = 1500`, ma in realtà il valore sarà `saldo = 500`. Questo perché ogni transazione, una volta letto un valore (in questo caso il `saldo` di c1), lo salva in un suo spazio di memoria privato. Dunque in questo caso la prima transazione salva il valore 0, e anche la seconda. La prima quindi aggiorna il saldo al valore 1000, ma la seconda lo sovrascrive a 500.

Si potrebbe pensare di risolvere questo problema semplicemente verificando a posteriori che il risultato sia corretto, ma questo è praticamente impossibile: infatti, in un DBMS reale arrivano transazioni di continuo, e dunque per fare queste verifiche dovremmo perdere un sacco di tempo e annullare molte transazioni mentre verifichiamo che sia tutto ok.

Proposizione 1.2 – Transazioni concorrenti

Per risolvere il problema della concorrenza, bisogna far sì che più transazioni in parallelo **lavorino come se fossero in serie**. Esistono per questo scopo dei **protocolli** con regole rigide a priori che tutte le transazioni devono rispettare.

1.6 Progettare una base di dati

Per progettare una base di dati si intende progettarne la struttura dei dati e delle applicazioni. Questa è la fase più importante e delicata, ma non è l'unica: fa parte di un contesto più grande, ossia il **ciclo di vita dei sistemi informativi**.

Definizione 1.13 – Ciclo di vita dei sistemi informativi

Il **ciclo di vita dei sistemi informativi** è la sequenza di attività svolte da analisti, progettisti, utenti... che permette di sviluppare e usare una base di dati.

Prende il nome di ciclo perché una fase può essere ripetuta anche più volte prima di arrivare al risultato finale, magari per risolvere dei problemi che si sono presentati solo dopo averla conclusa. Questo ciclo è composto da 6 fasi:

1° **Studio della fattibilità**: si valuta se il progetto è effettivamente fattibile con le risorse disponibili.

2° **Raccolta e analisi dei requisiti**: si decide cosa deve fare il sistema nel concreto chiedendo agli utenti quali funzioni sono necessarie e quali no (spesso eseguita in contemporanea con la fase di Progettazione).

3° **Progettazione**: si traducono le richieste degli utenti in modelli astratti e schemi formali.

4° **Implementazione**: si scrive il codice, si creano le tabelle e le si iniziano a popolare con i dati iniziali.

5° **Validazione e collaudo**: si verifica che il sistema rispetti tutti i requisiti e vincoli e che non ci siano errori.

6° **Funzionamento**: si utilizza il sistema normalmente, monitorandolo costantemente per verificare il corretto funzionamento.

Questo ciclo di vita dei sistemi informativi si traduce per le basi di dati in 4 fasi:

1° **Analisi dei requisiti**

2° **Progettazione concettuale**

3° **Progettazione logica**

4° **Progettazione fisica**

Analizzeremo più avanti ogni singola fase, ma è importante capire che la progettazione si divide sempre in due aspetti:

- **Progettazione di dati**
- **Progettazione di applicazioni**

Entrambe le progettazioni, però, avvengono attraverso lo stesso mezzo: i **modelli informatici**.

Definizione 1.14 – Modello informatico

Un **modello informatico** (o, più in generale, **modello astratto**) è la rappresentazione formale di idee e di conoscenze relative a un fenomeno.

La realtà è enorme e disordinata: per questo usiamo dei modelli che selezionano solo alcune conoscenze utili a descrivere il fenomeno che ci interessa, tramite costrutti matematici o schemi fissi. La stessa realtà, quindi, in base alle necessità può essere modellata in più modi, conservando ogni volta informazioni diverse.

Proprietà 1.3 – Tipi di modelli

Esistono due tipi di modelli:

- **modelli logici**: indipendenti dalle strutture fisiche ma supportati nativamente dai DBMS, come ad esempio il modello relazionale.
- **modelli concettuali**: indipendenti dalle modalità di realizzazione, rappresentanti le entità del mondo reale e le loro relazioni nelle prime fasi della progettazione del DB, come ad esempio il modello ER (Entità-Relazione) che studieremo nello specifico più avanti. Si trovano a un livello di astrazione ancora più alto dei modelli logici.

Definizione 1.15 – Modello di dati

Un **modello di dati** è un insieme di strutture utilizzate per organizzare i dati di interesse e le loro relazioni. Il componente fondamentale in queste strutture sono i **costruttori di tipo**, ossia un meccanismo che permette di combinare tipi semplici per formare tipi complessi.

Per capire cosa sono i costruttori di tipo facciamo un esempio. Dei tipi semplici per un oggetto di tipo Studente possono essere ad esempio Matricola (testo), Nome (testo) e Cognome (testo). Se prendiamo un altro tipo semplice Voto (intero), combinando Matricola e Voto possiamo creare un Record (Matricola, Voto) che rappresenta un esame. Questo ci è possibile farlo proprio grazie al costruttore di tipo.

1.6.1 Problemi della progettazione di DB

Durante la progettazione di una base di dati vi sono diversi aspetti problematici da analizzare:

1) **Aspetto ontologico**: cosa bisogna rappresentare della realtà nel nostro DB?
2) **Aspetto logico**: come modelliamo i dati?
3) **Aspetto linguistico**: che linguaggio formale usiamo?
4) **Aspetto pragmatico**: per chi stiamo creando questa base di dati? Di cosa necessitano i nostri utenti?

Parlando dell'aspetto ontologico, bisogna distinguere tre tipi di conoscenza:

- Conoscenza *concreta*, ossia i fatti da rappresentare
- Conoscenza *astratta*, ossia i vincoli sulla conoscenza concreta
- Conoscenza *procedurale*, ossia le operazioni che gli utenti potranno effettuare sui dati

Ci concentreremo in futuro in particolare sui primi due tipi di conoscenza.

Per l'aspetto logico invece ci concentriamo sullo studio dei modelli di dati, che sono dei meccanismi per descrivere la struttura della conoscenza concreta. Questo definisce le classi, gli oggetti, le relazioni... usati per descrivere la realtà, e definisce anche lo schema usato. Per il momento useremo una notazione che prende il nome di **diagramma ER** (**Entità-Relazione**).

1.6.2 Modellare la realtà

Definizione 1.16 – Entità

Un'**entità** rappresenta un **elemento del mondo reale** di cui ci interessa rappresentare alcune **proprietà**. Ogni entità appartiene a un **tipo**, che ne specifica la natura, e può far parte di una **collezione**, ossia un insieme di entità omogenee variabile nel tempo.

Definizione 1.17 – Proprietà di un'entità

Le **proprietà** sono fatti che ci interessano solo perché **descrivono caratteristiche di determinate entità**. Queste sono rappresentate da una coppia di valori del tipo (Attributo, valore di un certo tipo).
N.B. Alcuni fatti in un contesto potrebbero essere interpretati come proprietà e in altri come entità. Ad esempio, un Autore in una Biblioteca può essere una proprietà di un'entità Libro, ma può essere un'entità a parte per una Casa Editrice.

Ad esempio, Diego è un'entità di tipo Persona, con proprietà (Nome, "Diego":string) e (Eta, 20:int), che fa parte della Collezione delle persone che studiano alla Sapienza.

Definizione 1.18 – Oggetto

Un **oggetto** è la definizione informatica formale di un'entità del dominio, caratterizzato da 3 aspetti:

- **stato**, ossia l'insieme delle costanti e variabili che lo modellano
- **comportamento**, ossia l'insieme delle sue procedure locali che prendono il nome di *metodi*
- **identità**, che lo rende unico e distinguibile dagli altri

Un oggetto può poi rispondere a una serie di messaggi attraverso valori memorizzati o calcolati al momento. Un **tipo oggetto** è l'insieme dei messaggi a cui un insieme di oggetti può rispondere. I nomi di questi messaggi sono detti **attributi degli oggetti**.

Definizione 1.19 – Classe

Una **classe** è un **insieme di oggetti** dello stesso tipo, modificabile in modo da includere o escludere elementi specifici dall'insieme. È la controparte informatica della collezione di entità.

Una stessa classe può essere rappresentata a diversi livelli di specifica. Ad esempio, la stessa classe Persona può essere rappresentata in tre modi differenti:

Persona

Persona
Nome
Cognome
DataNascita
Sesso
Indirizzo
LinguaParlate

Persona	
Nome:	string
Cognome:	string
DataNascita:	date
Sesso:	(M; F)
Indirizzo:	[Via: string; Citta: string]
LinguaParlate:	seq string

Definizione 1.20 – Gerarchie di classi

Spesso le classi sono organizzate in una **gerarchia**, dove vi è una classe più generale detta **superclasse** e una o più classi più specifiche dette **sottoclassi**. Esistono le **relazioni di sottoclasse**, con le seguenti proprietà:

- Sono *antisimmetriche, riflessive e transitive*
- Se C è sottoclasse di C', allora il tipo degli elementi di C è sottotipo degli elementi di C' (*vincolo intensionale*)
- Se C è sottoclasse di C', allora gli elementi di C sono un sottoinsieme degli elementi di C' (*vincolo estensionale*)

Nelle gerarchie le proprietà sono **ereditate**, e gli elementi che compongono una sottoclasse sono un sottoinsieme degli elementi della superclasse. Ad esempio, in ambito universitario gli Studenti sono una superclasse, mentre gli Studenti di Ingegneria e gli Studenti di Medicina sono due sottoclassi.

Definizione 1.21 – Gerarchia tra tipi oggetto

Oltre alle gerarchie di classe vi sono anche le **gerarchie di tipi oggetto**. Esistono anche qui le **relazioni di sottotipo**, con diverse proprietà:

- Sono *antisimmetriche, riflessive e transitive*
- Se T è sottotipo di T', allora gli elementi di T possono essere usati in ogni contesto in cui sono usati gli elementi di T' (*principio di sostitutività*)

Definizione 1.22 – Ereditarietà

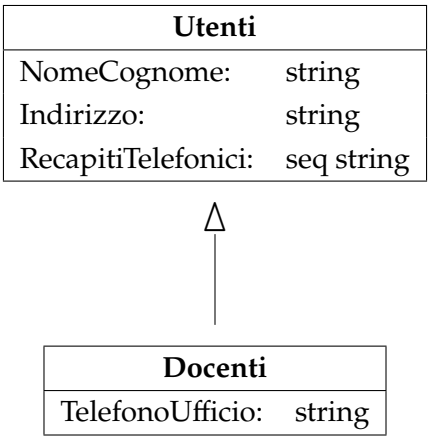
Abbiamo già definito il concetto di **ereditarietà** con le gerarchie di classi e di tipo oggetto, ma vi sono due diversi tipi di ereditarietà:

- **Ereditarietà di tipo**, che definisce solo quali attributi e metodi deve avere la classe derivata
- **Ereditarietà di implementazione**, che rappresenta concretamente il codice della classe derivata

Si parla poi di **ereditarietà stretta** quando l'ereditarietà viene usata non solo per riciclare codice, ma per creare delle vere sottoclassi logiche. Qui gli attributi delle sottoclassi ereditati possono solo essere aggiunti, e non cancellati (non si possono perdere attributi delle superclassi, così da mantenere il principio di sostitutività), ma si possono ridefinire solo specializzandone il tipo.

Per indicare l'ereditarietà tra classi usiamo una freccia che va dalla sottoclasse alla superclasse che si chiude con un triangolo vuoto. Ad esempio, per definire Docenti come una sottoclasse di

Utenti si usa uno schema di questo tipo:



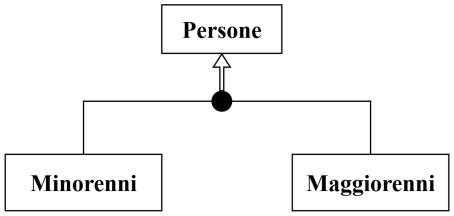
Proprietà 1.4 – Proprietà sottoclassi

Quando una superclasse viene suddivisa in più sottoclassi, si definiscono due proprietà:

- **Disgiunzione** (Esclusività): le sottoclassi sono esclusive se gli elementi di una non fanno parte di nessun'altra
- **Copertura**: le sottoclassi sono totali se l'unione di tutte queste copre l'intera popolazione della superclasse

Se entrambe le proprietà sono rispettate, le sottoclassi formano una **partizione** della superclasse.

Per dire che le sottoclassi sono disgiunte si usa un pallino nero, mentre per dire che coprono la superclasse si usa una doppia freccia.



In questo esempio, Minorenni e Maggiorenni sono due sottoclassi esclusive e coprono anche tutte le persone, dunque c'è sia il pallino che la doppia freccia.

1.6.3 Associazioni

Definizione 1.23 – Associazioni

Un'**associazione** è un **fatto che correla due o più entità**, creando un legame tra loro. Un'associazione **R(X, Y)** fra due collezioni di entità **X** e **Y** è un **insieme di istanze di associazione** tra gli elementi di X e di Y. Il prodotto cartesiano $X \times Y$ è detto **dominio dell'associazione**. Le associazioni possono avere poi anche delle proprie **proprietà**, e possono essere anche **ricorsive** (o riflessive).
N.B. Il concetto di associazione riprende in modo chiaro il concetto di **relazione matematica**.

Prendiamo per esempio due insiemi di entità: l'insieme Autori = {"Pirandello", "Leopardi"} e l'insieme Testi = {"Uno, Nessuno, Centomila", "La Ginestra"}. Un'istanza di associazione sarebbe ad esempio la coppia ("Pirandello", "Uno, Nessuno, Centomila"), mentre il dominio dell'associazione sarebbe l'insieme Autori × Testi = {("Pirandello", "Uno, Nessuno, Centomila"), ("Pirandello", "La Ginestra"),("Leopardi", "Uno, Nessuno, Centomila"), ("Leopardi", "La Ginestra")}.

Proprietà 1.5 – Vincolo di molteplicità

Un'associazione R(X,Y) è **univoca rispetto a X** se per ogni elemento x di X esiste al più un elemento y di Y associato a x. Se questo vincolo non è rispettato, allora l'associazione è **multivalore rispetto a X**.

- R(X,Y) è **(1:N)** se è *multivalore su X* e *univoca su Y*. Infatti, preso un elemento y di Y, esiste solo una corrispondenza in X, mentre preso un elemento x in X esistono più corrispondenze in Y.
- R(X,Y) è **(N:1)** se è *univoca su X* e *multivalore su Y*.
- R(X,Y) è **(N:M)** se è *multivalore su X* e *multivalore su Y*.
- R(X,Y) è **(1:1)** se è *univoca su X* e *univoca su Y*.

Ad esempio, l'associazione Frequenta(Studenti, Corsi) ha **cardinalità** (N:M). Infatti, uno studente può frequentare più corsi e un corso può essere frequentato da più studenti, mentre l'associazione NatoA(Persone, Citta) ha cardinalità (N:1), dato che una persona può nascere in una sola città, ma in una città possono nascere più persone.

Proprietà 1.6 – Vincolo di totalità

Un'associazione R(X,Y) è **totale su X** se per ogni elemento x di X esiste almeno un elemento y di Y associato a x. Se questo vincolo non è rispettato, allora l'associazione è **parziale su X**.

Ad esempio, l'associazione Frequenta(Studenti, Corsi) è totale su Corsi, dato che non può esistere un corso non frequentato da nessuno (verrebbe cancellato), ed è totale anche su Studenti (non esiste studente che non partecipi a nessun corso: in tal caso non sarebbe uno studente). Mentre l'associazione NatoA(Persone, Citta) è totale su Persone, dato che ogni persona è nata per forza in una città, ma parziale su Citta, dato che può esistere una città dove non è nato nessuno.

1.6.4 Realizzare una vera base di dati

In una base di dati sono quindi individuabili due componenti:

- **lo schema**, ossia la struttura di rappresentazione dei dati (*aspetto intensionale*), che rimane invariante nel tempo
- **l'istanza**, ossia i valori attuali che sono salvati nel DB (*aspetto estensionale*), che possono cambiare anche molto rapidamente

Per creare una base di dati è necessario costruire in totale 3 schemi:

- **Schema concettuale** (per la progettazione concettuale)
- **Schema logico** (per la progettazione logica)
- **Schema fisico** (per la progettazione fisica)

Gli schemi logici e fisici sono creati prendendo in input quello a loro precedente, mentre lo schema concettuale prende in input i dati analizzati. Nel prossimo capitolo vedremo come creare uno schema concettuale.

Capitolo 2

Progettazione concettuale

2.1 Il modello Entità-Relazione

Definizione 2.1 – Modello Entità-Relazione

Il **modello Entità-Relazione** (o **ER**) è il più diffuso modello concettuale.

Avevamo già accennato a questo modello nelle pagine precedenti, ma ora lo studieremo nel dettaglio. I suoi costrutti principali sono:

- **Entità** (ossia *classi di oggetti*)
- **Relazioni** (ossia le *associazioni*)
- **Attributi** (ossia le *proprietà* delle entità o delle relazioni)

N.B. Avevamo già parlato del concetto di entità nella Definizione 1.16, ma lì avevamo descritto un'entità come un singolo elemento nel mondo reale, mentre in questo modello parliamo di una classe di elementi. Per indicare un singolo elemento che fa parte di questa classe usiamo il nome di **istanza** (o **occorrenza**) di entità. Nello schema concettuale, però, non rappresentiamo mai le singole istanze ma sempre le entità. Esistono poi anche le occorrenze di associazione, ossia una n-tupla ordinata di occorrenze di entità legate tra loro da questa associazione.

Proprietà 2.1 – Convenzioni per i nomi di entità e associazioni

Il nome di un'**entità** deve essere *espressivo* e *al singolare*.
Il nome di un'associazione deve essere *espressivo*, *al singolare* e deve essere *un sostantivo* se possibile, altrimenti un *verbo* (all'infinito, e evitando i verbi avere, essere e tutti quelli modali)

Vediamo alcuni esempi di schemi ER molto basilari:



Figura 2.1: Un'associazione tra due entità

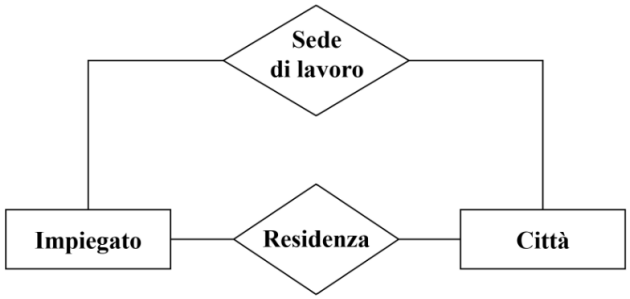


Figura 2.2: Due associazioni diverse tra due entità

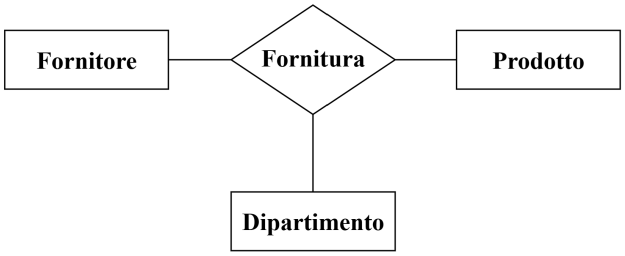


Figura 2.3: Un'associazione tra 3 entità

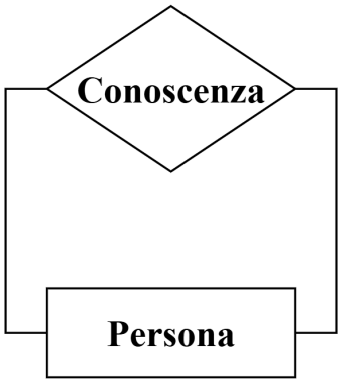


Figura 2.4: Un'associazione sulla stessa entità (**ricorsiva**)

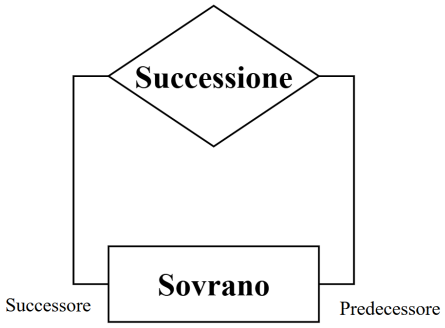


Figura 2.5: Un'associazione ricorsiva **asimmetrica**

Definizione 2.2 – Attributo

Un **attributo** è una proprietà di un'entità o di un'associazione.

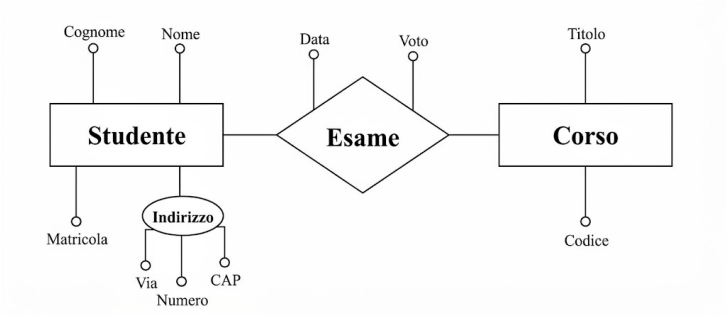
Negli schemi ER non sono scritti tutti gli attributi, ma solo **quelli di interesse**. Ad esempio, in uno schema ER riguardo un'università, per l'entità studenti non salveremo il colore degli occhi, perché non è un attributo che ci interessa.

Definizione 2.3 – Attributo composto

Un **attributo composto** è un raggruppamento di attributi di una stessa entità o associazione che sono collegati fra loro e possono essere usati come un attributo unico.

Ad esempio, per un'entità persona un attributo composto classico è il suo **indirizzo**, che si divide in attributi più semplici quali via, numero civico e CAP.

Vediamo con un esempio come si rappresentano graficamente gli attributi:



2.2 Cardinalità, identificatori e generalizzazione

2.2.1 Cardinalità

Definizione 2.4 – Cardinalità di una associazione

La **cardinalità di una associazione** è una **coppia di valori** che si associa ad ogni entità che partecipa all'associazione, che specifica il **minimo** e il **massimo** di occorrenze dell'altra (o altre) entità a cui può essere associata.

Ad esempio, nell'associazione Studente — Esame — Corso, per lo studente la cardinalità è (0,N), dato che uno studente può dare minimo 0 esami e massimo N esami (ossia un numero indeterminato di esami), mentre per l'entità Corso la cardinalità sarà ad esempio (1,25), ipotizzando che un corso abbia minimo un esame e massimo 25 (25 è un numero arbitrario che abbiamo deciso ora per questo esempio).

Proprietà 2.2 – Valori della cardinalità

In realtà, per la cardinalità si usano per convenzione solo **3 simboli**:

- Per la cardinalità **minima** si usano solo lo **0**, che indica che un'istanza potrebbe anche non partecipare a nessuna associazione, come uno studente che non ha ancora effettuato esami, o l'**1**, che invece indica che ogni istanza deve partecipare per forza ad almeno un'associazione, come l'entità Corso che deve per forza avere almeno un esame.
- Per la cardinalità **massima** si usano solo l'**1**, che indica che al massimo un'istanza può partecipare a un'associazione, e l'**N**, che indica che un'istanza può partecipare ad un numero indeterminato di istanze.

Proprietà 2.3 – Classificazione di associazioni

In base alle cardinalità **massime**, nelle associazioni a 2 possiamo distinguere tre tipi di associazioni:

- **uno a uno**
- **uno a molti**
- **molti a molti**

Vediamo alcuni esempi:



Figura 2.6: Associazione uno a uno

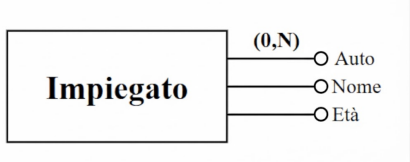


Figura 2.7: Associazioni uno a molti



Figura 2.8: Associazione molti a molti

N.B. La cardinalità riferita a un'entità la scriviamo sul ramo che collega questa entità all'associazione. Ad esempio, la cardinalità (0,1) nella prima figura indica che un Impiegato può dirigere uno o nessun progetto, mentre la cardinalità (1,1) vicino a Progetto indica che un progetto è diretto sempre da un impiegato.



È possibile associare la cardinalità anche agli attributi. In questo caso, abbiamo un impiegato che ha 0 o più automobili, e che ha uno e un solo nome e una e una sola età (la cardinalità uno a uno è omessa).

Inoltre, è bene sapere che è possibile incontrare anche la cardinalità non scritta in modo esplicito tramite parentesi ma tramite frecce. Questa rappresentazione la si vede nei diagrammi UML, che noi non usiamo, ma è importante conoscerla così da poterla comprendere quando la si incontra.



Ad esempio, con questa **doppia freccia e barretta** indichiamo una **cardinalità (0,N)** (la doppia freccia indica la cardinalità **N**, mentre la barretta indica la cardinalità **0**). Per una cardinalità **(1,1)**, invece avremmo usato una normale **freccia**. Troviamo quindi che la cardinalità (1,N) si fa con una doppia freccia normale, mentre la cardinalità (0,1) si fa con una freccia e barretta.

2.2.2 Identificatori

Definizione 2.5 – Identificatore di un'entità

Esistono poi gli **identificatori** delle entità, che permettono di identificare in modo univoco le istanze di un'entità. Sono costituiti da

- uno o più attributi dell'entità (**identificatori interni**)
- una o più entità esterne attraverso delle associazioni (**identificatori esterni**)

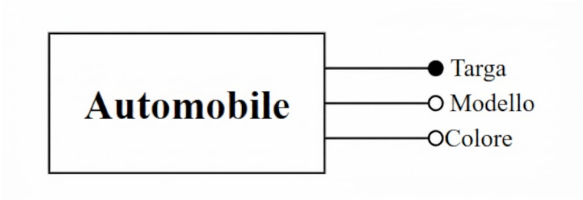


Figura 2.9: Singolo identificatore interno

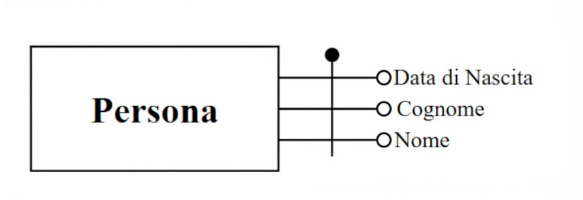


Figura 2.10: Più identificatori interni

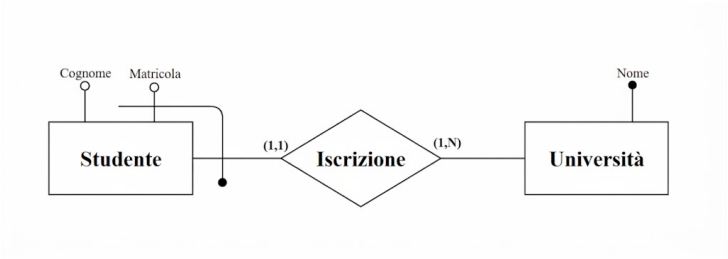


Figura 2.11: Identificatore esterno

In questo caso la matricola non è sufficiente per identificare in modo univoco uno studente: infatti, questo valore lo identifica solo in un'università, e non in generale nel mondo. Dunque è necessario unire questo identificatore esterno all'associazione Iscrizione tra Università e Studente. Studente in questo caso è un'**entità debole**, dato che la sua identificazione dipende da un'associazione con un'entità esterna.

Osservazione 2.1 – Osservazioni sugli identificatori

- Ogni entità deve avere **almeno un identificatore**
- Un identificatore esterno è possibile solo se l'entità partecipa all'associazione con cardinalità **(1,1)**
- Un'associazione **non ha bisogno di un identificatore**, dato che è già identificata univocamente dalla combinazione degli identificatori delle entità che lega

2.2.3 Generalizzazione

Definizione 2.6 – Generalizzazione

La **generalizzazione** è un costrutto che mette in relazione una o più entità $(E_1, \ldots, E_n)$ con un'entità genitore E che le comprende come casi particolari. Dunque $E_1, \ldots, E_n$ hanno tutte le stesse caratteristiche di E più altre caratteristiche personali.

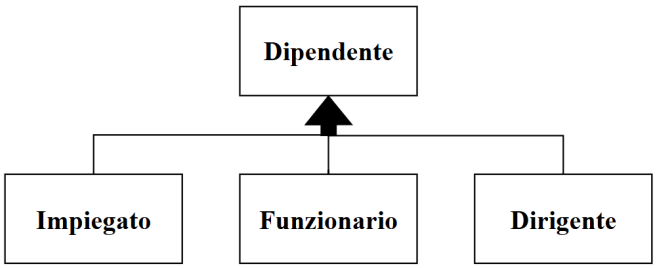


Figura 2.12: Esempio di generalizzazione con 3 specializzazioni

Proprietà 2.4 – Proprietà delle generalizzazioni

Presa un'entità padre E e delle entità figlie $E_1, \ldots, E_n$:

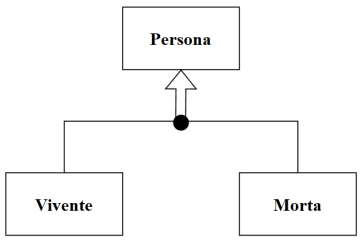
- ogni proprietà di E è **significativa** per $E_1, \ldots, E_n$, ossia viene **ereditata**
- ogni occorrenza di $E_1, \ldots, E_n$ è **un'occorrenza anche di E**
- ogni occorrenza di E può essere **un'occorrenza di 0 o più entità tra $E_1, \ldots, E_n$.**
- se una generalizzazione ha solo una figlia si parla di **sottoinsieme**

Proprietà 2.5 – Classificazione delle generalizzazioni

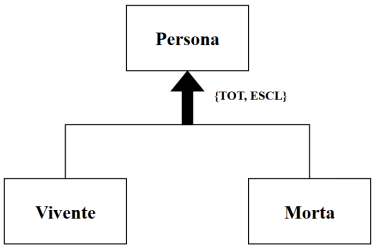
Una generalizzazione è:

- **totale** se ogni occorrenza del padre è un'occorrenza di almeno una delle figlie, altrimenti è **parziale**
- **esclusiva** se ogni occorrenza del padre è al più un'occorrenza di una delle figlie, altrimenti è **sovrapposta**

Queste proprietà ricordano molto le Proprietà 1.4 viste per le sottoclassi.



In base al tipo di generalizzazione cambia anche la rappresentazione. Presa infatti una generalizzazione totale e esclusiva, questa deve essere rappresentata da una **freccia vuota** (che indica che la generalizzazione è **totale**), e da un **pallino pieno** sotto la freccia (che indica che la generalizzazione è **esclusiva**).



Oltre alla rappresentazione vista prima con freccia vuota e pallino pieno, è possibile anche semplicemente scrivere, affianco la freccia che indica la generalizzazione, il tipo di generalizzazione tra parentesi graffe. In questo caso abbiamo scritto **{TOT,ESCL}** per indicare quindi una generalizzazione totale e esclusiva.

2.3 Esempio: Società di formazione - Progettazione Concettuale

Vediamo ora un esempio concreto.

"Si vuole realizzare una base di dati per una società che eroga corsi, di cui vogliamo rappresentare i dati dei partecipanti e dei docenti. Per gli studenti (circa 5000), identificati da un codice, si vuole memorizzare il codice fiscale, il cognome, l'età, il sesso, il luogo di nascita, il nome dei loro attuali datori di lavoro, i posti dove hanno lavorato in precedenza insieme al periodo, l'indirizzo, il numero di telefono, i corsi che hanno frequentato (che in totale sono circa 200) e il giudizio finale. Rappresentiamo poi anche i seminari che stanno attualmente frequentando e, per ogni giorno, i luoghi e le ore dove sono tenute le lezioni. I corsi hanno un codice, un titolo e possono avere varie edizioni con date di inizio e fine e numero di partecipanti. Se gli studenti sono liberi professionisti, vogliamo conoscere l'area di interesse e, se lo possiedono, il titolo. Per quelli che invece lavorano alle dipendenze di altri, vogliamo conoscere il livello e la posizione ricoperta. Per gli insegnanti (circa 300), rappresentiamo il cognome, l'età, il posto dove sono nati, il nome del corso che insegnano, quelli che hanno insegnato nel passato e quelli che possono insegnare. Rappresentiamo anche tutti i loro recapiti telefonici. I docenti possono essere dipendenti interni della società o collaboratori esterni."

Sembrano un sacco di informazioni, ma per questo è fondamentale schematizzare questo testo e prendere solo ciò di veramente interessante. Costruiamo innanzitutto un glossario dei termini per rendere il testo più chiaro:

Termine	Descrizione	Sinonimi	Collegamenti
Partecipante	Persona che partecipa ai corsi	Studente	Corso, Società
Docente	Docente dei corsi. Può essere esterno	Insegnante	Corso
Corso	Corso organizzato dalla società. Può avere più edizioni.	Seminario	Docente
Società	Ente presso cui i partecipanti lavorano o hanno lavorato	Posti	Partecipante

Riorganizziamo ora le frasi in base a ciò che dicono.

- Contesto **generale**: Si vuole creare una base di dati per una società che eroga corsi, di cui vogliamo sapere i dati dei partecipanti e dei docenti.
- Riguardo i **partecipanti**: I partecipanti (circa 5000) sono identificati da un codice. Di loro rappresentiamo il codice fiscale, cognome, età, sesso, città di nascita, i nomi dei loro attuali datori di lavoro e di quelli precedenti e le date di inizio e eventuale fine rapporto, le edizioni dei corsi che stanno frequentando e quelli che hanno frequentato, e la votazione generale in decimi.
- Riguardo i **datori di lavoro**: Dei datori di lavoro presenti e passati dei partecipanti rappresentiamo il nome, l'indirizzo e il numero di telefono.
- Riguardo i **corsi**: Dei corsi (circa 200) rappresentiamo il titolo, il codice e le varie edizioni con date di inizio e fine e, per ogni edizione, rappresentiamo il numero di partecipanti, i giorni della settimana, le aule e le ore dove sono tenute le lezioni.
- Riguardo i **tipi di partecipanti**: Dei partecipanti liberi professionisti rappresentiamo l'area di interesse e l'eventuale titolo personale. Per i dipendenti invece rappresentiamo il loro livello e la loro posizione ricoperta.
- Riguardo i **docenti**: Dei docenti (circa 300) rappresentiamo il cognome, l'età, la città di nascita, il numero di telefono, il titolo del corso che insegnano, di quelli che hanno insegnato e di quelli che possono insegnare. Questi possono essere dipendenti interni o collaboratori esterni.

Cerchiamo di capire ora quale costrutto del modello ER utilizzare in base alla situazione:

- Se ha proprietà significative e descrive oggetti autonomi, allora è un'**entità**
- Se è semplice e non ha proprietà ma è invece una proprietà di un oggetto autonomo, allora è un **attributo**
- Se mette in relazione due o più concetti è un'**associazione**
- Se è un caso particolare di un altro è una **specializzazione**

Definizione 2.7 – Strategie di progetto

Vi sono due possibili strategie adottabili:

- **top-down**: partiamo da pochi concetti generali e li raffiniamo man mano rendendoli sempre più precisi
- **bottom-up**: partiamo da concetti già dettagliati per poi generalizzare sempre di più

Di solito, però, si tende a seguire una strategia **mista**: si individuano i concetti, si crea uno scheletro e si divide il progetto in aree, dove ciascuna area è sviluppata nel dettaglio.

Quando si crea lo scheletro del progetto si devono individuare i concetti più importanti, che possono essere quelli più citati oppure quelli esplicitamente dichiarati come tali e li si organizza in uno schema concettuale. Questo schema deve essere corretto, completo, leggibile e minimale. Proviamo a modellare lo schema concettuale del contesto generale del database visto in precedenza:

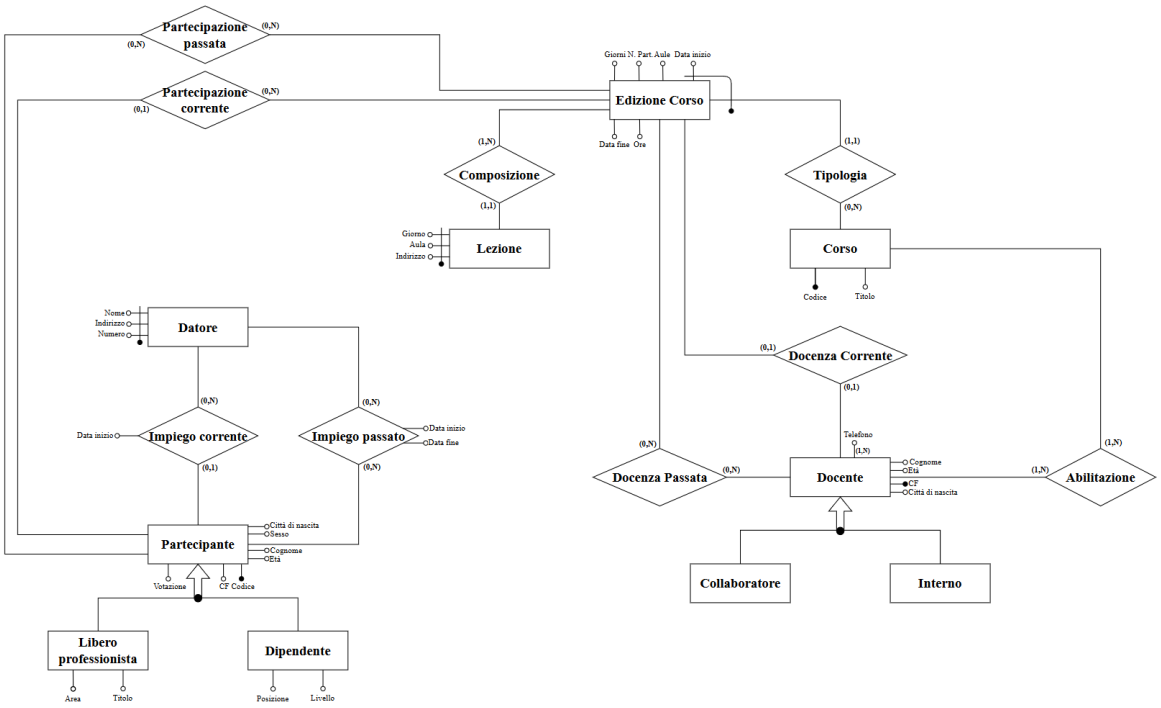


Figura 2.13: Schema ER completo

Questo schema può sembrare enorme, ma vediamo passo passo come è stato possibile realizzarlo velocemente e senza impazzire.

Partiamo con le informazioni generali: vogliamo creare una base di dati per una società che eroga corsi, e vogliamo i dati dei partecipanti e dei docenti. Già qui è facile trovare 3 entità principali: Corso, Docente e Partecipante.

Riguardo i partecipanti sappiamo che sono identificati dal codice e che hanno vari attributi (codice fiscale, cognome, età, sesso, città di nascita e votazione in decimi). Di loro vogliamo conoscere gli attuali datori di lavoro e quelli precedenti, con tanto di date di inizio e fine rapporto. Vogliamo poi sapere le edizioni dei corsi che stanno o hanno frequentato. Abbiamo quindi una serie di attributi per questi partecipanti, e troviamo due nuove entità: Datore e Edizione Corso. A questo punto lo schema avrebbe più o meno questo aspetto:

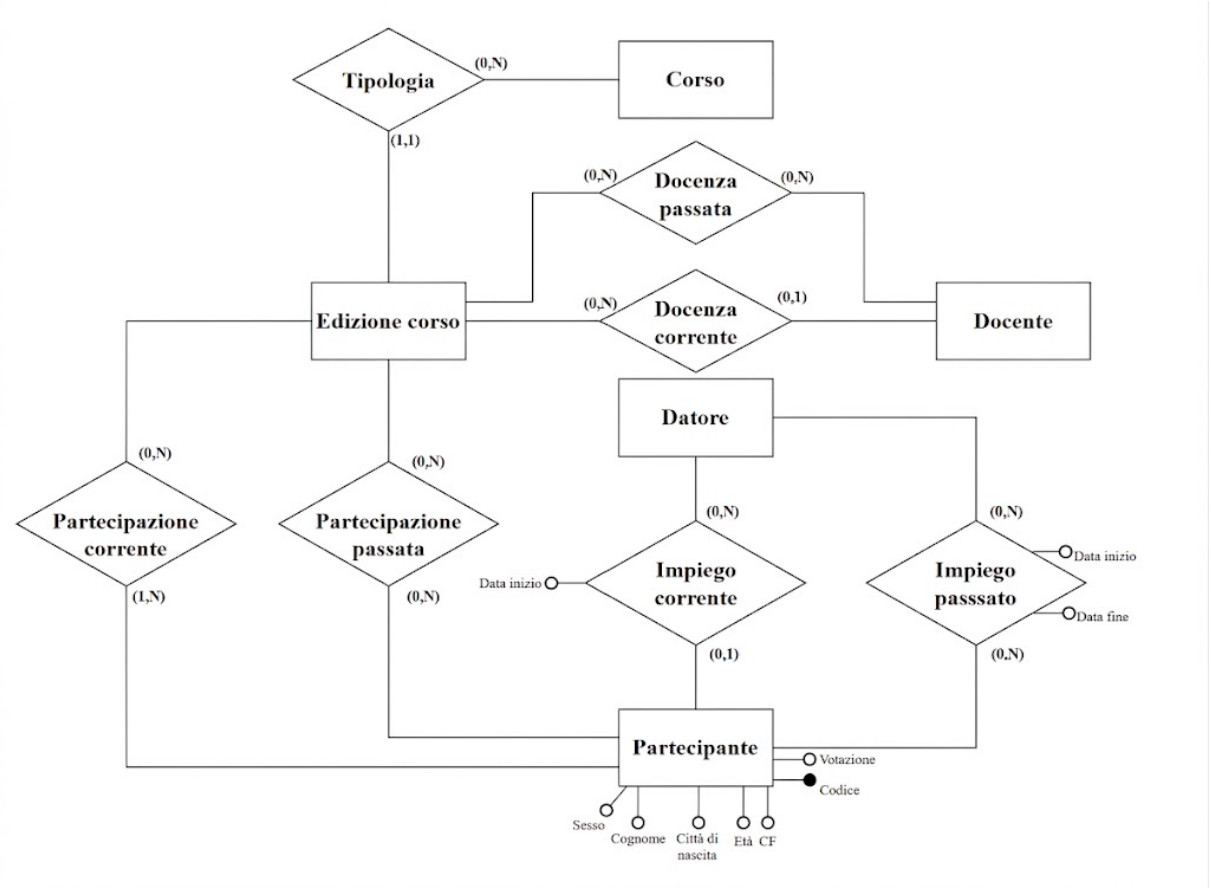


Figura 2.14: Schema ER parziale

Vediamo alcune cardinalità di questo schema parziale:

- Edizione Corso — **(1,1)** Tipologia **(0,N)** — Corso
  Un corso può avere più edizioni o nessuna, dato che un corso appena aggiunto non ha edizioni. Ma un'edizione di un corso si riferisce a uno e un solo corso sempre.
- Datore — **(0,N)** Impiego corrente **(0,1)** — Partecipante
  Un partecipante può avere nel proprio impiego 0 datori (se è un libero professionista) o un datore (se è un dipendente). E un datore può avere 0 dipendenti che attualmente lavorano per lui e svolgono corsi o più di 1. Nel caso ne avesse 0, questo datore potrebbe esser comunque salvato nel database nel caso un suo dipendente passato sia un partecipante.

- Edizione Corso — **(0,N)** Docenza corrente **(0,1)** — Docente
  Un'edizione di un corso può avere 0 docenti correnti (se non le sono ancora stati assegnati), o più di 1 (quando un corso è insegnato da più docenti). Ma un docente può insegnare nello stesso momento 0 o al massimo 1 corso.

Aggiungendo sempre più informazioni riguardo i corsi, i docenti, i partecipanti e i datori di lavoro si arriva pian piano ad avere uno schema ER come quello visto sopra.

Capitolo 3

Progettazione logica

3.1 Modello relazionale

Definizione 3.1 – Modello relazionale

Il **modello relazionale** è un modello logico basato sulla nozione matematica di relazione.
N.B. Queste relazioni si traducono in modo naturale in tabelle, e per questo al posto di dire tabelle diremo sempre relazioni.

Definizione 3.2 – Dominio

Un **dominio** D è un **insieme** (finito o infinito) di valori.
Presi $D_1, \ldots, D_k$ domini,

$$D_1 \times \ldots \times D_k = \{(v_1, \ldots, v_k) \mid v_1 \in D_1, \ldots, v_k \in D_k\}$$

Definizione 3.3 – Relazioni matematiche

Una relazione matematica è un qualsiasi **sottoinsieme** del prodotto cartesiano di uno o più domini. Una relazione sottoinsieme di un prodotto cartesiano tra k domini si dice di **grado** k. Gli elementi di una relazione sono **tuple** (o n-tuple o ennuple), e il numero di tuple in una relazione è la sua **cardinalità**. Ogni tupla di una relazione di grado k ha k **componenti ordinate**, ma non c'è ordinamento tra le tuple, e le tuple di una relazione sono **tutte distinte**. Le possibili relazioni matematiche di un prodotto cartesiano tra k domini saranno $\mathbf{2^n}$, dove n è il prodotto tra il numero degli elementi presenti nei diversi domini.

Vediamo un esempio. Presi i domini D_1 = {bianco,nero} e $D_2 = \{0, 1, 2\}$, $D_1 \times D_2 = \{(\text{bianco}, 0), (\text{bianco}, 1), (\text{bianco}, 2), (\text{nero}, 0), (\text{nero}, 1), (\text{nero}, 2)\}$. Una relazione di grado 2 e cardinalità 3 è ad esempio la relazione $r = \{(\text{bianco}, 0), (\text{nero}, 0), (\text{nero}, 2)\}$. $t = (\text{nero}, 0)$ è una delle tuple del prodotto cartesiano, e con `t[i]` indichiamo l'i-esimo elemento della tupla (`t[1] = nero`). Possiamo anche prendere più elementi di una stessa tupla: ad esempio `t[1,2] = (nero, 0)` prende il primo e il secondo elemento della tupla (nel nostro

esempio funziona poco, ma con n-ple ha più senso questa notazione). Se prendiamo un'altra tupla `t = (0,1,2,3)`, `t[1,3] = (0,2)`. Inoltre, con questi domini abbiamo 2^6 possibili relazioni.

Un altro esempio può essere anche la tabella

Paolo	Rossi	2	26.5
Lionel	Messi	10	24.5

In questo caso abbiamo 4-ple con 2 String, un Integer e un Real. Ma non avendo informazioni su cosa rappresentano questi dati la tabella ci è inutile. Per questo è necessario dare i nomi alle colonne:

Nome	**Cognome**	**Esami sostenuti**	**Media**
Paolo	Rossi	2	26.5
Lionel	Messi	10	24.5

Definizione 3.4 – Attributi

Un **attributo** è una coppia di valori: un **nome** A e un **dominio** $dom(A)$. Preso R un insieme di attributi, questo prende il nome di **schema di relazione**, e una ennupla è definibile come una *funzione* su R che ad ogni attributo associa un elemento del dominio. Se t è un'ennupla di R e A è un attributo in R, allora `t[A]` indica il valore assunto dalla funzione t in corrispondenza dell'attributo A.
N.B. Se una relazione è chiamata R e i suoi attributi sono $A_1, \ldots, A_k$, lo schema è indicato scrivendo $\boldsymbol{R(A_1, \ldots, A_k)}$.

Presa una tabella (o schema di relazione), le diverse righe sono le tuple della relazione, sono tutte diverse dalle altre ma sono omogenee (ossia provengono dallo stesso dominio). Le colonne sono i diversi domini e hanno dei nomi univoci all'interno della tabella, che prendono il nome di **nomi degli attributi**.
Lo schema di relazione è invariabile nel tempo, mentre le diverse istanze possono cambiare anche spesso.

Definizione 3.5 – Schema di base di dati

Uno **schema di base di dati** è un insieme di schemi di relazione con nomi diversi. Uno schema di base di dati relazionale è quindi un insieme $\{R_1, \ldots, R_n\}$ di schemi di relazione.

Citta	Regione	Popolazione
Roma	Lazio	2.748.109
Milano	Lombardia	1.354.196
Napoli	Campania	913.462
Torino	Piemonte	841.600

Ad esempio, la relazione qui a sinistra ha 3 attributi, e il proprio schema è Info_Citta(Citta, Regione, popolazione).

Nei modelli relazionali più recenti, le componenti di una relazione sono indicate dai nomi degli attributi e non dalla posizione. Scriviamo `t[`A_i`]` per indicare il valore dell'attributo di nome A_i, e non alla posizione A_i. Nell'esempio precedente per la seconda tupla avremmo che t[Regione] = Lombardia.

3.1.1 Chiavi

Definizione 3.6 – Chiave

Una **chiave** di una relazione (non è detto che ce ne sia solo una) è un attributo o un insieme di attributi che **identificano univocamente** una tupla. Questi valori permettono di mettere in relazione dati in tabelle diverse.
N.B. Esiste sempre una chiave, dato che R è un insieme e quindi non può avere dei duplicati.

Una relazione può avere anche più chiavi alternative. Di solito si sceglie quella più usata o composta da un numero minore di attributi che prende il nome di **chiave primaria**.

Un insieme di attributi X di una relazione R è una chiave di R se:

1) Per ogni istanza di R non esistono due tuple distinte t_1 e t_2 tali che $t_1[X] = t_2[X]$.
2) Nessun sottoinsieme proprio (ossia un sottoinsieme diverso da X stesso) di X soddisfa la prima condizione. La chiave è quindi un insieme minimale: funziona solo con tutti i suoi elementi, non se ne possono levare.

Il problema è che per trovare una chiave secondo questa definizione dovrei generare **TUTTE** le possibili istanze di R. Questo non solo è spesso dispendioso, ma è a volte **impossibile**, se si hanno domini infiniti. Bisognerà quindi trovare un modo diverso per stabilire quali attributi usare come chiavi.

3.1.2 Riferimenti per valore

Definizione 3.7 – Riferimento per valore

Il modello relazionale è basato su **riferimenti per valore**. Ciò significa che le correlazioni logiche tra tuple di relazioni diverse si stabiliscono confrontando i valori contenuti nei rispettivi attributi.
Dunque per collegare un'informazione memorizzata in una tabella a un'altra, si include una copia del valore della chiave primaria di una tabella all'interno dell'altra.

Consideriamo ad esempio tre schemi di relazione:

- `Studente`(Matricola, Nome, Cognome)
- `Corso`(CodiceCorso, NomeCorso, CFU)
- `Esame`(Matricola, CodiceCorso, Data, Voto)

Nella tabella `Esame` non è presente alcun puntatore diretto alle righe di `Studente` o `Corso`: il legame è garantito unicamente dalla presenza dei campi `Matricola` e `CodiceCorso`, i cui valori puntano concettualmente ai corrispondenti valori presenti nelle altre due relazioni.

3.1.3 NULL

Definizione 3.8 – NULL

NULL è un valore speciale usato nelle basi di dati per indicare una mancanza di informazione o il fatto che l'informazione non è applicabile.

Ad esempio, se abbiamo una tabella PERSONA(CF, Nome, Lavoro), l'attributo Lavoro può assumere valore NULL in una tupla se la persona associata a tale tupla non ha un lavoro.

Osservazione 3.1 – Osservazioni sul valore NULL

- NULL non appartiene a nessun dominio ma può **sostituire valori in tutti i domini**
- Due valori NULL, anche se nello stesso dominio, sono considerati **diversi**
- **NULL ≠ 0**
- Gli attributi chiave non devono **mai assumere valore NULL**

3.1.4 Vincoli

Definizione 3.9 – Vincoli di integrità

In una relazione possono essere definiti dei **vincoli di integrità** sui domini degli attributi. Questi vincoli possono essere:

- **vincoli di dominio**, ossia vincoli sul valore di un attributo (ad esempio VOTO <= 30 AND VOTO >= 0)
- **vincoli di tupla**, ossia vincoli sul valore di due o più attributi nella stessa tupla (ad esempio (VOTO = 30) OR (NOT LODE = 'Si'))
- **vincoli tra tuple della stessa relazione**, ossia vincoli sul valore di uno o più attributi nelle diverse tuple della stessa relazione (ad esempio MATRICOLA UNIQUE)
- **vincoli tra tuple di relazioni diverse**, ossia i vincoli delle foreign key (ad esempio Esame.Studente REFERENCES Studente.Matricola)

3.2 Passare dal modello concettuale a quello logico

Vogliamo ora tradurre lo schema concettuale in quello logico, rappresentandone i dati in modo corretto (e efficiente, ma ci concentreremo maggiormente sulla correttezza). Non è una traduzione semplice, perché non vi è una vera e propria procedura da seguire alla lettera. Questo perché vi sono diversi aspetti che non sono direttamente rappresentabili, e anche perché è necessario prestare attenzione alle prestazioni.

3.2.1 Ristrutturare lo schema ER

Per prima cosa dobbiamo semplificare la traduzione e ottimizzare le prestazioni (noi non lo faremo, dato che tralasceremo per questo corso l'aspetto dell'efficienza). Questo schema ER ristrutturato sarà meno concettuale perché si inizieranno a implementare più dettagli per facilitare il passaggio al modello logico.

La ristrutturazione si divide in 4 attività:

1) **Analisi delle ridondanze**: verifichiamo se nel nostro schema ER sono presenti oggetti non necessari
2) **Eliminazione delle generalizzazioni**: se possibile dobbiamo eliminare le generalizzazioni
3) **Partizionamento/Accorpamento di entità e relazioni**
4) **Scelta degli identificatori primari**

Analisi delle ridondanze

Definizione 3.10 – Ridondanza

Una **ridondanza** in uno schema ER è un'informazione significativa ma **derivabile dalle altre**.

Vi sono

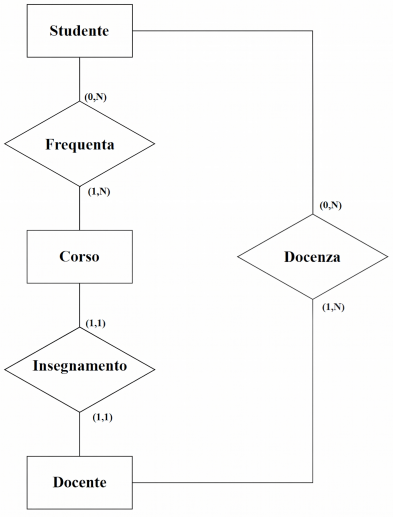
- **attributi derivabili**, che possono essere derivati dalla stessa entità, da attributi di altre entità o relazioni o da operazioni di conteggio sulle partecipazioni ad associazioni
- **relazioni derivabili** dalla composizione di altre relazioni in presenza di cicli

Un attributo **derivabile** lo vediamo quando abbiamo come attributi ad esempio l'importo lordo, l'IVA e l'importo netto: in questo caso uno tra i due importi è ridondante, dunque va eliminato.

Un attributo **derivabile da un'altra entità** ad esempio è l'attributo importo totale di un'entità Acquisto che è collegata tramite un'associazione a un'entità Prodotto che ha come unico attributo il prezzo. In questo caso importo totale è ridondante dato che possiamo calcolarlo sommando i prodotti tra il prezzo e la quantità di ogni prodotto.

Un attributo **derivabile per conteggio** ad esempio è l'attributo numero di prodotti di un'entità Acquisto che è collegata tramite un'associazione a un'entità Prodotto che ha come unico attributo il prezzo. In questo caso il numero di prodotti è ridondante dato che possiamo calcolarlo contando le istanze di associazione che riguardano quel determinato acquisto.

Una relazione **derivabile dovuta a un ciclo** ad esempio è l'associazione Docenza in questo schema ER. Infatti, non sta aggiungendo nulla di nuovo. Tramite le associazioni Frequenza e Insegnamento infatti possiamo capire se un insegnante insegna a uno studente o meno, rendendo ridondante l'associazione Docenza.



Eliminazione delle generalizzazioni

Il modello relazionale non permette la rappresentazione delle generalizzazioni, dato che le tabelle piatte utilizzate non permettono la definizione di sottotipi. Per questo dobbiamo eliminarle, e per farlo abbiamo 3 possibilità:

1) Accorpare le **specializzazioni nella generalizzazione**
2) Accorpare le **generalizzazioni nelle specializzazioni** (se possibile)
3) Sostituire le **generalizzazioni con le associazioni**

Conviene accorpare le **specializzazioni nella generalizzazione** quando le operazioni non fanno molta distinzione tra le diverse specializzazioni. Si creerà in questo caso un attributo `tipo` che specifica il tipo di istanza, così da mantenere distinte le istanze di due specializzazioni diverse. Se la generalizzazione è sovrapposta bisognerà prevedere per l'attributo `tipo` un valore preciso per quelle istanze che fanno parte di più specializzazioni. Ad esempio, se abbiamo due specializzazioni studente e lavoratore creeremo un tipo chiamato studente-lavoratore.
Lo svantaggio è che avremo molti valori nulli per gli attributi delle specializzazioni (la matricola è un valore che va salvato solo per gli studenti e gli studenti-lavoratori, ma per le istanze dei lavoratori il valore rimarrà vuoto).

Conviene accorpare le **generalizzazioni nelle specializzazioni** quando quasi tutte le operazioni avvengono in modo differente tra le diverse specializzazioni. Questo metodo, però, è applicabile solo se la generalizzazione è **totale**: infatti, se non lo fosse, avremmo delle istanze della generalizzazione che non potremmo rappresentare in alcun modo.

Conviene sostituire le **generalizzazioni con le associazioni** quando le operazioni avvengono in modo differenziato tra padre e figli. Ad esempio, se si ha l'entità Persona e le specializzazioni sono studente e lavoratore, si può sostituire la generalizzazione con delle associazioni quando le operazioni sul padre richiedono solo le informazioni generali dell'entità Persona mentre le operazioni sui figli richiedono le informazioni specifiche di quel figlio. Inoltre, è necessario che la generalizzazione sia **parziale**.
Lo svantaggio è che per un'operazione che ha bisogno delle informazioni sia del padre che del figlio il database non trova tutto in una riga ma deve andare a cercare in tabelle diverse queste informazioni.

Comunque, è possibile sempre trovare delle soluzioni *ibride*, e anzi nelle gerarchie a più livelli possono essere l'idea migliore.

Partizionamento/Accorpamento di entità e relazioni

Si vuole in questa fase ridurre gli accessi al database lavorando su come gli attributi sono interrogati, se insieme o separatamente. Esistono 3 modi per farlo:

1) **Partizionamento verticale di entità**: Supponiamo che la nostra applicazione acceda in modo separato ai dati anagrafici e a quelli lavorativi di una persona. In questo caso, ci conviene separare nel nostro database questi dati così da renderlo più efficiente.
2) **Partizionamento orizzontale di associazioni**: Supponiamo che la nostra applicazione acceda in modo separato alla composizione attuale della squadra rispetto a quella degli anni passati. Ci conviene creare due diverse associazioni per indicare la composizione attuale o passata per rendere lo schema più chiaro e le operazioni più efficienti.
3) **Eliminazione di attributi multivalore**: Dato che nel modello relazionale gli attributi multivalore non sono rappresentabili direttamente, ci conviene creare una nuova entità per rappresentarli.

Scelta degli identificatori primari

Vi sono diversi criteri per la scelta degli identificatori:

- l'attributo o gli attributi **non devono essere opzionali**
- scegliere tra i possibili identificatori **quello con meno attributi**
- preferire identificatori **interni**
- scegliere un attributo che renda le operazioni più ricorrenti **il più veloce possibile**

N.B. Ricordiamo che **ogni entità ha bisogno di un identificatore**.

3.2.2 Tradurre le associazioni

Definizione 3.11 – Vincolo di integrità referenziale

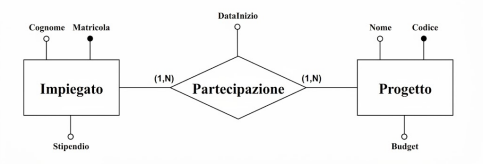
Per indicare che due tabelle/relazioni R_1 e R_2 (le vecchie entità o associazioni del modello ER) nel modello relazionale sono in relazione si utilizzano delle chiavi. Un **vincolo di integrità referenziale**, noto come **foreign key**, serve a vietare i riferimenti a chiavi che nella tabella di riferimento non esistono.

Il tipo di traduzione cambia in base al tipo di associazione:

- **Molti a molti**
- **Uno a molti**
- **Uno a uno**

Molti a molti

L'associazione diventa una **relazione** che ha come **chiave** le **chiavi primarie delle entità collegate**, e che include anche gli attributi dell'associazione.

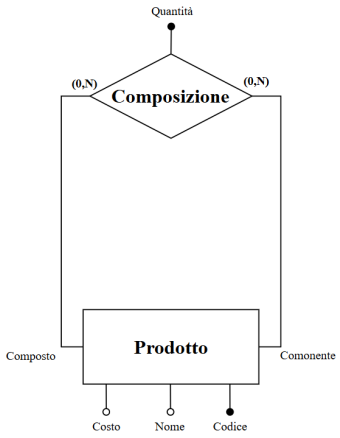


Ad esempio, quest'associazione si traduce nel modello relazionale in 3 tabelle:

- **Impiegato(Matricola, Cognome, Stipendio)**
- **Progetto(Codice, Nome, Budget)**
- **Partecipazione(Matricola, Codice, DataInizio)**

Vi sono poi dei vincoli di integrità referenziale tra Matricola in Partecipazione e la chiave di Impiegato e fra Codice in Partecipazione e la chiave di Progetto.

Spesso è consigliato rinominare gli attributi della chiave della relazione che rappresenta l'associazione. Ad esempio, qui piuttosto che scrivere Matricola e Codice in Partecipazione, era più chiaro scrivere Impiegato e Progetto, trasformando quindi Partecipazione in **Partecipazione(Impiegato, Progetto, DataInizio)**.



Quello che era un consiglio nelle relazioni molti a molti normali, diventa un obbligo in quelle ricorsive. In questo caso infatti la ridenominazione è fondamentale per definire il ruolo di ogni partecipante. Nel modello relazionale avremo quindi 2 tabelle:

- **Prodotto(Codice, Nome, Costo)**
- **Composizione(Composto, Componente, Quantita)**

Uno a molti

In questo caso è possibile trasformare l'associazione in una **relazione** con un procedimento identico a quello visto nelle associazioni molti a molti, ma la chiave non sarà costituita dalle chiavi primarie delle entità collegate, ma solo dalla **chiave primaria dell'entità con cardinalità massima 1**.

È però possibile anche un'altra soluzione. Al posto di trasformare l'associazione in una relazione, si possono **trasferire tutti gli attributi dell'associazione** all'entità con cardinalità massima 1 e aggiungere a questa entità come attributo anche la chiave primaria dell'altra entità (che diventa quindi la **foreign key**). Questo ha senso perché l'entità con cardinalità massima 1 può

essere associata a un solo elemento dell'altra entità, dunque da sola è in grado di identificare l'occorrenza di associazione.

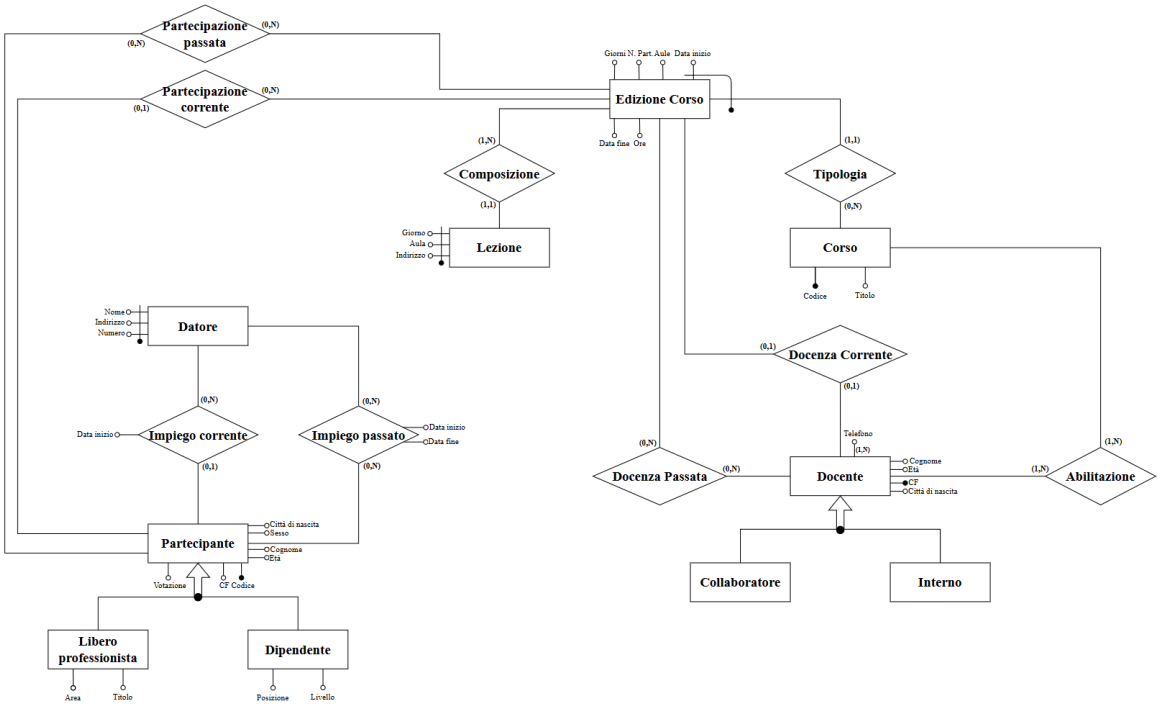
Uno a uno

In questo caso è preferibile **creare un'unica relazione** che unisca le due entità e l'associazione quando la partecipazione è obbligatoria per entrambe le entità o quando le due entità accedono quasi insieme alle informazioni. In questo caso la tabella risultante ha tutti gli attributi delle due entità e dell'associazione e ha come chiave primaria l'identificatore di una delle due entità. Altrimenti, si possono creare due relazioni, una per entità, e procedere allo stesso modo visto con le associazioni uno a molti, scegliendo una delle due relazioni a cui aggiungere la foreign key (se però nell'associazione vi è un'entità con cardinalità minima 0 e una con cardinalità minima 1, bisogna mettere la foreign key SEMPRE nell'entità con cardinalità minima 1). Si può, infine, anche scegliere di creare una terza relazione apposta per l'associazione, anche se è una soluzione raramente adottata.

3.3 Esempio: Società di formazione - Progettazione Logica

3.3.1 Ristrutturare lo schema ER

Riprendiamo ora l'esempio della società di formazione di cui abbiamo creato lo schema ER nel secondo capitolo. Eravamo arrivati a questo schema ER:



Per passare dal modello concettuale a quello logico dobbiamo quindi ora ristrutturare questo schema ER.

Analisi delle ridondanze

L'unica ridondanza evidente in questo schema è l'attributo N. Part. presente nell'entità Edizione Corso. Infatti, questo potrebbe essere ricavato contando le occorrenze dell'associazione Partecipazione Corrente. Tuttavia, possiamo assumere che questa informazione sia utilizzata spesso nella base di dati, il che renderebbe il conteggio molto inefficiente. Dunque lasciamo tutto com'è senza effettuare modifiche per ora, facendo attenzione in futuro ad aggiornare questa informazione tramite *vincoli dinamici* (ossia dovremo fare in modo che questa informazione si aggiorni da sola ogni volta che un partecipante si iscrive a un'edizione di un corso).

Eliminazione delle generalizzazioni

Per quanto riguarda la generalizzazione Partecipante — Professionista — Dipendente, dato che queste non hanno associazioni specifiche e dato che dividere la superclasse nelle due sottoclassi richiederebbe di duplicare l'associazione Impiego Passato (infatti, anche un libero professionista può in passato aver lavorato come dipendente), decidiamo di accorpare le sottoclassi nella superclasse. Modifichiamo quindi quest'entità aggiungendole 4 attributi opzionali provenienti dalle sue sottoclassi, e aggiungendo anche un nuovo attributo TipoP che indica il tipo di partecipante (professionista o dipendente).

Per quanto riguarda invece la generalizzazione Docente — Collaboratore — Interno, dato che queste non hanno attributi propri e non hanno associazioni specifiche ha anche qui molto senso accorpare le sottoclassi nella superclasse. Modifichiamo quindi quest'entità, aggiungendole un attributo TipoD che indica il tipo di docente (collaboratore o interno).

Partizionamento/accorpamento di entità e relazioni

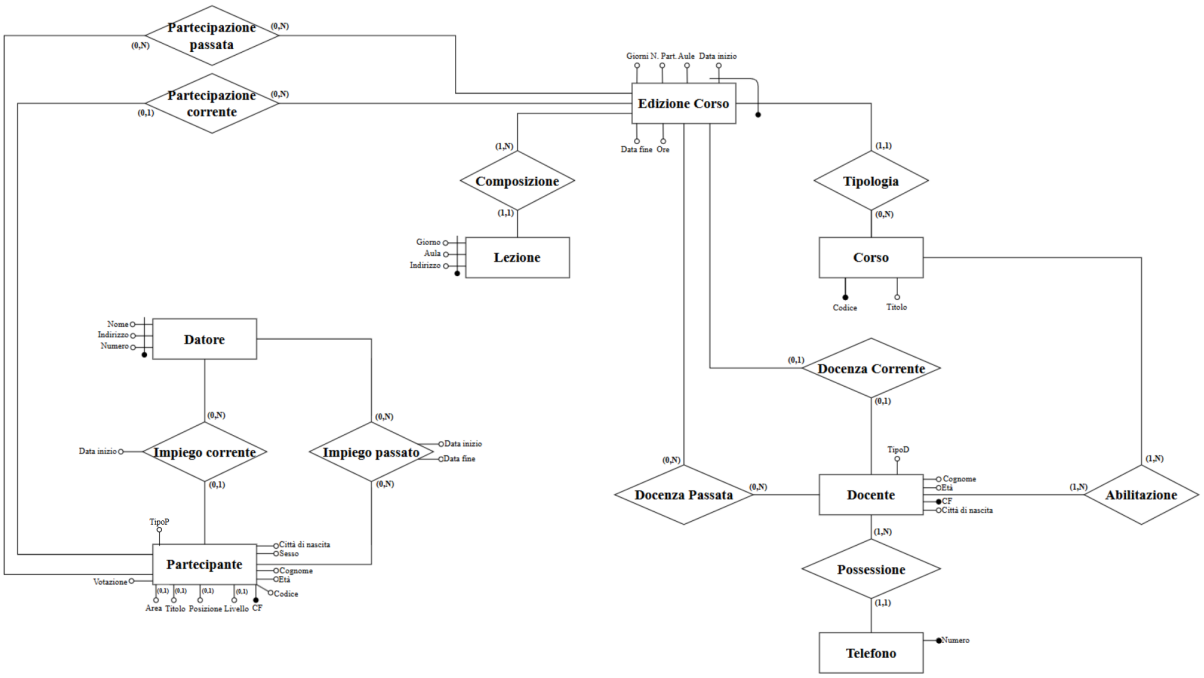
Non è necessario nessun partizionamento/accorpamento di entità e relazioni in questo caso, ma abbiamo un attributo multivalore da analizzare. Si tratta dell'attributo Telefono dell'entità Docente. Creiamo quindi una nuova entità Telefono che è collegata all'entità Docente tramite una relazione uno a molti (un telefono ha solo un docente, ma un docente ha più telefoni).

Scelta degli identificatori primari

L'unico dubbio riguarda l'entità Partecipante. Questa ha infatti due valori univoci: CF e Codice. Nello schema ER avevamo scelto Codice come identificatore primario, ma scegliamo di passare a CF.

Schema ER finale

Lo schema ER finale si presenta quindi in questo modo:



3.3.2 Passaggio allo schema logico

Una volta ristrutturato lo schema ER non ci resta che tradurlo in uno schema logico.

Per questa traduzione, adottiamo fin da subito delle convenzioni che saranno utili per la successiva implementazione in SQL:

- Nomi delle tabelle in maiuscolo
- Niente spazi o punteggiatura (nei nomi delle tabelle al posto degli spazi useremo gli _ mentre nei nomi degli attributi uniremo i termini)
- Le chiavi usate per tradurre le associazioni possono essere rinominate come già visto per rendere la loro funzione più chiara

Traduzione delle entità

La traduzione delle entità non crea problemi: ogni entità diventa una tabella (che potrebbe *non essere ancora nella sua forma finale*) e gli identificatori primari vengono sottolineati per rappresentare la chiave primaria scelta. Creiamo quindi una serie di nuove tabelle:

- **PARTECIPANTE(CF, Codice, Cognome, Eta, Sesso, CittaNascita, Area, Titolo, Posizione, Livello, TipoP)**

- **DATORE(Nome, Indirizzo, Numero)**
- **DOCENTE(CF, Eta, CittaNascita, Cognome, TipoD)**
- **TELEFONO(Numero)**
- **CORSO(Codice, Titolo)**
- **EDIZIONE_CORSO(CodiceCorso, DataInizio, DataFine, NPart)** (CodiceCorso è la chiave che collega EDIZIONE_CORSO a CORSO, ed è necessaria perché nello schema ER l'entità Edizione Corso è un'entità debole, con un identificatore esterno)
- **LEZIONE(Giorno, Aula, Orario)**

Traduzione delle associazioni

Le traduzioni delle associazioni sono un po' più complesse. Vediamole una alla volta.

L'associazione IMPIEGO_PASSATO tra PARTECIPANTE e DATORE è un'associazione molti a molti, dunque è necessario creare una relazione apposta per lei, ossia **IMPIEGO_PASSATO(CFPart, NomeDatore, DataInizio, DataFine)**.

L'associazione IMPIEGO_CORRENTE è invece un'associazione uno a molti, dove la cardinalità massima 1 è riferita alla relazione PARTECIPANTE. Dunque possiamo modificare questa relazione aggiungendo la chiave di DATORE e l'attributo DataInizio. Dunque PARTECIPANTE diventa **PARTECIPANTE(CF, Codice, Cognome, Eta, Sesso, CittaNascita, Area, Titolo, Posizione, Livello, TipoP, NomeDatore, DataInizio)**.

L'associazione PARTECIPAZIONE_PASSATA è un'altra associazione molti a molti, dunque creiamo anche per lei una relazione apposta, ossia **PARTECIPAZIONE_PASSATA(CFPart, CodiceCorso, DataInizio)**.

L'associazione PARTECIPAZIONE_CORRENTE è un'associazione uno a molti, dove la cardinalità massima 1 è riferita alla relazione PARTECIPANTE. Questa volta scegliamo di creare una relazione a parte per questa associazione, così da non appesantire la tabella PARTECIPANTE che ha già incorporato l'associazione IMPIEGO_CORRENTE. Dunque creiamo la nuova tabella **PARTECIPAZIONE_CORRENTE(CFPart, CodiceCorso, DataInizio)** che ha come chiave solo la chiave dell'entità con cardinalità massima 1, ossia la chiave di PARTECIPANTE.

L'associazione COMPOSIZIONE è anche questa un'associazione uno a molti, dove la cardinalità massima 1 è riferita alla relazione LEZIONE. Possiamo quindi incorporare l'associazione COMPOSIZIONE in essa aggiungendo la chiave di EDIZIONE_CORSO, ossia CodiceCorso e DataInizio. Dunque LEZIONE diventa **LEZIONE(Giorno, Aula, Orario, CodiceCorso, DataInizio)**.

L'associazione TIPOLOGIA è stata già tradotta quando abbiamo tradotto l'entità debole EDIZIONE_CORSO, dunque non dobbiamo fare nulla.

L'associazione POSSESSIONE è un'associazione uno a molti. In questo caso, dato che TELEFONO (l'entità con cardinalità massima 1) ha solo un attributo, decidiamo di incorporare l'associazione in questa relazione, aggiungendo come foreign key la chiave di DOCENTE. Dunque TELEFONO diventa **TELEFONO(Numero, CFDocente)**.

L'associazione ABILITAZIONE è un'associazione molti a molti, dunque creiamo in questo caso una nuova relazione con chiave le due chiavi delle due entità collegate (DOCENTE e CORSO). Dunque creiamo la tabella **ABILITAZIONE(CFDocente, CodiceCorso)**.

L'associazione DOCENZA_PASSATA è anche questa un'associazione molti a molti. Dunque creiamo anche qui una nuova relazione **DOCENZA_PASSATA(CFDocente, CodiceCorso, DataInizio)**.

L'associazione DOCENZA_CORRENTE è invece un'associazione uno a uno. Tra le tante possibilità che abbiamo scegliamo in questo caso di incorporare questa associazione nella relazione EDIZIONE_CORSO aggiungendo come foreign key la chiave di DOCENTE. Dunque la relazione diventa **EDIZIONE_CORSO(CodiceCorso, DataInizio, DataFine, NPart, CFDocente)**.

Riepilogo

Riepiloghiamo quindi tutte le relazioni che costituiscono il nostro nuovo schema logico:

- **PARTECIPANTE(CF, Codice, Cognome, Eta, Sesso, CittaNascita, Area, Titolo, Posizione, Livello, TipoP, NomeDatore, DataInizio)**
- **DATORE(Nome, Indirizzo, Numero)**
- **DOCENTE(CF, Eta, CittaNascita, Cognome, TipoD)**
- **TELEFONO(Numero, CFDocente)**
- **CORSO(Codice, Titolo)**
- **EDIZIONE_CORSO(CodiceCorso, DataInizio, DataFine, NPart, CFDocente)**
- **LEZIONE(Giorno, Aula, Orario, CodiceCorso, DataInizio)**
- **IMPIEGO_PASSATO(CFPart, NomeDatore, DataInizio, DataFine)**
- **PARTECIPAZIONE_PASSATA(CFPart, CodiceCorso, DataInizio)**
- **PARTECIPAZIONE_CORRENTE(CFPart, CodiceCorso, DataInizio)**
- **ABILITAZIONE(CFDocente, CodiceCorso)**
- **DOCENZA_PASSATA(CFDocente, CodiceCorso, DataInizio)**

Capitolo 4

SQL

4.1 Cos'è SQL?

Definizione 4.1 – SQL

Il linguaggio **SQL** (Structured Query Language, ossia Linguaggio di Interrogazione Strutturato) è il linguaggio standard dei database relazionali usato per **definire** le basi di dati, **manipolarle** e **interrogarle** tramite **query** (interrogazioni).
Questo linguaggio nasce nel 1974 con SEQUEL, diventando uno standard di fatto dal 1983, ma formalmente solo dal 1986.

Il linguaggio SQL si può dividere in diversi sottolinguaggi con pacchetti di comandi specifici per alcune operazioni:

- **DDL** (Data Definition Language), che serve per **creare e strutturare** le tabelle e gli oggetti del database
- **DQL** (Data Query Language), che serve per **interrogare** i dati
- **DML** (Data Manipulation Language), che serve per **manipolare** i dati nelle tabelle
- **DCL** (Data Control Language), che serve per gestire i **permessi** di accesso ai dati

4.2 DDL

Una volta tradotta la nostra base di dati in schema logico, vediamo come creare la nostra base di dati in SQL, tramite le istruzioni del sottolinguaggio **DDL**.

Domini

Definizione 4.2 – Domini

Dentro SQL esistono due tipi di domini: **domini elementari**, che sono predefiniti, o **domini definiti dall'utente**.

I domini elementari sono:

- **Carattere** (singoli caratteri o stringhe)
- **Bit** (singoli bit o stringhe)
- **Numerici**
- **Data, ora, intervalli di tempo**
- **Boolean** (introdotto dal 1999)
- **BLOB, CLOB** (introdotti dal 1999, utilizzati per grandi immagini e testi; non possono essere oggetto di query)

Ogni tipo di dati elementare è supportato da funzioni specifiche, come `year()`.

Per creare un dominio esiste la funzione **`CREATE DOMAIN`**. Se ad esempio vogliamo creare il dominio dei voti, dobbiamo scrivere

```
CREATE DOMAIN Voti
    AS SMALLINT DEFAULT NULL
    CHECK(Value >= 18 AND Value <= 30)
```

In questo modo stiamo creando un dominio che parte come uno SMALLINT (dominio elementare di interi), con default NULL e che può avere solo valori compresi tra 18 e 30. Sono molto utili se abbiamo più volte definito lo stesso vincolo di integrità: possiamo infatti creare un singolo dominio con questo vincolo così da definirlo una volta sola.

Tabelle

Per creare una nuova tabella si utilizza il comando **`CREATE TABLE`**, che definisce lo schema della tabella e ne crea un'istanza vuota. Ci permette di specificarne attributi, domini e vincoli di integrità. Se ad esempio vogliamo creare una relazione Impiegato con i suoi vari attributi, dobbiamo scrivere

```
CREATE TABLE Impiegato(
    Matricola VARCHAR(6) PRIMARY KEY,
    CF VARCHAR(16) UNIQUE,
    Nome VARCHAR(20) NOT NULL,
    Cognome VARCHAR(20) NOT NULL,
    Dipart VARCHAR(15),
    FOREIGN KEY (Dipart) REFERENCES Dipartimento(Codice),
    Stipendio NUMERIC(9) DEFAULT 0,
    UNIQUE(Cognome, Nome)
)
```

Vediamo cosa significano queste diverse parole chiave:

- **`PRIMARY KEY`** indica che l'attributo Matricola è la nostra chiave primaria
- **`NOT NULL`** indica che quell'attributo non può mai essere NULL (**N.B.** In SQL di default un campo può essere nullo)
- **`VARCHAR(n)`** indica una stringa che può avere al massimo n caratteri.
- **`UNIQUE`** indica che il CF deve essere unico tra tutte le tuple della relazione Impiegato
- **`FOREIGN KEY`** + **`REFERENCES`** indica che Dipart è una foreign key di un'altra tabella chiamata Dipartimento, la cui chiave è Codice
- **`DEFAULT`** indica che lo stipendio di ogni nuovo impiegato si trova come default a 0

Sia `UNIQUE` che `PRIMARY KEY` che `FOREIGN KEY` possono racchiudere più campi insieme (come già visto nell'istruzione `UNIQUE(Cognome, Nome)`, che rende unica la coppia di attributi (Cognome, Nome), e non i singoli campi). Questo è fondamentale quando abbiamo una chiave primaria / foreign key composta da più attributi (infatti non si può scrivere più volte `PRIMARY KEY` affianco ai diversi attributi, ma bisogna scrivere a parte `PRIMARY KEY(att1, att2, ...)`).

Vincoli

Abbiamo già visto come inserire dei vincoli a un campo, ad esempio scrivendo `NOT NULL` o `UNIQUE`. È possibile specificare delle **reazioni alla violazione di vincoli d'integrità** che eseguono delle istruzioni specifiche quando si prova a violare un vincolo (se queste non sono presenti le azioni che violerebbero i vincoli semplicemente non vengono eseguite). Nell'esempio di prima avevamo una foreign key `Dipart` in riferimento a una tupla in `Dipartimento` (chiamiamo la tabella Impiegato **slave** mentre la tabella Dipartimento **master**). Normalmente, non potremmo eliminare una riga in Dipartimento prima di aver eliminato tutte le references a quella riga in Impiegato. È possibile però definire una reazione alla violazione di questo vincolo di integrità che in automatico elimina tutte le righe di Impiegato che fanno riferimento a quel dipartimento ed elimina anche la riga in Dipartimento. Per fare ciò scriviamo **`ON DELETE CASCADE`** (**N.B.** Questo funziona perfettamente, ma è MOLTO rischioso, dato che con un'operazione eliminiamo più righe in tabelle diverse). Se invece non vogliamo eliminare le righe di Impiegato ma semplicemente impostare a NULL (se ammesso) il valore della foreign key Dipart, possiamo scrivere **`ON DELETE SET NULL`** (non serve aggiungere `CASCADE`: `CASCADE`, `SET NULL` e `SET DEFAULT` sono reazioni alternative tra loro, e qualunque sia quella scelta il DBMS la applica automaticamente a tutte le righe di Impiegato che fanno riferimento al dipartimento eliminato).
E se invece un dipartimento non viene eliminato ma modificato? Esiste l'operazione **`ON UPDATE CASCADE`**, che aggiorna a cascata tutte le reference a quel dipartimento in Impiegato.
Dunque, se vogliamo che una volta eliminato un dipartimento mettiamo a NULL le references in Impiegato, e una volta modificato invece a cascata modifichiamo tutto in Impiegato, dobbiamo scrivere `FOREIGN KEY (Dipart) REFERENCES Dipartimento(Codice) ON DELETE SET NULL ON UPDATE CASCADE`.

Per aggiungere poi dei veri vincoli, oltre a NOT NULL o UNIQUE, dobbiamo usare la parola chiave **CHECK**. Ad esempio, se vogliamo imporre che lo Stipendio sia sempre maggiore o uguale a 0, dobbiamo scrivere `Stipendio NUMERIC(9) DEFAULT 0 CHECK(Stipendio >= 0)`, oppure possiamo aggiungere la sola condizione in una riga a parte scrivendo semplicemente `CHECK(Stipendio >= 0)`. Questa seconda scelta è obbligatoria se abbiamo un vincolo che coinvolge più attributi. Ad esempio, se abbiamo due attributi `DataInizio` e `DataFine` e vogliamo porre `DataInizio <= DataFine`, dobbiamo scrivere **`CHECK(DataInizio <= DataFine)`** in una riga a parte. Inoltre, essendo `CHECK` un tipo particolare di CONSTRAINT, possiamo usare proprio la parola chiave **`CONSTRAINT`** che non solo definisce ma **nomina** il vincolo così che in futuro possa essere modificato o eliminato. Ad esempio, possiamo scrivere `CONSTRAINT CHK_StipendioNonNegativo CHECK (Stipendio >= 0)` per creare un nuovo CONSTRAINT su Stipendio chiamato `CHK_StipendioNonNegativo` che controlli che ogni Stipendio sia ≥ 0. Per modificare/eliminare poi un vincolo si usano le parole chiave **`ALTER TABLE`**. Per eliminare un vincolo usiamo poi la parola chiave **`DROP`**. Ad esempio, se vogliamo eliminare il vincolo `CHK_StipendioNonNegativo` dovremo scrivere `ALTER TABLE Impiegato DROP CONSTRAINT CHK_StipendioNonNegativo`. Se volessimo invece modificare il vincolo, dovremmo prima eliminarlo con **`DROP`** e poi ricrearlo con la parola chiave **`ADD`**. Ad esempio, se volessimo imporre che lo stipendio non sia più solo non negativo ma sempre positivo, scriveremmo `ALTER TABLE Impiegato DROP CONSTRAINT CHK_StipendioPositivo` e poi `ALTER TABLE Impiegato ADD CONSTRAINT CHK_StipendiPositivo CHECK (Stipendio > 0)`.

Indici

Ogni volta che definiamo la chiave primaria, il DBMS automaticamente genera un indice sulla chiave primaria. Questo indice, che si trova a livello fisico, consente di accedere più velocemente al campo a cui si riferisce. Per capire perché sono importanti vediamo un esempio. Ipotizziamo di voler recuperare i dati di uno studente con matricola 2238123, con gli indici fisici possiamo fare una ricerca sul file che contiene solo gli indici. Questo rende la ricerca più veloce rispetto a una ricerca direttamente sulle matricole perché il file contenente le matricole è decisamente più grande del file con gli indici (dato che contiene anche tutte le informazioni riguardanti gli studenti). Dunque la ricerca sul file degli indici è una ricerca su un numero di blocchi in memoria decisamente minore rispetto a una ricerca sul file con tutti gli studenti, il che la rende una ricerca più veloce.
Oltre agli indici sulle chiavi primarie è possibile creare anche un nuovo indice con il comando **`CREATE INDEX`**, scrivendo semplicemente CREATE INDEX nome_indice ON nome_tabella (colonna1, colonna2, ...) per creare un indice sulle colonne di quella tabella.
N.B. È possibile anche creare degli indici unici tramite la parola chiave `UNIQUE` posta tra `CREATE`

e `INDEX`, che impedisce l'inserimento di valori duplicati nella colonna indicizzata. Nonostante ciò, è molto più chiaro rendere unici i campi e non gli indici (comunque il risultato è lo stesso).

Tra i vantaggi degli indici troviamo:

- **Velocizzano le ricerche filtrate** (operazioni che vedremo in seguito)
- **Ottimizzano le relazioni e i collegamenti**: è fondamentale quindi avere indicizzate le colonne che collegano una tabella a un'altra
- **Rendono più rapido l'ordinamento e il raggruppamento** (operazioni che vedremo in seguito)

Tra gli svantaggi invece troviamo:

- **Rallentano le operazioni di scrittura** (operazioni che vedremo in seguito), dato che ogni volta che si modifica, inserisce o cancella un record il database non solo deve aggiornare la tabella base ma anche il file degli indici
- **Consumano più spazio sul disco**, essendo questi indici fisici

È consigliato quindi usare indici solo sulle colonne usate frequentemente nelle operazioni di ricerca filtrata e sulle colonne che collegano più tabelle.

Modifiche agli schemi

Abbiamo già visto che per modificare un vincolo `CONSTRAINT CHECK` si può usare il comando **`ALTER TABLE`**. Questo è uno dei diversi comandi che permettono di **modificare gli schemi**. Andrebbero evitate queste modifiche una volta già creata la base di dati, ma possono capitare situazioni particolari dove queste operazioni possono rivelarsi utili. Oltre a `ALTER TABLE` esistono:

- **`ALTER DOMAIN`**
- **`DROP DOMAIN`**
- **`DROP TABLE`**

Con il comando **`ALTER DOMAIN`** possiamo impostare (`SET DEFAULT`) o rimuovere (`DROP DEFAULT`) un valore di default di un dominio. È possibile modificare i vincoli di nullabilità, scrivendo `SET NOT NULL` per impostare un campo NOT NULL, o scrivendo `DROP NOT NULL` per eliminare un vincolo NOT NULL a un campo. Si possono poi anche aggiungere (`ADD domain_constraint`) o eliminare (`DROP CONSTRAINT domain_constraint`) un vincolo di dominio. Infine, si può anche rinominare un dominio con il comando `RENAME TO`.
Se ad esempio abbiamo un dominio Voti, e vogliamo mettere come default il valore 0, possiamo scrivere `ALTER DOMAIN Voti SET DEFAULT 0`. Se invece

vogliamo aggiungere il vincolo che ogni voto sia $\leq$ 30, possiamo scrivere `ALTER DOMAIN Voti ADD CONSTRAINT CHK_VotiMassimo30 CHECK(Value <= 30)`.

Con il comando **`ALTER TABLE`**, oltre che modificare/eliminare vincoli già esistenti sui campi, possiamo creare dei campi totalmente nuovi a una tabella, scrivendo `ADD nome_colonna tipo_dato`. Si può poi anche modificare il tipo di un campo di una tabella scrivendo `MODIFY nome_colonna nome_tipo`. Si può anche eliminare (`DROP`) o rinominare `RENAME TO` un campo di una tabella. Se rinominiamo un campo dobbiamo però stare attenti a eventuali foreign key su tale campo. Inoltre si possono anche aggiungere dei nuovi CONSTRAINT, oltre che modificarli/eliminarli, scrivendo `ADD CONSTRAINT`. Si può infine rinominare una tabella sempre con il comando `RENAME TO` (anche qui dovremo stare attenti per eventuali riferimenti esterni).
Se ad esempio vogliamo aggiungere a una tabella Matricola il campo Nome di tipo `VARCHAR(16)`, scriveremo `ALTER TABLE Matricola ADD Nome VARCHAR(16)`. Se invece vogliamo rinominare una tabella Voti in VotiStudenti scriveremo `ALTER TABLE Voti RENAME TO VotiStudenti`. Se infine vogliamo aggiungere un vincolo di foreign key su un campo Matricola di una tabella Voti che fa riferimento a un campo Matricola di una tabella Studenti, possiamo scrivere `ALTER TABLE Voti ADD CONSTRAINT FK_MatricolaForeignKey FOREIGN KEY (Matricola) REFERENCES Studenti(Matricola)`.

Con il comando **`DROP`** è poi possibile eliminare:

- un **dominio**
- una **tabella** (cancellandone sia i dati che lo schema, se si vuole cancellare solo le righe bisogna usare o il comando **`TRUNCATE`**, che cancella tutte le righe, o il comando **`DELETE`** che elimina tutte le righe che seguono determinate condizioni da noi imposte)
- un **indice**
- una **vista**

View

Il comando **`CREATE VIEW`** crea una **tabella virtuale** con i dati ottenuti da una query sulla tabella originale. Crea dunque un nuovo oggetto strutturale del database, e poiché non manipola i record ma ne seleziona solo alcuni da mettere in una nuova tabella virtuale, rientra perfettamente nei comandi di definizione. Possiamo ad esempio scrivere `CREATE VIEW nome_vista AS risultato_query` e creare una nuova vista che può essere interrogata come una tabella normale, e che viene aggiornata automaticamente quando i dati vengono aggiornati (al contrario delle tabelle create con `SELECT` che, come vedremo, crea delle tabelle statiche che non si aggiornano). Torneremo a parlare di viste in modo più specifico quando tratteremo il linguaggio di query DQL, essendo queste per natura strettamente collegate alle query.

4.3 Algebra relazionale e DQL

4.3.1 Introduzione all'algebra relazionale

Prima di studiare il DQL (la componente di SQL dedicata all'interrogazione dei dati), è fondamentale comprendere un linguaggio formale più astratto: l'**algebra relazionale**. Si tratta di un insieme di operatori che si applicano a una o due relazioni per produrre come risultato una *nuova relazione*.
L'algebra relazionale è poi un linguaggio *procedurale*: ogni espressione specifica in modo esplicito la sequenza di operazioni da eseguire per calcolare il risultato.

Tra gli operatori fondamentali dell'algebra relazionale ci sono:

- **Selezione**
- **Proiezione**
- **Ridenominazione**
- **Unione**
- **Differenza**
- **Prodotto cartesiano**

Tra gli operatori derivati (ossia sostituibili con una combinazione di operatori fondamentali) dell'algebra relazionale ci sono:

- **Join naturale**
- **θ join**
- **Intersezione**

Operatori a una relazione

Definizione 4.3 – Proiezione

La **proiezione** è un operatore che permette di tagliare alcune colonne di una relazione, selezionando così delle **specifiche colonne di una relazione**. Per indicare una proiezione scriviamo $\pi_{A_1,\ldots,A_k}(r)$, che indica una proiezione che prende dalla relazione r le colonne riferite agli attributi $A_1, \ldots, A_k$.

Seguendo le regole dell'insiemistica, in una proiezione **NON prendiamo duplicati**. Se ad esempio prendiamo solo un attributo di una relazione in una proiezione, avremo l'insieme dei valori distinti di quell'attributo. Se invece prendiamo più attributi, elimineremo solo i duplicati che hanno valori uguali in TUTTI i campi presi dalla proiezione.

Definizione 4.4 – Selezione

La **selezione** è un operatore che permette di tagliare alcune righe di una relazione, selezionando così delle **specifiche righe di una relazione**. Per indicare una selezione scriviamo $\sigma_C(r)$, che indica una selezione che prende dalla relazione r tutte le righe che soddisfano la condizione C, scritta tramite operatori booleani.

Ad esempio, C può essere $A < B \land \neg(A = 10)$, dove A e B sono attributi con lo stesso dominio o valori costanti, oppure può essere $\neg(A = B)$ per indicare che il valore dell'attributo A non deve essere uguale a quello dell'attributo B.

Se prendiamo quindi la relazione r

Nome	C#	Citta
Rossi	1	Roma
Rossi	2	Milano
Rossi	3	Roma
Bianchi	4	Roma
Verdi	5	Roma

La proiezione $\pi_{\text{Nome, Citta}}(r)$ sarà una tabella del genere:

Nome	Citta
Rossi	Roma
Rossi	Milano
Bianchi	Roma
Verdi	Roma

mentre la selezione $\sigma_{C\#<3}(r)$ sarà una tabella del genere:

Nome	C#	Citta
Rossi	1	Roma
Rossi	2	Milano

Operatori a due relazioni

Definizione 4.5 – Union compatibili

Due relazioni r_1 e r_2 sono **union compatibili** se:

- hanno lo stesso **numero di attributi**
- gli attributi corrispondenti **nell'ordine** sono **definiti sullo stesso dominio** e **hanno un significato omogeneo** (ad esempio gli attributi Eta e Altezza sono definiti sullo stesso dominio dei numeri interi, ma hanno due significati molto distanti)

Ad esempio, le due relazioni

Attaccanti

Nome	Cognome	Eta	Gol
Paolo	Rossi	32	400
Lionel	Ronaldo	23	2

Calciatori

Nome	Cognome	Ruolo
Antonio	Raimondo	Attaccante
Franco	Messi	Portiere

Non sono union compatibili, perché hanno un numero di attributi diverso, e il loro terzo attributo è definito su domini diversi. Se vogliamo fare un'unione tra queste relazioni, dobbiamo fare una proiezione su entrambe prendendo solo le prime due colonne di entrambe. Dunque possiamo dire che le relazioni

$\pi_{\text{Nome, Cognome}}(\text{Attaccanti})$

Nome	Cognome
Paolo	Rossi
Lionel	Ronaldo

$\pi_{\text{Nome, Cognome}}(\text{Calciatori})$

Nome	Cognome
Antonio	Raimondo
Franco	Messi

sono **union compatibili**.

N.B. Non era importante che il nome dei campi fosse lo stesso. Se avessimo avuto "Nome" e "Soprannome" come nomi dei campi, essendo definiti questi sullo stesso dominio e avendo significato simile, le due relazioni sarebbero comunque rimaste union compatibili.

Definizione 4.6 – Unione

L'**unione** è un operatore che permette di creare una nuova relazione partendo da due relazioni *union compatibili* con tuple che appartengono ad almeno uno dei due operandi. Per indicare un'unione scriviamo $\boldsymbol{r_1 \cup r_2}$, che indica un'unione tra le relazioni r_1 e r_2.

Proviamo a fare l'unione tra le due proiezioni viste prima:

$$\pi_{\text{Nome, Cognome}}(\text{Attaccanti}) \cup \pi_{\text{Nome, Cognome}}(\text{Calciatori})$$

Nome	Cognome
Paolo	Rossi
Lionel	Ronaldo
Antonio	Raimondo
Franco	Messi

Vediamo ora un altro esempio. Ipotizziamo di voler unire le seguenti relazioni:

Docenti

Nome	CodDoc	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Bianchi	C3	Lingue

Amministrativi

Nome	CodAmm	Dipartimento
Riccio	C1	Lingue
Pierro	C2	Lettere
Bianchi	C3	Lingue

Avremmo come risultato questa relazione:

Docenti ∪ Amministrativi

Nome	CodDoc	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Bianchi	C3	Lingue
Riccio	C1	Lingue
Pierro	C2	Lettere

Notiamo una cosa importante: la tupla (Bianchi, C3, Lingue) si trova in tutte e due le relazioni, dunque nell'unione questa viene scritta una volta sola. Ma ciò non significa che questa tupla indichi in entrambe le relazioni la stessa persona. È infatti possibile che due signor Bianchi, entrambi al dipartimento di Lingue, abbiano lo stesso numero di codice ma che uno sia un Docente e uno sia un Amministrativo, rimanendo due persone diverse. Dobbiamo quindi modificare le relazioni iniziali cambiando i codici, così da eliminare questa ambiguità:

Docenti

Nome	CodDoc	Dipartimento
Rossi	D1	Matematica
Rossi	D2	Lettere
Bianchi	D3	Lingue

Amministrativi

Nome	CodAmm	Dipartimento
Riccio	A1	Lingue
Pierro	A2	Lettere
Bianchi	A3	Lingue

L'unione tra queste due relazioni sarà quindi:

Docenti ∪ Amministrativi

Nome	CodDoc	Dipartimento
Rossi	D1	Matematica
Rossi	D2	Lettere
Bianchi	D3	Lingue
Riccio	A1	Lingue
Pierro	A2	Lettere
Bianchi	A3	Lingue

Definizione 4.7 – Differenza

La **differenza** è un operatore che permette di creare una nuova relazione partendo da due relazioni *union compatibili* con tuple che appartengono al primo operando e NON appartengono al secondo. Per indicare una differenza scriviamo $r_1 - r_2$, che indica una differenza tra le relazioni r_1 e r_2.

Riprendendo l'esempio di prima dei docenti (quello con ancora i codici uguali, altrimenti la differenza sarebbe superflua essendo tutte le tuple diverse), troviamo che la relazione Docenti − Amministrativi (che rappresenta tutti i docenti non amministrativi) è pari a

Docenti − Amministrativi

Nome	CodDoc	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere

Definizione 4.8 – Intersezione

L'**intersezione** è un operatore che permette di creare una nuova relazione partendo da due relazioni *union compatibili* con tuple che appartengono a entrambi gli operandi. Per indicare un'intersezione scriviamo $r_1 \cap r_2$.
È un operatore derivato perché equivale a effettuare una doppia differenza ($r_1 \cap r_2 = (r_1 - (r_1 - r_2))$).

Riprendendo sempre lo stesso esempio di prima, troviamo che la relazione Docenti ∩ Amministrativi è pari a

Docenti ∩ Amministrativi

Nome	CodDoc	Dipartimento
Bianchi	C3	Lingue

4.3.2 Algebra relazionale in SQL 1

Abbiamo visto delle prime operazioni nell'algebra relazionale. Prima di andare a vedere operatori più complessi, vediamo come si traducono queste prime operazioni in SQL, introducendo quindi a tutti gli effetti il nuovo linguaggio DQL, che riguarda l'interrogazione dei dati.

Per effettuare una **proiezione** utilizziamo il costrutto **`SELECT`**, mentre per effettuare una **selezione** utilizziamo **`WHERE`**. È importante però subito definire una differenza tra il comando `SELECT` e le proiezioni: sappiamo che nelle proiezioni in algebra relazionale i doppioni vengono presi una volta sola, ma in SQL usando il comando `SELECT` questo non accade. Se si vuole questo comportamento è necessario aggiungere anche la parola chiave **`distinct`**.
Se ad esempio abbiamo una relazione Studenti con attributi Nome, Cognome e Eta, e vogliamo fare una proiezione sulle prime due colonne prendendo però solo gli studenti con età maggiore di 18 e minore di 60 o studenti con età maggiore di 90 (stiamo dunque effettuando una proiezione e una selezione insieme), scriveremmo:

```
SELECT Nome, Cognome
FROM Studenti
WHERE (Eta > 18 AND Eta < 60) or (Eta > 90)
```

effettuando una vera e propria **query**.
La parola chiave **`FROM`** ci indica da quali relazioni prendere i dati (si possono prendere anche più tabelle insieme), in questo caso quindi da Studenti. Se si avessero più relazioni gli attributi andrebbero specificati scrivendo nome_relazione.nome_attributo. In questo caso, è comodo usare anche delle **abbreviazioni** (scrivendo ad esempio `FROM Studenti s` e utilizzando poi la lettera `s` per indicare gli studenti, o scrivendo `SELECT s.nome AS nome` per utilizzare la parola `nome` per indicare l'attributo nome di studenti). Ipotizziamo di avere anche un'altra relazione Esami con attributi Codice, MatricolaStudente, Voto e Data. Se volessimo effettuare una query su queste due tabelle che restituisce una relazione con il nome dello studente e il rispettivo voto, scriveremmo:

```
SELECT Studenti.Nome, Esami.Voto
FROM Studenti, Esami
WHERE Studenti.Matricola = Esami.MatricolaStudente
```

N.B. In questo caso stiamo effettuando un vero e proprio prodotto cartesiano tra le due relazioni, concetto che approfondiremo tra poco ritornando a studiare l'algebra relazionale.

Inoltre, in SQL (ma non in algebra relazionale) è possibile effettuare anche un `SELECT` prendendo tutti gli attributi usando l'*, scrivendo ad esempio `SELECT Studenti.*`.

È possibile anche effettuare delle operazioni su tutta una colonna scrivendo ad esempio **`SELECT Studenti.Eta / 2 AS EtaDimezzata`**. In questo caso creiamo quindi una nuova relazione che ha come colonna l'età di ogni studente ma divisa già per due, che prende il nome di EtaDimezzata.

Tra le condizioni che possiamo effettuare esistono anche le **condizioni like**, che permettono di effettuare controlli sulle stringhe. Ad esempio, se vogliamo tutti i nomi che hanno come primo carattere una "A" e come terzo carattere una "d", scriveremmo **`WHERE Nome like 'A_d%'`**, dove usiamo l'_ per indicare che la "d" deve essere la terza lettera (se doveva essere la quinta avremmo usato 3 _ al posto di uno).

Per verificare poi che un valore sia `NULL` non usiamo gli operatori normali ma scriviamo **`IS NULL`** o **`IS NOT NULL`**. Ad esempio, possiamo effettuare una query scrivendo `WHERE Eta IS NOT NULL`.

Esempio

Presa la relazione

Studenti

Matricola	Cognome	Universita	Media
7309	Neri	Sapienza	23
5998	Neri	Roma Tre	22
9553	Rossi	Sapienza	18
5698	Rossi	Sapienza	29

creare una nuova relazione con gli studenti con la media maggiore di 27 o minore di 20 e con campi con solo il cognome e l'università, eliminando eventuali duplicati.

Dobbiamo quindi effettuare questa query:

```
SELECT distinct Cognome, Universita
FROM Studenti
WHERE Media > 27 or Media < 20
```

che ci restituirà questa relazione:

Cognome	Universita
Rossi	Sapienza

4.3.3 Altri operatori dell'algebra relazionale

Definizione 4.9 – Prodotto cartesiano

Il **prodotto cartesiano** tra due relazioni permette di unire le informazioni di due relazioni in una nuova relazione creando delle nuove tuple ottenute concatenando le tuple della prima relazione con quelle della seconda. Per indicare un prodotto cartesiano scriviamo $r_1 \times r_2$, che indica un prodotto cartesiano tra le relazioni r_1 e r_2.

Vediamo un esempio. Prendiamo le relazioni

Operatore

Nome	CodiceOperatore	Citta
Rossi	C1	Napoli
Rossi	C2	Roma
Bianchi	C3	Milano

Ordine

Ordine	CodiceOperatore	NPezzi
O1	C2	300
O2	C2	200
O3	C3	100

La nuova relazione Operatore × Ordine non è applicabile senza effettuare dei cambiamenti alle altre relazioni. Questo perché le due relazioni hanno una colonna con lo stesso nome (CodiceOperatore), dunque nella nuova relazione formata dal prodotto cartesiano avremmo due colonne con lo stesso nome, cosa non possibile. Per risolvere questo problema, introduciamo un nuovo operatore chiamato **ridenominazione**.

Definizione 4.10 – Ridenominazione

La **ridenominazione** è un operatore che permette di modificare lo schema di una relazione (è l'unico operatore che non lavora sulle istanze ma sullo schema), permettendo di cambiare il nome di alcuni campi. Per indicare una ridenominazione scriviamo **Nuova_Relazione** = $\rho_{\text{nome1, nome2,...}\rightarrow\text{nuovo_nome1, nuovo_nome2,...}}$ oppure **Nuova_Relazione** = $\rho_{\text{nuovo_nome1, nuovo_nome2,...}\leftarrow\text{nome1, nome2,...}}$, che indica una ridenominazione dei vecchi nomi con quelli nuovi.

Possiamo quindi effettuare una ridenominazione sulla relazione Ordine, creando una nuova relazione in questo modo: OrdineR = $\rho_{\text{CodiceOperatore}\rightarrow\text{Codice}}$(Ordine). Abbiamo quindi ora queste due relazioni su cui fare il prodotto cartesiano:

Operatore

Nome	CodiceOperatore	Citta
Rossi	C1	Napoli
Rossi	C2	Roma
Bianchi	C3	Milano

Ordine

Ordine	Codice	NPezzi
O1	C2	300
O2	C2	200
O3	C3	100

Dunque troviamo ora una nuova relazione:

Operatore × Ordine

Nome	CodiceOperatore	Citta	Ordine	Codice	NPezzi
Rossi	C1	Napoli	O1	C2	300
Rossi	C1	Napoli	O2	C2	200
Rossi	C1	Napoli	O3	C3	100
Rossi	C2	Roma	O1	C2	300
Rossi	C2	Roma	O2	C2	200
Rossi	C2	Roma	O3	C3	100
Bianchi	C3	Milano	O1	C2	300
Bianchi	C3	Milano	O2	C2	200
Bianchi	C3	Milano	O3	C3	100

Questa nuova relazione, però, ha delle righe che sono inutili. Guardiamo ad esempio la prima riga: abbiamo l'operatore C1 collegato a un ordine eseguito dall'operatore C2. Questa cosa non ha senso, dunque dobbiamo effettuare una **selezione**, prendendo solo le tuple dove CodiceOperatore = Codice. Dunque dobbiamo prendere la relazione $\sigma_{\text{CodiceOperatore = Codice}}(\text{Operatore} \times \text{Ordine})$, ossia la tabella

$\sigma_{\text{CodiceOperatore = Codice}}(\text{Operatore} \times \text{Ordine})$

Nome	CodiceOperatore	Citta	Ordine	Codice	NPezzi
Rossi	C2	Roma	O1	C2	300
Rossi	C2	Roma	O2	C2	200
Bianchi	C3	Milano	O3	C3	100

Notiamo, però, che in questa nuova relazione c'è sia la colonna CodiceOperatore che la colonna Codice. Queste due colonne portano la stessa informazione, dunque è ridondante scriverle due volte. Per questo, effettuiamo una proiezione, eliminando una delle due colonne. Prendiamo quindi la tabella $\pi_{\text{Nome, CodiceOperatore, Citta, Ordine, NPezzi}}(\sigma_{\text{CodiceOperatore = Codice}}(\text{Operatore} \times \text{Ordine}))$, ossia la tabella

$\pi_{\text{Nome, CodiceOperatore, Citta, Ordine, NPezzi}}(\sigma_{\text{CodiceOperatore = Codice}}(\text{Operatore} \times \text{Ordine}))$

Nome	CodiceOperatore	Citta	Ordine	NPezzi
Rossi	C2	Roma	O1	300
Rossi	C2	Roma	O2	200
Bianchi	C3	Milano	O3	100

N.B. Questa sequenza di operatori che abbiamo scritto non è molto efficiente. Infatti, effettuiamo prima il prodotto cartesiano e poi la selezione e la proiezione, facendo quindi un prodotto cartesiano su più tuple e su tuple più lunghe del necessario. Questo però in SQL non è

un problema dato che il DBMS in automatico sceglie l'ordine delle operazioni più efficiente possibile.

Definizione 4.11 – Join Naturale

Il **Join Naturale** è un operatore che permette di effettuare un prodotto cartesiano, una selezione e una proiezione tutto insieme, risolvendo i problemi che abbiamo appena visto con il prodotto cartesiano base. Prese infatti due relazioni r_1 e r_2 con $A_1, \ldots, A_k$ attributi in comune (ossia con lo stesso nome), scriviamo $\boldsymbol{r_1 \bowtie r_2}$ per indicare un join naturale tra r_1 e r_2, che ritornerà una nuova relazione con come colonne gli attributi di r_1 e r_2 senza duplicati, e come tuple tutte le tuple di $r_1 \times r_2$ dove $r_1.A_1 = r_2.A_1 \wedge r_1.A_2 = r_2.A_2 \wedge \ldots \wedge r_1.A_k = r_2.A_k$.

Dunque possiamo dire che $r_1 \bowtie r_2 = \pi_{XY}(\sigma_C(r_1 \times r_2))$, dove X sono gli attributi di r_1, Y gli attributi di r_2 non in comune con r_1 e $C = r_1.A_1 = r_2.A_1 \wedge r_1.A_2 = r_2.A_2 \wedge \ldots \wedge r_1.A_k = r_2.A_k$.

La query che quindi prima abbiamo scritto con una proiezione, selezione e prodotto cartesiano può quindi essere scritta ora facilmente con un semplice join naturale, scrivendo Operatore ⋈ Ordine.

Proprietà 4.1 – Casi limite del join naturale

Vediamo ora due casi limite del join naturale:

- Se le due relazioni hanno attributi con lo stesso nome ma **non hanno tuple con lo stesso valore per quegli attributi** il join naturale restituirà una **relazione vuota**.
- Se le due relazioni **non hanno attributi con lo stesso nome** il join degenera in un **prodotto cartesiano normale**.

Il secondo caso è quello più pericoloso: riguardando infatti le tabelle iniziali Operatore e Ordine, queste hanno l'attributo CodiceOperatore in comune, ma nella tabella Operatore poteva avere molto senso scrivere semplicemente Codice (lasciando sottinteso Operatore). Se questo fosse stato fatto, il join naturale non avrebbe fatto altro che restituire un prodotto cartesiano tra le due tabelle, nonostante il codice dell'operatore si trovi in entrambe, solo con un nome diverso. È fondamentale in questo caso la *ridenominazione*, così da modificare il nome dei vari attributi così che il join naturale possa confrontarli.

Un altro problema importante del join naturale è che, nonostante due attributi possano avere lo stesso nome, non è detto che abbiano lo stesso significato. Pensiamo ad esempio a due relazioni, Docente e Studente, che hanno entrambe l'attributo Codice. È scontato per noi che Docente.Codice e Studente.Codice non indichino lo stesso codice, ma piuttosto indichino, rispettivamente, il codice del docente e quello dello studente.

Definizione 4.12 – θ Join

Il **θ join** è un prodotto cartesiano effettuato tra due relazioni che prende solo delle tuple di tale join che rispettano una certa condizione $A\theta B$, dove A è un attributo dello schema della prima relazione, B della seconda (ricordiamo $\text{dom}(A) = \text{dom}(B)$), e θ è un operatore di confronto ($=, \leq, \geq, \ldots$). Per indicare un θ join scriviamo $r_1 \bowtie_{A\theta B} r_2$, che indica un θ join tra le relazioni r_1 e r_2 con la condizione $A\theta B$.

Possiamo quindi dire che $r_1 \bowtie_{A\theta B} r_2 = \sigma_{A\theta B}(r_1 \times r_2)$.

Ad esempio, prese le relazioni

Tutor

Nome	Eta	Stipendio
Mario	30	300
Erling	20	2000
Arturo	25	5000

FasceStipendio

Fascia	Min	Max
Basso	0	1000
Medio	1001	2000
Alto	2001	10000

il θ join Tutor $\bowtie_{\text{Stipendio>=Min}\wedge\text{Stipendio<=Max}}$ FasceStipendio sarà questa tabella:

Tutor $\bowtie_{\text{Stipendio>=Min}\wedge\text{Stipendio<=Max}}$ FasceStipendio

Nome	Eta	Stipendio	Fascia	Min	Max
Mario	30	300	Basso	0	1000
Erling	20	2000	Medio	1001	2000
Arturo	25	5000	Alto	2001	10000

Dato che però qui abbiamo ancora le colonne Min e Max che nella tabella finale sono inutili perché servono solo per stabilire in che fascia mandare un Tutor, possiamo effettuare una proiezione, creando dunque la relazione $\pi_{\text{Nome, Eta, Stipendio, Fascia}}$(Tutor $\bowtie_{\text{Stipendio>=Min}\wedge\text{Stipendio<=Max}}$ FasceStipendio), ossia la tabella

$\pi_{\text{Nome, Eta, Stipendio, Fascia}}$(Tutor $\bowtie_{\text{Stipendio>=Min}\wedge\text{Stipendio<=Max}}$ FasceStipendio)

Nome	Eta	Stipendio	Fascia
Mario	30	300	Basso
Erling	20	2000	Medio
Arturo	25	5000	Alto

4.3.4 Algebra relazionale in SQL 2

Abbiamo già accennato quando abbiamo parlato della parola chiave `FROM` che aggiungendo più relazioni a questa è possibile effettuare dei prodotti cartesiani. Non solo: questa è anche la sintassi utilizzata per effettuare dei join. Ad esempio, il join naturale visto in precedenza Operatore ⋈ Ordine in SQL diventa:

```
SELECT distinct Nome, CodiceOperatore, Citta, Ordine, NPezzi
FROM Operatore, Ordine
WHERE CodiceOperatore = Codice
```

In realtà, sia un prodotto cartesiano che un qualsiasi join in SQL si scrivono sempre allo stesso modo, ossia scrivendo le due (o più) relazioni interessate accanto alla parola chiave `FROM` separate da una virgola. Sarà poi il DBMS a decidere quale operazione eseguire per rendere il tutto il più efficiente possibile.

Torna qui importante un costrutto che avevamo già accennato quando abbiamo iniziato a tradurre l'algebra relazionale in SQL, ossia le **abbreviazioni**. Queste sono fondamentali solitamente quando abbiamo più tabelle con i campi con lo stesso nome e vogliamo essere più chiari nella decisione di quali campi scegliere nella selezione. Se ad esempio abbiamo le relazioni

Docenti

Nome	**Cognome**	**Codice**
Antonio	Messi	1
Lionel	Ronaldo	2
Cristiano	Raimondo	3

Esami

Nome	**Codice**	**Data**
Chimica	1	02/03/2002
Fisica	2	02/02/1900
Geometria	3	02/03/1700

se vogliamo fare una query per prendere tutti gli esami effettuati dal docente col codice 1 (Codice come campo in Esame è una foreign key che collega la relazione a Docente), le abbreviazioni ci renderebbero la vita più facile. Infatti, al posto di scrivere

```
SELECT distinct Esami.Nome, Data, Docenti.Codice
FROM Docenti, Esami
WHERE Docenti.Codice = Esami.Codice AND Docenti.Codice = 1
```

potremmo semplicemente scrivere

```
SELECT distinct e.Nome AS NomeEsame, Data, d.Codice AS CodiceDocente
FROM Docenti d, Esami e
WHERE d.Codice = e.Codice AND d.Codice = 1
```

Entrambe le query restituirebbero comunque la relazione

NomeEsame	Data	CodiceDocente
Chimica	02/03/2002	1

Queste sono comode quando abbiamo due o più relazioni nella query, ma sono necessarie se abbiamo una query su un prodotto cartesiano / join eseguito sulla stessa relazione. Se ad esempio abbiamo le relazioni

Paternita

Padre	Figlio
Giuseppe	Antonio
Luigi	Alberto
Luigi	Mario

Persone

Nome	Eta	Reddito
Giuseppe	45	1000
Antonio	21	1500
Luigi	30	1200
Alberto	2	0
Mario	1	0

E vogliamo creare una nuova relazione con tutti i figli che guadagnano più del padre, in cui salviamo anche il reddito del figlio e quello del padre, dovremmo scrivere una query di questo tipo:

```
SELECT Figlio, f.Reddito AS Reddito, p.Reddito AS RedditoPadre
FROM Persone p, Paternita, Persone f
WHERE p.Reddito < f.Reddito AND p.Nome = Padre AND f.Nome = Figlio
```

che ci restituirebbe la seguente tabella:

Figlio	Reddito	RedditoPadre
Antonio	1500	1000

Abbiamo detto che SQL da solo decide quando effettuare un join e quando effettuare un prodotto cartesiano senza che noi facciamo nulla, ma in realtà è possibile anche effettuare un **join esplicito**. Per capire meglio come vediamo un esempio. Ipotizziamo di voler creare una relazione con Figlio, Padre e Reddito Padre. Normalmente noi scriveremmo la query

```
SELECT Figlio, Padre, Reddito AS RedditoPadre
FROM Paternita, Persone
WHERE Nome = Padre AND Reddito > 1000
```

Potremmo però anche scriverla in quest'altro modo, utilizzando proprio un join esplicito:

```
SELECT Figlio, Padre, Reddito AS RedditoPadre
FROM Paternita JOIN Persone ON Nome = Padre
WHERE Reddito > 1000
```

Entrambe le query restituirebbero la relazione

Figlio	Padre	ReddितoPadre
Alberto	Luigi	1200
Mario	Luigi	1200

La logica è molto semplice: nel `FROM` scriviamo tutte le condizioni riguardanti il join tra le due relazioni, separate tutte dalla parola chiave `ON` (dunque se avessimo avuto altre condizioni avremmo dovuto scrivere `FROM r1 x r2 ON C1 ON C2...`), e se abbiamo altre condizioni che non riguardano il join le aggiungiamo come delle normali condizioni affianco la parola chiave `WHERE`.
N.B. Il join esplicito non è sempre implementato: soprattutto in sistemi più vecchi è possibile che questo non sia presente.

Infine, in SQL (ma non in algebra relazionale), a seguito di una query è possibile **riordinare le tuple** seguendo il valore di una delle colonne, aggiungendo, alla fine della query, la scritta **`ORDER BY`** + il nome del campo desiderato. Ad esempio, la query

```
SELECT Nome, Eta, Reddito
FROM Persone
WHERE Eta > 10
ORDER BY Eta
```

restituirebbe la relazione

Nome	Eta	Reddito
Antonio	21	1500
Luigi	30	1200
Giuseppe	45	1000

Notiamo che le tuple sono state ordinate in modo crescente, dalla tupla con l'età più piccola a quella con l'età più grande. Se volessimo il contrario, dovremmo aggiungere accanto a Eta la parola chiave **`DESC`**.

4.3.5 Problemi particolari nell'algebra relazionale

Fino ad ora abbiamo studiato query con condizioni facili da rispondere, del tipo "Nomi di tutte le persone con **età > 50**", oppure "Clienti che hanno effettuato **almeno** un ordine da 50 o più pezzi". Queste sono facili perché per ognuna di queste condizioni basta prendere in sequenza ogni tupla della relazione iniziale (o di un eventuale join, ad esempio tra Clienti e Ordini), e verificare la condizione richiesta. Arriviamo ora a studiare però query più complesse, che non

chiedono più se ci sono persone che hanno effettuato almeno un ordine da 50 o più pezzi, ma piuttosto se esistono clienti che hanno effettuato **sempre** ordini da 50 o più pezzi.
Per capire come risolvere queste situazioni, vediamo un esempio concreto prendendo le tabelle Clienti e Ordini:

Clienti

Nome	**CodiceCliente**	**Citta**
Giuseppe	C1	Roma
Giuseppe	C2	Milano
Antonio	C3	Roma

Ordini

CodiceCliente	**Codice**	**NPezzi**
C1	A1	200
C1	A2	10
C2	A3	1000
C2	A4	500
C2	A5	300

In questo caso scansionare le tuple una a una e inserire i clienti che rispettano la condizione non basta più: infatti, guardando solo la prima tupla dovremmo aggiungere Giuseppe nella soluzione, ma guardando poi la seconda notiamo che questi non soddisfa interamente la condizione. Come traduciamo però questo in algebra relazionale? Possiamo risolvere il problema usando la **doppia negazione**. Infatti, l'insieme dei clienti che hanno sempre effettuato ordini da almeno 50 pezzi è pari all'insieme dei clienti - l'insieme dei clienti che almeno una volta hanno effettuato un ordine da meno di 50 pezzi. Dunque la query $\pi_{\textbf{Nome}}(\pi_{\textbf{Nome,CodiceCliente}}(\textbf{Cliente} \bowtie \textbf{Ordine}) - \pi_{\textbf{Nome,CodiceCliente}}(\sigma_{\textbf{NPezzi<50}}(\textbf{Cliente} \bowtie \textbf{Ordine})))$ contiene tutti i clienti che hanno fatto sempre ordini da almeno 50 pezzi. Dunque con questa query troviamo la relazione

Nome
Giuseppe

C'è però un piccolo particolare: scrivendo la query in questo modo stiamo eliminando anche tutti i clienti che non hanno mai effettuato ordini. Ciò può essere corretto o meno in base alla query: in questo caso non è esplicitato se questi clienti dovrebbero essere considerati nella soluzione o meno, dunque è sempre preferibile non inserirli. Ma nel caso fosse stato specificato avremmo semplicemente dovuto scrivere $\pi_{\text{Nome}}(\pi_{\text{Nome,CodiceCliente}}(\text{Cliente}) - \pi_{\text{Nome,CodiceCliente}}(\sigma_{\text{NPezzi<50}}(\text{Cliente} \bowtie \text{Ordine})))$ (in questa nostra situazione con questa seconda query oltre a Giuseppe avremmo avuto in relazione anche Antonio). **N.B.** Abbiamo qui prima proiettato sia il Nome che il CodiceCliente e poi solo il nome perché questo da solo non è un identificativo sufficiente. Infatti, proiettando fin da subito solo sul nome entrambi i Giuseppe sarebbero stati eliminati, nonostante Giuseppe con codice C2 rispettasse la condizione.

Se avessimo avuto come query quella di trovare tutti i clienti che non hanno ad esempio **mai** effettuato ordini con più di 200 pezzi il ragionamento non cambiava di una virgola: sfruttando la doppia negazione bisognava trovare tutti i clienti che hanno mai fatto un ordine con più di 200 pezzi e eliminarli dalla lista dei clienti che hanno effettuato ordini (o solo dei clienti, se si vogliono anche considerare i clienti con 0 ordini), così da avere infine solo i clienti che rispettano la nostra condizione.

Vediamo ora invece un'altra situazione. Ipotizziamo di avere la seguente relazione:

Impiegati

Nome	**C#**	**Stip**	**Capo#**
Rossi	C1	100	C3
Pirlo	C2	200	C3
Bianchi	C3	500	NULL
Verdi	C4	200	C2
Neri	C5	150	C1
Tosi	C6	100	C1

La query ci chiede di trovare il nome e il codice di ogni impiegato che guadagna quanto o di più del proprio capo. Notiamo subito una cosa particolare: le informazioni dei capi sono sempre nella relazione impiegato. Dunque stiamo effettuando una query che ci richiede di fare un θ join tra una relazione e sé stessa. Per poterlo fare, però, è necessaria prima di tutto una ridenominazione: ImpiegatiC = $\rho_{\text{Nome, C\#, Stip, Capo\#}\rightarrow\text{CNome, CC\#, CStip, CCapo\#}}$(Impiegati) (non è per forza necessaria, potremmo esplicitare i nomi degli attributi scrivendo ogni volta Impiegati.Nome o ImpiegatiC.Nome, ma con la ridenominazione rendiamo sicuramente questa query più chiara). Possiamo ora studiare la relazione Impiegati$\bowtie_{\text{Capo\#=CC\#}}$ImpiegatiC, che si traduce nella tabella

Impiegati$\bowtie_{\text{Capo\#=CC\#}}$ImpiegatiC

Nome	**C#**	**Stip**	**Capo#**	**CNome**	**CC#**	**CStip**	**CCapo#**
Rossi	C1	100	C3	Bianchi	C3	500	NULL
Pirlo	C2	200	C3	Bianchi	C3	500	NULL
Verdi	C4	200	C2	Pirlo	C2	200	C3
Neri	C5	150	C1	Rossi	C1	100	C3
Tosi	C6	100	C1	Rossi	C1	100	C3

Notiamo che in questa tabella non sono presenti dalla parte della relazione Impiegati le informazioni riguardanti Bianchi. Questo accade perché semplicemente Bianchi non ha un capo, dunque Capo# = CC# per lui non è mai vero, perché non esiste nessun impiegato che ha come *C#* il valore NULL.
A questo punto non ci resta che confrontare il valore Stip con il valore CStip e prendere le tuple tali che Stip $\geq$ CStip, e infine proiettare. Troviamo quindi la relazione $\pi_{\text{Nome,C\#}}$(Impiegati $\bowtie_{\text{Capo\#=CC\#}\wedge\text{Stip}\geq\text{CStip}}$ ImpiegatiC), che si traduce nella relazione

$$\pi_{\text{Nome,C\#}}(\text{Impiegati} \bowtie_{\text{Capo\#=CC\#}\wedge\text{Stip}\geq\text{CStip}} \text{ImpiegatiC})$$

Nome	**C#**
Verdi	C4
Neri	C5
Tosi	C6

E se invece volessimo ora il nome e il codice di tutti i capi che guadagnano di più di tutti i loro impiegati? In questo caso torna fondamentale la doppia negazione. Possiamo sfruttare la query appena studiata per trovare tutti i capi che guadagnano quanto o meno dei loro impiegati, per poi prendere la relazione con tutti i capi e fare una differenza proprio con questa query. Se chiamiamo r la relazione Impiegati $\bowtie_{\text{Capo\#=CC\#} \wedge \text{Stip} \geq \text{CStip}}$ ImpiegatiC (ossia la query precedente prima della proiezione), troviamo che la soluzione a questo problema è la query $\pi_{\text{CNome,CC\#}}(\text{Impiegati} \bowtie_{\text{Capo\#=CC\#}} \text{ImpiegatiC}) - \pi_{\text{CNome,CC\#}}(r)$, che si traduce nella relazione

$$\pi_{\text{CNome,CC\#}}(\text{Impiegati} \bowtie_{\text{Capo\#=CC\#}} \text{ImpiegatiC}) - \pi_{\text{CNome,CC\#}}(r)$$

CNome	**CC#**
Bianchi	C3

N.B. È importante prendere CNome e CC# come campi e non Nome e C#. Infatti, noi vogliamo i capi degli impiegati e non gli impiegati stessi, altrimenti staremmo inserendo nella relazione finale anche impiegati che non sono capi, e questo non è corretto.

Vediamo infine un'altra query interessante. Ci troviamo in un database con le seguenti relazioni:

- DIPENDENTE(Id, Nome, Cognome, DataN)
- PROGETTO(Id, Titolo, Data)
- TEAM(ProgId, DipenId, Ruolo)

con l'attributo Ruolo che può assumere tre valori: "Project Leader", "Programmer" o "Designer". Ci viene chiesto di trovare il titolo del progetto **più vecchio** di ogni dipendente a cui egli ha partecipato come Project Leader, salvando come informazioni nella relazione il Nome, il Cognome, la DataN del dipendente e il Titolo del progetto. La parte interessante di questa query è la ricerca del progetto *più vecchio*. Anche qui ci troviamo nello stesso vicolo cieco già visto quando ci siamo trovati davanti la parola *sempre*: come possiamo essere sicuri che un progetto sia il più vecchio fra tutti? Prima di vedere come risolvere questo problema, troviamoci una relazione con all'interno diverse tuple con i progetti e i project leader di ogni progetto.
Innanzitutto prendiamo tutti i project leader dalla relazione Team con i rispettivi progetti di cui erano a capo: ProjectLeader = $\pi_{\text{ProgId,DipenId}}(\sigma_{\text{Ruolo="Project Leader"}}(\text{Team}))$. Creiamo ora una relazione con tutti i progetti associati al loro rispettivo project leader: ProgettoPL = $\pi_{\text{ProgId,Data,DipenId}}(\text{Progetto} \bowtie_{\text{Id=ProgId}} \text{ProjectLeader})$. Siamo così arrivati a una tabella di questo tipo:

ProgettoPL

ProgId	Data	DipenId
P1	15/04/2023	D1
P2	13/02/2023	D1
P3	11/04/2010	D3
P4	11/05/2020	D2
P5	11/03/2019	D1
P6	09/09/1990	D3

Per trovare il minimo di ogni dipendente, prendiamo questa tabella e effettuiamo una ridenominazione: ProgettoPL2 = $\rho_{\text{ProgId,Data,DipenId}\to\text{ProgId2,Data2,DipenId2}}$(ProgettoPL). A questo punto possiamo effettuare un θ join, ponendo come condizione che DipenId sia uguale a DipenId2, così da avere una nuova tabella con tutte le coppie di progetti con lo stesso project leader. Troviamo quindi la tabella

ProgettoPL $\bowtie_{\text{DipenId=DipenId2}}$ ProgettoPL2

ProgId	Data	DipenId	ProgId2	Data2	DipenId2
P1	15/04/2023	D1	P1	15/04/2023	D1
P1	15/04/2023	D1	P2	13/02/2023	D1
P1	15/04/2023	D1	P5	11/03/2019	D1
P2	13/02/2023	D1	P1	15/04/2023	D1
P2	13/02/2023	D1	P2	13/02/2023	D1
P2	13/02/2023	D1	P5	11/03/2019	D1
P3	11/04/2010	D3	P3	11/04/2010	D3
P3	11/04/2010	D3	P6	09/09/1990	D3
P4	11/05/2020	D2	P4	11/05/2020	D2
P5	11/03/2019	D1	P1	15/04/2023	D1
P5	11/03/2019	D1	P2	13/02/2023	D1
P5	11/03/2019	D1	P5	11/03/2019	D1
P6	09/09/1990	D3	P3	11/04/2010	D3
P6	09/09/1990	D3	P6	09/09/1990	D3

A questo punto, prendiamo solo le tuple dove la data a destra è strettamente maggiore di quella a sinistra:

ProgettoPL $\bowtie_{\text{DipenId=DipenId2}\wedge\text{Data<Data2}}$ ProgettoPL2

ProgId	Data	DipenId	ProgId2	Data2	DipenId2
P2	13/02/2023	D1	P1	15/04/2023	D1
P5	11/03/2019	D1	P1	15/04/2023	D1
P5	11/03/2019	D1	P2	13/02/2023	D1
P6	09/09/1990	D3	P3	11/04/2010	D3

A questo punto notiamo una cosa: tutte le date che si trovano nella colonna Data2 sicuramente non possono corrispondere al progetto più vecchio del project leader a cui si riferiscono. Infatti, si riferiscono a progetti avvenuti sicuramente dopo altri seguiti sempre dallo stesso project leader. Dunque, per ottenere i progetti più vecchi, non ci resta che prendere la tabella iniziale e eliminare tutte le tuple con i progetti che sono presenti nella parte destra di questa nostra tabella. Se chiamiamo la tabella qui sopra r, arriviamo alla query Min = ProgettoPL $- \pi_{\text{ProgId2,Data2,DipenId2}}(r)$, ossia alla tabella

Min

ProgId	Data	DipenId
P4	11/05/2020	D2
P5	11/03/2019	D1
P6	09/09/1990	D3

Questo procedimento è quello standard per trovare il minimo di una relazione in algebra relazionale (dove purtroppo l'operatore `MIN` di SQL non è presente).

Arrivati a questo punto non ci resta che fare un θ join con la relazione Dipendente e poi con la relazione Progetto e proiettare tutto. Troviamo quindi che Sol = $\pi_{\text{Nome,Cognome,DataN,DipenId,Titolo,ProgId}}((\text{Min} \bowtie_{\text{DipenId=Id}} \text{Dipendente}) \bowtie_{\text{ProgId=Id}} \text{Progetto})$.
N.B. Abbiamo aggiunto nella soluzione anche DipenId e ProgId. Nonostante questi non fossero esplicitamente richiesti dalla traccia è sempre buona norma portare con sé le chiavi delle entità coinvolte, che possono essere sempre utili per identificare in modo univoco una o più istanze.

4.3.6 Operatori avanzati in SQL

Operatori aggregati

Definizione 4.13 – Operatori aggregati

Gli **operatori aggregati** sono operatori che lavorano non sulle singole tuple ma sugli insiemi di tuple. Questi sono messi a disposizione da SQL, ma non esiste alcuna controparte in algebra relazionale.

L'operatore **`COUNT`** è un operatore aggregato che conta tutte le tuple che rispettano una certa condizione. Ad esempio, se vogliamo contare il numero di figli di Paolo scriveremmo

```
SELECT count(*) AS NumFigliDiPaolo
FROM Paternita
WHERE Padre = 'Paolo'
```

Possiamo anche aggiungere alla funzione count il nome di un attributo, scrivendo `count(attributo)`, contando così il numero di tuple in cui quell'attributo non è nullo. Se scriviamo poi anche `count(distinct attributo)`, non solo contiamo le tuple dove l'attributo non è nullo, ma evitiamo di contare anche tutti i duplicati.

Esistono poi altri operatori aggregati come **`SUM, AVG, MAX`** e **`MIN`** che ci permettono, rispettivamente, di calcolare la somma, la media, il massimo e il minimo di un insieme di valori. Ovviamente queste operazioni hanno senso solo su attributi numerici (non possiamo fare la media di tante stringhe, ovviamente), e in questi operatori tutte le tuple con valore `NULL` sono sempre evitate.

Vediamo ora una query errata con gli operatori aggregati:

```
SELECT Nome, MAX(Reddito)
FROM Persone
```

Quale nome dovremmo affiancare al valore `MAX(Reddito)` nella relazione risultante? Nessuno dei nomi presenti nella relazione Persone è un nome corretto da affiancare al valore `MAX(Reddito)`, dunque ci ritroveremmo con una relazione con tante righe con tutti i nomi affiancati dal `MAX(Reddito)`, che non ha senso. In questo caso, diciamo che la **target list** (ossia l'insieme degli attributi di una query) è errata. Infatti questa deve essere *omogenea*, dunque o sono presenti solo operatori aggregati oppure non ne deve essere presente nemmeno uno.

Un'altra clausola importante è il **`GROUP BY`**. Scrivendo `GROUP BY ListaAttributi` possiamo raggruppare tutte le tuple di una relazione che hanno valori uguali per tutti gli attributi presenti nella lista di attributi. Questa clausola ci permette anche di avere target list non omogenee che funzionano lo stesso. Ad esempio, la query

```
SELECT Padre, count(*) AS NumFigli
FROM Paternita
GROUP BY Padre
```

crea una relazione con il numero di figli non solo di Paolo, ma di tutti i diversi padri. Ad esempio, partendo dalla relazione

Paternita

Padre	Figlio
Paolo	Luigi
Paolo	Mario
Fabrizio	Antonio
Diego	Giuseppe
Diego	Pietro

troveremmo la tabella

Padre	NumFigli
Paolo	2
Fabrizio	1
Diego	2

Dunque il GROUP BY ci permette di creare target list non omogenee, con operatori aggregati e non insieme. La condizione fondamentale, però, è che tutto ciò che nel SELECT non è un operatore aggregato **vada nel GROUP BY**. Ad esempio, la query

```
SELECT Padre, AVG(f.Reddito), p.Reddito
FROM Persone f JOIN Paternita ON Figlio = Nome JOIN Persone p ON Padre = p.Nome
GROUP BY Padre
```

è errata, perché non abbiamo messo `p.Reddito` nel GROUP BY. Se avessimo un padre con più redditi il SELECT su `p.Reddito` non avrebbe più senso (torniamo al problema del Nome visto prima con la query riguardo il MAX(Reddito)).

Possiamo poi aggiungere delle condizioni ai raggruppamenti. Ad esempio, se volessimo solo i padri i cui figli hanno un reddito medio ≥ 25, possiamo aggiungere questa condizione scrivendo **`HAVING AVG(f.Reddito) >= 25`**. Non possiamo scrivere questa condizione con il `WHERE` perché questo opera sulle singole tuple, dunque aggiungergli una condizione con un operatore aggregato in mezzo non ha senso.

Operatori insiemistici

Vediamo ora tutti gli operatori insiemistici in SQL che già abbiamo accennato in algebra relazionale. Ricordiamo che queste operazioni sono effettuabili solo tra tabelle *union compatibili*.

Per effettuare un'unione aggiungiamo la parola chiave **`UNION`** tra due query. Ad esempio, la query

```
SELECT Padre, Figlio
FROM paternita
UNION
SELECT Madre, Figlio
FROM maternita
```

restituirà una relazione con tutti i padri e i rispettivi figli e tutte le madri e i rispettivi figli. Notiamo però un problema: che nome diamo al primo attributo? Madre o Padre? Per convenzione viene sempre data la priorità ai nomi degli attributi del primo operando, dunque avremo come risultato una relazione con attributi Padre e Figlio. In questo caso, dato che Padre Figlio non ha molto senso per la tabella risultante, dovremmo effettuare una ridenominazione sulla prima relazione, scrivendo `SELECT Padre AS Genitore, Figlio` al posto del semplice `SELECT Padre, Figlio`.
N.B. L'ordine degli attributi è importante: non aveva senso scrivere `SELECT Figlio, Padre AS Genitore` e poi `SELECT Madre, Figlio`, perché avremmo avuto una relazione con tutte le madri nella colonna dei figli e tutti i figli nella colonna dei genitori.

Per effettuare una differenza invece aggiungiamo la parola chiave **`EXCEPT`** tra due query. Anche qui, come per l'unione, i nomi degli attributi della relazione finale saranno quelli ereditati dalla prima relazione. Ricordiamo che, come nell'algebra relazionale, le differenze in SQL NON sono commutative. Ad esempio, la query

```
SELECT Nome
FROM Impiegato
EXCEPT
SELECT Cognome
FROM Impiegato
```

restituisce tutti i nomi di impiegati che non sono cognomi di qualche impiegato.

Infine, per effettuare un'intersezione aggiungiamo la parola chiave **`INTERSECT`** tra due query.

Interrogazioni nidificate

È possibile anche effettuare una **query in una query**. Per capire come funzionano vediamo un esempio. Ipotizziamo di voler trovare il nome e il reddito del padre di Franco. Possiamo scrivere

```
SELECT Nome, Reddito
FROM Persone, Paternita
WHERE Nome = Padre AND Figlio = 'Franco'
```

ma possiamo anche scriverla in modo equivalente attraverso le **interrogazioni nidificate**, in questo modo:

```
SELECT Nome, Reddito
FROM Persone
WHERE Nome = (SELECT Padre
              FROM Paternita
              WHERE Figlio = 'Franco')
```

In questo caso abbiamo scritto Nome = query_interna. Questo funziona perché figlio nella relazione Paternita è chiave, dunque sappiamo che da quella query uscirà al più un valore. Se invece avessimo come risultato una relazione con più di una tupla, non avremmo dovuto scrivere = ma avremmo dovuto utilizzare l'operatore di appartenenza **= ANY** o **IN** (che si traducono entrambi in "appartiene a"). Se invece volessimo tutti i Nomi che NON appartengono alla relazione restituita dalla query interna dovremmo scrivere **<> ALL** o **NOT IN** (che si traducono in "non appartiene a").

Proprietà 4.2 – Regole delle interrogazioni nidificate

- Le query più interne NON possono contenere al proprio interno degli operatori insiemistici
- Non si può fare riferimento a variabili definite in blocchi più interni (con ad esempio nomi temporanei definiti con l'operatore `as`), ma si può fare riferimento a variabili definite in blocchi più esterni

Le interrogazioni nidificate ci permettono di risolvere anche query che prima non sapevamo come gestire. Se ad esempio vogliamo i nomi di tutte le persone con il reddito massimo, possiamo scrivere semplicemente

```
SELECT *
FROM Persone
WHERE Reddito = (SELECT MAX(Reddito) FROM Persone)
```

Proprietà 4.3 – Tipi di query annidate

Distinguiamo due tipi di query annidate:

- Query **subordinata** (o **correlata**), dove la query interna dipende da una o più colonne di quella esterna. La query interna viene qui eseguita riga per riga per ogni record della query principale (ossia quella esterna).
- Query **coordinata** (o **non correlata**), dove le query interne ed esterne sono totalmente indipendenti. La query interna qui viene eseguita una sola volta, e il risultato viene poi utilizzato dalla query principale

Proposizione 4.1

Non è possibile sfruttare la nidificazione con **having**.

Ad esempio, la query

```
SELECT Dipartimento
FROM Impiegato
GROUP BY Dipartimento
HAVING SUM(Stipendio) >= ALL (SELECT SUM(Stipendio)
                              FROM Impiegato
                              GROUP BY Dipartimento)
```

è errata, ed andrebbe risolta creando una **nuova view**, che si fa in questo modo:

```
CREATE VIEW Budget(Dip,TotStipendi) AS
    SELECT Dipartimento, SUM(Stipendio)
    FROM Impiegato
    GROUP BY Dipartimento
```

e facendo poi la query

```
SELECT Dip
FROM Budget
WHERE TotStipendi = (SELECT MAX(TotStipendi)
                     FROM Budget)
```

Proposizione 4.2

La nidificazione di operatori aggregati **non è ammessa**.

Ad esempio, la query

```
SELECT AVG(COUNT(DISTINCT Ufficio))
FROM Impiegato
GROUP BY Dipartimento
```

è errata, e va sempre risolta creando una nuova view:

```
CREATE VIEW DipartUffici(NomeDip, NumeroUffici) as
    SELECT Dipartimento, COUNT(DISTINCT Ufficio)
    FROM Impiegato
    GROUP BY Dipartimento
```

e facendo poi la query

```
SELECT AVG(NumeroUffici)
FROM DipartUffici
```

4.4 DML

Una volta compreso come creare un database e come interrogarne i dati bisogna passare allo step successivo, e capire come **manipolare i dati** di un database. Per fare ciò abbiamo a disposizione tre diverse operazioni:

- **`INSERT`**, per **inserire i dati** in una o più relazioni
- **`DELETE`**, per **eliminare i dati** di una o più relazioni
- **`UPDATE`**, per **modificare i dati** di una o più relazioni (già accennato in precedenza quando abbiamo visto il comando `ON UPDATE CASCADE`)

`INSERT`

Per inserire una tupla `Valore` in una relazione `Tabella` già esistente basta scrivere

```
INSERT INTO Tabella (Attributi)
VALUES(Valore)
```

dove `Attributi` sono alcuni degli attributi della relazione `Tabella` (è possibile anche non specificare questa tupla di `Attributi`: in tal caso verranno presi tutti gli attributi della relazione nell'ordine descritto dallo schema). La tupla `Valore` deve avere valori nello stesso numero e ordine di quelli in `Attributi`, ma non è necessario che questi siano nello stesso ordine definiti

nella `CREATE TABLE`. Infatti, è possibile anche saltare alcuni attributi definiti nella relazione: questi verranno segnati con un valore di default se presente o come `NULL`. Se però saltiamo un attributo definito come `NOT NULL` ci verrà restituito un errore. Ad esempio non possiamo inserire una tupla che abbia come valore della chiave `NULL`.

È possibile anche inserire una o più tuple da altre relazioni tramite una query:

```
INSERT INTO Persone
SELECT Nome, Eta
FROM Padre
WHERE Eta > 50
```

Ad esempio, con questa query stiamo inserendo nella tabella Persone tutti i padri con un'età > 50 anni. Dato che abbiamo selezionato `Nome` e `Eta` nella query, nella tabella Persone dovremo avere questi due attributi con questo ordine e, nel caso, possiamo avere solo attributi con default o che ammettono il valore `NULL`.

DELETE

Per eliminare invece una serie di tuple di una tabella che rispondono a una certa condizione basta scrivere

```
DELETE FROM Tabella
WHERE Condizione
```

Nel caso si vogliano eliminare tutte le tuple di una relazione, basta scrivere semplicemente `DELETE FROM Tabella`, senza alcuna condizione (che per SQL equivale a dire `WHERE TRUE`), come già accennato in precedenza. Ovviamente, se una tupla è presente come foreign key in un'altra relazione ed è specificata una politica `CASCADE`, la sua eliminazione può modificare o eliminare delle tuple in altre relazioni.

UPDATE

Per aggiornare una serie di tuple di una tabella che rispondono a una certa condizione modificandone un attributo con un nuovo valore basta scrivere

```
UPDATE Tabella
SET Attributo = Espressione
WHERE Condizione
```

L'espressione definita può essere un valore predefinito, come un numero o una stringa, ma può essere anche una sottoquery. Ad esempio, è totalmente lecito scrivere

```
UPDATE Impiegato
SET Stipendio = (SELECT AVG(Stipendio) FROM Impiegato)
WHERE Stipendio < 1000
```

per impostare tutti gli stipendi minori di 1000 pari alla media degli stipendi di questa stessa tabella.

Anche qui, nel caso si vogliano modificare tutte le tuple di una relazione cambiando un attributo con lo stesso valore per tutte basta non porre nessuna condizione.

4.5 DCL

Passiamo ora all'ultimo sottolinguaggio di SQL, ossia **DCL**, dedito alla gestione dei permessi di accesso ai dati. Tramite questo linguaggio è possibile specificare chi può accedere a quali dati e con quali permessi.

Definizione 4.14 – Privilegio

Un **privilegio** è un diritto di accesso che un utente possiede per poter accedere a una certa risorsa, che può essere una tabella ma anche un singolo attributo/dominio o una vista.

Proprietà 4.4 – Tipi di privilegi

Esistono diversi tipi di privilegi:

- **`INSERT`**, che permette di inserire nuovi oggetti
- **`UPDATE`**, che permette di modificare il contenuto di oggetti
- **`DELETE`**, che permette di eliminare oggetti
- **`SELECT`**, che permette di leggere la risorsa selezionata
- **`REFERENCES`**, che permette la definizione di nuovi vincoli sulla risorsa selezionata
- **`USAGE`**, che permette l'utilizzo della risorsa selezionata (ad esempio l'utilizzo di un dominio creato)

Per concedere uno o più privilegi a un utente su una risorsa bisogna usare la parola chiave **`GRANT`**, scrivendo ad esempio `GRANT Privilegi ON Risorsa TO Utente`. Inoltre, se si vuole che l'utente a cui garantiamo i privilegi possa a sua volta concedere questi stessi privilegi a qualcun altro, dobbiamo aggiungere la scritta **`WITH GRANT OPTION`**.
Per revocare invece uno o più privilegi a un utente su una risorsa bisogna usare la parola

chiave **REVOKE**, scrivendo ad esempio `REVOKE Privilegi ON Risorsa FROM Utente`. Oltre a ciò, se all'utente sono stati dati anche i permessi per poterli concedere lui stesso ad altri utenti, bisogna anche specificare **RESTRICT** se si vuole annullare la revoca nel caso l'utente a cui stiamo revocando i permessi li abbia concessi a qualcun altro, o specificare **CASCADE** se si vuole revocare i permessi anche a tutti gli utenti che li hanno ricevuti da colui che li sta perdendo.

4.6 DTL

Esiste in realtà un altro sottolinguaggio di SQL con un solo obiettivo: effettuare delle **transazioni**, e prende proprio il nome di **DTL** (Data Transaction Language).

Definizione 4.15 – Transazione

Una **transazione** è una sequenza di operazioni che deve rispettare una serie di proprietà:

- **Atomicità**: le operazioni devono essere eseguite SEMPRE tutte insieme. Se una transazione viene annullata a metà, tutte le operazioni eseguite al suo interno devono essere annullate
- **Consistenza**: al termine di ogni istruzione non deve essere violato alcun vincolo di integrità
- **Isolamento**: le transazioni devono essere indipendenti fra loro
- **Durabilità**: una transazione completata correttamente deve riportare risultati durabili nel tempo

Per effettuare una transazione in SQL esistono tre istruzioni fondamentali:

- **`BEGIN TRANSACTION`**, che specifica l'inizio della transazione (in questa fase le operazioni non vengono ancora eseguite sulla base di dati)
- **`COMMIT WORK`**, che conclude la transazione (rendendo definitive tutte le operazioni a partire da `BEGIN TRANSACTION`)
- **`ROLLBACK WORK`**, che rinuncia completamente alla transazione e riporta la base di dati allo stato precedente alla transazione

Ad esempio una transazione classica viene effettuata quando si passa un tot di soldi da un saldo a un altro, scrivendo in questo modo:

```
BEGIN TRANSACTION
UPDATE ContoCorrente
    SET Saldo = Saldo - 10
    WHERE NumeroConto = 10000
UPDATE ContoCorrente
    SET Saldo = Saldo + 10
    WHERE NumeroConto = 55555
```

```
COMMIT WORK
```

4.7 DBMS attivi

N.B. La sintassi delle operazioni sui DBMS Attivi varia da DBMS a DBMS. Daremo quindi dei comandi generali che poi andranno approfonditi a seconda del DBMS utilizzato.

Tutti i DBMS che abbiamo visto fino ad ora sono detti database **passivi**. Questo perché reagiscono SOLO a richieste effettuate dagli utenti. Spesso, però, si ha necessità di far reagire il DBMS in modo automatico ad alcuni eventi, e per questo motivo nascono i **DBMS attivi**.

Definizione 4.16 – DBMS attivo

Un DBMS viene detto **attivo** quando consente la definizione e gestione di **trigger**. Questi trigger vengono attivati in modo automatico al verificarsi di eventi specifici sulla base di dati e dopo aver verificato eventuali condizioni, e nell'attivarsi eseguono delle operazioni sulla base di dati.
Il paradigma che adottiamo per gestire i trigger è il paradigma **ECA** (Evento-Condizione-Azione).

Tra gli eventi possibili che possono attivare un trigger ci sono i vari `INSERT`, `DELETE`, `UPDATE` o `UPDATE OF Lista attributi`. In particolare, se si utilizza come evento `UPDATE OF Lista attributi`, il trigger si attiverà se e solo se l'evento modifica tutti e soli gli attributi della lista.

I comandi di SQL per gestire una base di dati attiva sono i seguenti:

- **`CREATE TRIGGER`**, per **creare** un nuovo trigger
- **`DROP TRIGGER`**, per **eliminare** un trigger
- **`ALTER TRIGGER`**, per **modificare** un trigger (supportato da alcuni DBMS)

La struttura generale per creare un trigger in SQL è la seguente:

```
CREATE TRIGGER Nome
{BEFORE/AFTER} Evento ON Relazione
REFERENCING OLD ROW AS Variabile / NEW ROW AS Variabile /
OLD TABLE AS Variabile / NEW TABLE AS Variabile
FOR EACH ROW/STATEMENT
WHEN Condizione
Comandi SQL da eseguire
```

Ci sono tante cose da analizzare qui:

- **`BEFORE/AFTER`** ci permettono di eseguire operazioni SQL sia prima sia dopo un evento
- **`REFERENCING`** ci permette di dare dei nomi a vecchie/nuove righe/tabelle per facilitare la scrittura delle condizioni. Nel caso questo non venga effettuato, si può scrivere **`NEW.`** o **`OLD.`** per riferirsi a righe/tabelle nuove/vecchie.
- **`FOR EACH ROW/STATEMENT`** è particolare: bisogna utilizzare `FOR EACH ROW` se si ha un trigger che modifica il valore delle singole tuple che vengono modificate, mentre bisogna usare `FOR EACH STATEMENT` se c'è un'azione che modifica globalmente tutte le tuple. **N.B.** Con il `FOR EACH STATEMENT` si potrà dare un nome usando `REFERENCING` solo alle tabelle vecchie/nuove, e non alle singole righe.

È possibile porre delle condizioni sia sulle righe/tabelle vecchie sia su quelle nuove grazie alle cosiddette **tabelle di transizione**, che congelano i dati prima che l'evento venga eseguito e li salvano in una nuova tabella. Ovviamente tutto ciò ha senso se il trigger si attiva dopo che l'evento che lo ha scatenato viene eseguito: se il trigger si attiva prima dell'evento non esiste nessuna tabella con all'interno i dati post-evento, dato che l'evento deve ancora avvenire.

4.7.1 Relazione tra trigger e vincoli d'integrità

Possiamo vedere i trigger come dei vincoli più generali: i vincoli ci permettono di fare dei controlli su delle operazioni, mentre i trigger ci permettono non solo di fare dei controlli, ma anche di effettuare delle operazioni in risposta a queste operazioni. Questa flessibilità in più può essere sia un vantaggio che uno svantaggio: è un vantaggio perché ci permette di muoverci come vogliamo in base alla situazione creata, ma è uno svantaggio perché ogni volta che creiamo un trigger dobbiamo definire come il DBMS deve muoversi quando registra un evento e rispetta delle condizioni.

Vediamo ora un esempio in cui abbiamo dei trigger e dei vincoli che lavorano insieme. Ipotizziamo di avere i seguenti trigger:

```
CREATE TRIGGER Trigger1 AFTER UPDATE ON Tabella1...;
CREATE TRIGGER Trigger2 BEFORE UPDATE ON Tabella1...;
CREATE TRIGGER Trigger3 AFTER UPDATE ON Tabella1...;
```

Decidiamo ora di modificare la nostra Tabella1, in questo modo:

```
ALTER TABLE Tabella1 ADD CONSTRAINT Vincolo1...;
```

A questo punto, quando arriva un `UPDATE` sulla Tabella1, cosa succede? Innanzitutto, il Trigger2 viene attivato perché definito come BEFORE. Viene poi eseguito l'UPDATE, viene controllato il

nuovo Vincolo1 che abbiamo aggiunto, viene poi attivato il Trigger1 (perché scritto prima del Trigger3) e infine il Trigger3.

Dunque, in generale, viene seguito questo ordine:

1° Trigger BEFORE STATEMENT
2° Per ogni tupla:
   - TRIGGER BEFORE ROW
   - Evento sulla riga e verifica dei vincoli sulla riga
   - TRIGGER AFTER ROW
3° Verifica dei vincoli sul comando completato
4° Trigger AFTER STATEMENT

e, come già visto nell'esempio, se esistono più trigger dello stesso tipo si considera l'ordine di creazione.

4.7.2 Quando i trigger sono veramente utili

Vediamo ora un esempio in cui i trigger diventano veramente utili. Ipotizziamo di avere una Warehouse nel nostro DB:

Warehouse

Part	QtyAvbl	QtyLimit	QtyReord
1	200	150	100
2	780	500	200
3	450	400	120

dove Part è l'identificativo dell'articolo, QtyAvbl sono i pezzi disponibili, QtyLimit è il numero minimo di pezzi da avere in magazzino e QtyReord è il numero di pezzi da riordinare se QtyAvbl scende sotto il limite imposto da QtyLimit. Vogliamo quindi un trigger che si attiva ogni volta che QtyAvbl scende sotto QtyLimit e che effettui un riordine di un numero di pezzi pari a QtyReord.
Creiamo quindi il seguente trigger:

```
CREATE TRIGGER Reorder
AFTER UPDATE OF QtyAvbl ON Warehouse
FOR EACH ROW
WHEN(NEW.QtyAvbl < NEW.QtyLimit)
DECLARE X Number
BEGIN
SELECT COUNT(*) INTO X
```

```
FROM PendingOrders
WHERE Part = NEW.Part;
IF X = 0
THEN
    INSERT INTO PendingOrders
    VALUES(NEW.Part, NEW.QtyReord, sysDate)
END IF;
END;
```

Notiamo intanto quattro cose:

- **DECLARE** dichiara una nuova variabile temporanea
- **BEGIN-END** racchiudono tutti i comandi SQL da eseguire quando il trigger viene attivato
- **;** è necessario per separare le diverse query/istruzioni che il trigger attiva
- **IF - THEN - END IF** servono per eseguire un'istruzione solo quando la condizione nell'if è rispettata

Riprendendo ora la tabella Warehouse, ipotizziamo di effettuare la seguente transazione:

```
UPDATE Warehouse
SET QtyAvbl = QtyAvbl - 70
WHERE Part = 1
```

Dato che non abbiamo nessun trigger BEFORE, effettuiamo subito l'operazione e otteniamo come risultato la tabella

Warehouse

Part	QtyAvbl	QtyLimit	QtyReord
1	130	150	100
2	780	500	200
3	450	400	120

A questo punto parte il nostro trigger, che verifica ogni riga della nostra tabella. Dato che nella prima tupla abbiamo effettivamente QtyAvbl < QtyLimit, la condizione è rispettata e vengono eseguite le operazioni prestabilite dal trigger. Aggiungiamo quindi alla nostra tabella PendingOrders un nuovo ordine per il nostro articolo con codice Part = 1. Se adesso effettuiamo un'altra transazione di questo tipo

```
UPDATE Warehouse
SET QtyAvbl = QtyAvbl - 70
WHERE Part = 1
```

il nostro trigger si attiverà lo stesso, la condizione sarà verificata, ma dato che avremo X = 1 non verrà inserito nessun nuovo ordine. In questo modo evitiamo di effettuare più di un ordine di restock.

Capitolo 5

Dipendenze funzionali

5.1 Cosa sono le dipendenze funzionali?

Abbiamo visto fino ad ora due tipi di vincoli:

- Vincoli **statici**, definiti tramite `CHECK` o `CONSTRAINT`
- Vincoli **dinamici** (solo per le basi di dati attive), definiti tramite `TRIGGER`

Aggiungiamo ora un terzo tipo di vincolo estremamente importante: le **dipendenze funzionali**.

Definizione 5.1 – Dipendenza funzionale

Dato uno schema R, una **dipendenza funzionale** su R è una coppia ordinata di sottoinsiemi non vuoti X e Y di attributi di R che stabilisce un particolare legame tra X e Y. Per indicare tale vincolo, scriviamo $\boldsymbol{X \longrightarrow Y}$. X prende il nome di **determinante**, mentre Y prende il nome di **dipendente**.
Indichiamo con $\boldsymbol{F}$ l'insieme delle dipendenze funzionali su uno schema, scrivendo quindi $F = \{X \longrightarrow Y, A \longrightarrow B, \ldots\}$.

Questi vincoli non sono matematici ma sono più dettati dal buon senso: ad esempio, in una tabella che registra diverse automobili con modello e casa di produzione segnate, ha senso creare una dipendenza funzionale che implichi che *le macchine dello stesso modello sono sempre prodotte dalla stessa casa automobilistica*, dato che non esistono due case automobilistiche differenti che producano automobili con lo stesso modello.
Ovviamente questi vincoli variano in base alle nostre specifiche: se abbiamo una tabella che gestisce dei voli, possiamo porre come vincolo che un volo con un certo codice parta sempre alla stessa ora solo se questo, per i nostri voli, è sempre vero, altrimenti non ha proprio senso inserire questa dipendenza funzionale.

Osservazione 5.1

Se si ha $X \longrightarrow Y$, non è detto che si abbia anche $Y \longrightarrow X$.

Banalmente, se tutti i voli con lo stesso codice partono alla stessa ora, non è detto che tutti i voli che partono alla stessa ora abbiano lo stesso codice.

5.2 Implementazione in SQL

L'implementazione delle dipendenze funzionali in SQL avviene tramite **TRIGGER**. Ad esempio, per implementare il vincolo riguardo i voli che abbiamo visto poco fa possiamo creare il seguente trigger:

```
CREATE TRIGGER Check_Ora
BEFORE INSERT OR UPDATE ON Voli
FOR EACH ROW
BEGIN
IF EXISTS(SELECT 1 FROM Voli WHERE CodiceVolo = NEW.CodiceVolo AND Ora <> NEW.Ora)
THEN
    RAISE APPLICATION ERROR 'Errore: Il volo % deve avere sempre lo stesso orario',
        NEW.CodiceVolo
END IF;
END;
```

Anche qui analizziamo alcune cose che non abbiamo mai incontrato:

- **`EXISTS(SELECT 1 FROM ...)`** restituisce TRUE se la sottoquery trova almeno una riga che rispetta la condizione definita (il valore 1 è solo un segnaposto: a `EXISTS` non interessa cosa viene selezionato, ma solo se esiste almeno una riga)
- **`RAISE APPLICATION ERROR`** alza un errore e manda in output un messaggio per far capire di che errore si tratta
- **`%`** serve come segnaposto per la variabile definita dopo la virgola, ossia `NEW.CodiceVolo`

In realtà, se le nostre tabelle sono ben progettate (ossia sono in Terza Forma Normale (3NF), che vedremo in seguito), quasi tutte le dipendenze funzionali sono soddisfatte senza dover creare dei trigger appositi.

5.3 Soddisfare una dipendenza funzionale

Definizione 5.2 – Soddisfare una dipendenza funzionale

Una relazione r (ossia un'istanza di R) con schema R **soddisfa** la dipendenza funzionale $X \longrightarrow Y$ se:

1) La dipendenza funzionale $X \longrightarrow Y$ è **applicabile** a R, ossia sia X che Y sono effettivamente dei sottoinsiemi di R
2) Prese due ennuple t_1, t_2 tali che $t_1[X] = t_2[X]$, ossia che hanno stesso valore su X (diciamo quindi che *concordano* su X), allora anche $t_1[Y] = t_2[Y]$.

Osservazione 5.2

- Le istanze **vuote** soddisfano tutte le dipendenze
- Le istanze **con una sola tupla** soddisfano tutte le dipendenze

Definizione 5.3 – Istanze legali

Un'istanza di uno schema R è detta **legale** se soddisfa tutte le dipendenze funzionali definite su R.

Vediamo ora un esempio per capire meglio questi concetti.
N.B. Useremo d'ora in avanti una notazione *astratta*, ossia utilizzeremo uno schema e delle istanze con delle lettere generiche di cui non ci interessa conoscere il significato.

Prendiamo un'istanza r dello schema R:

r

A	B	C	D
a_1	b_1	c_1	d_1
a_3	b_1	c_1	d_1
a_2	b_1	c_1	d_1
a_2	b_1	c_2	d_1

Questa relazione rispetta ad esempio la dipendenza funzionale $A \longrightarrow B$ e la dipendenza $B \longrightarrow D$, ma non rispetta la dipendenza $A \longrightarrow C$. Infatti, la terza tupla ha come valori di A e di C rispettivamente a_2 e c_1, mentre la quarta ha a_2 e c_2, dunque troviamo che $t_1[A] = t_2[A]$ ma $t_1[C] \neq t_2[C]$. Definiamo quindi $F = \{A \longrightarrow B, B \longrightarrow D\}$

Osservazione 5.3

Nel caso in cui abbiamo $X \cup Y = R$, presa una relazione r ci basta trovare due tuple t_1, t_2 tali che $t_1[X] = t_2[X]$ per dimostrare che tale istanza non è legale.

Questo è vero perché, dato che $X \cup Y = R$, se $t_1[X] = t_2[X]$ è impossibile avere anche $t_1[Y] = t_2[Y]$, altrimenti avremmo due tuple completamente identiche, il che non è ammesso.

Prendiamo ora un'altra istanza s dello schema R visto prima:

s

A	B	C	D
a_1	b_1	c_1	d_1
a_3	b_1	c_1	d_1
a_2	b_1	c_1	d_1
a_4	b_1	c_1	d_1

Questa nuova istanza rispetta non solo tutte le dipendenze funzionali in F, ma anche la dipendenza funzionale $A \longrightarrow C$ ad esempio, non rispettata da r. Questo significa che $A \longrightarrow C$ dovrebbe essere in F? Assolutamente no: questo è semplicemente un caso. È infatti possibile che in futuro venga aggiunta una nuova tupla, come ad esempio (a_1, b_1, c_2, d_1) che romperebbe completamente questa dipendenza fittizia che abbiamo ipotizzato.

Ora, guardando bene F notiamo una cosa interessante: nonostante $A \longrightarrow D$ non sia presente in F, questa dipendenza è rispettata da tutte le istanze possibili dello schema R, a causa delle altre due dipendenze $A \longrightarrow B$ e $B \longrightarrow D$ (notiamo quindi come la dipendenza funzionale sia una relazione **transitiva**). Questo, però, non modifica in alcun modo F: $A \longrightarrow D$ rimane fuori. Per questo motivo nasce un nuovo insieme di dipendenze funzionali, l'insieme $\boldsymbol{F^+}$.

Definizione 5.4 – F^+

Preso uno schema R, $\boldsymbol{F^+}$ è l'insieme di dipendenze funzionali che ogni istanza legale di R rispetta.

Proposizione 5.1 – Corollario

$\boldsymbol{F \subseteq F^+}$

Nell'esempio di prima avremo quindi $F^+ = \{A \longrightarrow B, B \longrightarrow D, A \longrightarrow D\}$. Ricordiamo sempre che ad esempio $C \longrightarrow D$ non è presente in F^+ nonostante sia rispettata sia da r che da s perché è semplicemente un caso che questo sia accaduto. È possibilissimo che un domani venga aggiunta

in r o in s una riga che rompa questa dipendenza, mantenendo allo stesso tempo l'istanza completamente legale.

Proposizione 5.2 – Dipendenza funzionale banale

Dato uno schema R e due sottoinsiemi non vuoti X, Y di R, $Y \subseteq X \implies X \longrightarrow Y$. Questo tipo di dipendenze prendono il nome di **dipendenze funzionali banali**.

Ad esempio $ABC \longrightarrow AB$ è una dipendenza funzionale banale.

5.4 Definizione di chiave

Finalmente, grazie al concetto di dipendenza funzionale, ci è possibile dare una definizione formale del concetto di chiave.

Definizione 5.5 – Chiave

Una **chiave** è definita come un attributo o un insieme di attributi K sottoinsieme di uno schema R che rispetta queste due proprietà:

1) $\boldsymbol{K \longrightarrow R \in F^+}$
2) $\boldsymbol{\nexists K' \subset K \text{ t.c. } K' \longrightarrow R \in F^+}$

La prima proprietà ci dice quindi che con K risuciamo a identificare univocamente una tupla in un'istanza di R, mentre la seconda proprietà ci dice che K deve essere **minimale** (e non minimo, dato che possono esistere più chiavi possibili all'interno di uno stesso schema R).
Una lista di attributi K che rispetta solo la prima proprietà ma non la seconda prende il nome di **superchiave**.

N.B. In SQL sappiamo che dobbiamo scegliere una chiave tra quelle possibili, che prende il nome di **chiave primaria** e non può assumere il valore NULL. Le altre chiavi, però, non devono essere ignorate ma devono essere dichiarate `UNIQUE`.

5.5 Un altro insieme di dipendenze funzionali

Calcolare F^+ è tutto tranne che semplice: infatti, sappiamo che tutte le dipendenze funzionali che sono in F sono anche in F^+, ma come facciamo a trovare tutte quelle che non lo sono? Per risolvere questo problema introduciamo un nuovo insieme di dipendenze funzionali, più semplice da calcolare, ossia $\boldsymbol{F^A}$.

Definizione 5.6 – F^A

Chiamiamo $\boldsymbol{F^A}$ l'insieme delle dipendenze funzionali di uno schema R tale che $\boldsymbol{F \subseteq F^A}$ e che rispetta i seguenti assiomi (che prendono il nome di **assiomi di Armstrong**):

1) $Y \subseteq X \implies X \longrightarrow Y \in F^A$ (**assioma della riflessività**, che inserisce in F^A tutte le dipendenze funzionali banali di R)
2) $X \longrightarrow Y \in F^A \implies XZ \longrightarrow YZ \in F^A \ \forall Z \subseteq R$ (**assioma dell'aumento**)
3) $X \longrightarrow Y \in F^A$ e $Y \longrightarrow Z \in F^A \implies X \longrightarrow Z \in F^A$ (**assioma della transitività**, che avevamo già accennato nel Corollario 5.1)

Gli ultimi due assiomi sono detti **ricorsivi**. Infatti, in questi usiamo delle dipendenze che già sappiamo essere in F^A per trovarne di nuove.

Proprietà 5.1 – Altri assiomi di Armstrong

Esistono altri 3 assiomi di Armstrong per trovare delle dipendenze funzionali in F^A:

4) $X \longrightarrow Y \in F^A$ e $X \longrightarrow Z \in F^A \implies X \longrightarrow YZ \in F^A$ (**regola dell'unione**)
5) $X \longrightarrow Y \in F^A$ e $Z \subseteq Y \implies X \longrightarrow Z \in F^A$ (**regola della decomposizione**)
6) $X \longrightarrow Y \in F^A$ e $WY \in Z \in F^A \implies WX \longrightarrow Z \in F^A$ (**regola della pseudotransitività**)

Queste regole ci rendono il calcolo di F^A più semplici, ma non sono necessarie per definire F^A stessa perché tutte e tre derivano dai tre assiomi visti prima.

Dimostrazione

4) Se $X \longrightarrow Y \in F^A$, per l'assioma dell'aumento abbiamo $X \longrightarrow XY \in F^A$, e allo stesso modo, se $X \longrightarrow Z \in F^A$, allora $XY \longrightarrow YZ \in F^A$. A questo punto, per l'assioma della transitività, troviamo che $X \longrightarrow YZ \in F^A$. ■
5) Se $Z \subseteq Y$, per l'assioma della riflessività, abbiamo che $Y \longrightarrow Z \in F^A$. Avendo anche $X \longrightarrow Y \in F^A$, troviamo per l'assioma della transitività che $X \longrightarrow Z \in F^A$. ■
6) Se $X \longrightarrow Y \in F^A$, per l'assioma dell'aumento abbiamo $WX \longrightarrow WY \in F^A$. Avendo anche $WY \longrightarrow Z \in F^A$, troviamo per l'assioma della transitività che $WX \longrightarrow Z \in F^A$. ■

Osservazione 5.4

Preso uno schema R e due sottoinsiemi non vuoti X e Y di R, dove $Y = A_1, \dots, A_n$, $\boldsymbol{X \longrightarrow Y \in F^A \iff X \longrightarrow A_1 \in F^A, \dots, X \longrightarrow A_n \in F^A}$.

Dimostrazione

- $X \longrightarrow Y \in F^A \implies X \longrightarrow A_1 \in F^A, \ldots, X \longrightarrow A_n \in F^A$
  Banalmente, per la regola della decomposizione, se $X \longrightarrow A_1, \ldots, A_n \in F^A$ allora $X \longrightarrow A_i \in F^A \ \forall i \in \{1, \ldots, n\}$. ■
- $X \longrightarrow Y \in F^A \impliedby X \longrightarrow A_1 \in F^A, \ldots, X \longrightarrow A_n \in F^A$
  Banalmente, per la regola dell'unione, se $X \longrightarrow A_i \in F^A \ \forall i \in \{1, \ldots, n\}$, allora $X \longrightarrow A_1, \ldots, A_n \in F_A$ (notiamo come la dimostrazione di questo secondo passaggio sia identica a quella del primo ma invertita). ■

Definizione 5.7 – Chiusura di un insieme di attributi

Preso uno schema R e un sottoinsieme di attributi X di R, la **chiusura** di X rispetto a un insieme di dipendenze funzionali F è definita nel seguente modo:

$$\boldsymbol{X_F^+ = \{A \mid X \longrightarrow A \in F^A\}}$$

È possibile anche scrivere X^+ e non X_F^+ quando F nel contesto in cui ci si trova è scontata.
N.B. Questa definizione serve per trovare tutti gli attributi determinati da X. Per fare ciò, dovremmo segnare $X \longrightarrow A \in F^+$, e non in F^A, ma dato che tra poco dimostreremo che $F^+ = F^A$, scriviamo in questa definizione e anche nel prossimo lemma di chiusura direttamente F^A.

Proposizione 5.3 – Lemma di chiusura

Preso uno schema R e un insieme di dipendenze funzionali F su R, $\boldsymbol{X \longrightarrow Y \in F^A \iff Y \subseteq X^+}$.

Dimostrazione

Poniamo per entrambi i punti della dimostrazione $Y = A_1, \ldots, A_n$.

- $X \longrightarrow Y \in F^A \implies Y \subseteq X^+$
  Banalmente, dato che $Y \subseteq X^+$, $\forall i \in \{1, \ldots, n\}$ si ha che $X \longrightarrow A_i \in F^A$. Dunque per la regola dell'unione troviamo che $X \longrightarrow Y \in F^A$. ■
- $X \longrightarrow Y \in F^A \impliedby Y \subseteq X^+$
  Banalmente, dato che $X \longrightarrow Y \in F^A$, per la regola della decomposizione abbiamo che, $\forall i \in \{1, \ldots, n\} \ X \longrightarrow A_i \in F^A$, ossia $A_i \in X^+$, dunque $Y \in X^+$ (notiamo anche qui come la dimostrazione di questo secondo passaggio sia identica a quella del primo ma invertita). ■

Proposizione 5.4 – Teorema

Preso uno schema R e un insieme di dipendenze funzionali F su R, $\boldsymbol{F^+ = F^A}$.

Dimostrazione

Dividiamo questa dimostrazione in due parti: nella prima dimostreremo che $F^A \subseteq F^+$, mentre nella seconda dimostreremo che $F^+ \subseteq F^A$.

- $F^A \subseteq F^+$
  Sia $X \longrightarrow Y$ una dipendenza funzionale in F^A. Essendo questa in F^A, sappiamo per gli assiomi di Armstrong che questa dipendenza si può costruire da un'altra dipendenza in F seguendo un numero i di passi. Dimostriamo ora per induzione su i che $F^A \subseteq F^+$.
  - Caso base: $i = 0$
    In questo caso $X \longrightarrow Y \in F$ e quindi $X \longrightarrow Y \in F^+$.
  - Ipotesi induttiva forte
    Assumiamo che qualsiasi dipendenza funzionale ottenuta partendo da una in F e applicando gli assiomi di Armstrong al più $i - 1$ volte sia in F^+.
  - Passo induttivo
    Abbiamo qui tre casi da analizzare:
    1° Caso: $X \longrightarrow Y$ è ottenuta mediante l'assioma della riflessività.
    In questo caso abbiamo $Y \subseteq X$, dunque, prese due tuple t_1 e t_2 di r istanza legale di R, troviamo che $t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$, ossia $X \longrightarrow Y \in F^+$.
    2° Caso: $X \longrightarrow Y$ è ottenuta mendiante l'assioma dell'aumento.
    In questo caso abbiamo un'altra dipendenza funzionale $V \longrightarrow W \in F^A$ da cui troviamo $X \longrightarrow Y$, ottenuta a sua volta applicando gli assiomi di Armstrong $i - 1$ volte. Dunque, per ipotesi induttiva, troviamo che $V \longrightarrow W \in F^+$. Abbiamo quindi $X = VZ$ e $Y = WZ$ con $Z \subseteq R$. Sia ora r un istanza legale di R e siano t_1 e t_2 due tuple in r tali che $t_1[X] = t_2[X]$. Abbiamo quindi $t_1[VZ] = t_2[VZ]$ ossia $t_1[V] = t_2[V]$ e $t_1[Z] = t_2[Z]$. Sappiamo che $t_1[V] = t_2[V] \implies t_1[W] = t_2[W]$, quindi troviamo che $t_1[WZ] = t_2[WZ]$, ossia $t_1[Y] = t_2[Y]$. Troviamo quindi che $X \longrightarrow Y \in F^+$.
    3° Caso: $X \longrightarrow Y$ è ottenuta mediante l'assioma della transitività.
    In questo caso abbiamo altre due dipendenze funzionali $X \longrightarrow Z$ e $Z \longrightarrow Y$ in F^A, ottenute applicando gli assiomi di Armstrong al più $i - 1$ volte. Dunque, per ipotesi induttiva forte, troviamo che $X \longrightarrow Z, Y \longrightarrow Z \in F^+$. Presa ora un'istanza legale r e due tuple t_1 e t_2 in r tali che $t_1[X] = t_2[X]$, sappiamo per la dipendenza $X \longrightarrow Z$ che $t_1[Z] = t_2[Z]$, e per la dipendenza $Z \longrightarrow Y$ sappiamo che $t_1[Y] = t_2[Y]$. Dunque troviamo che $X \longrightarrow Y \in F^+$.

Dato che in tutti e tre i casi trovimao che $X \longrightarrow Y \in F^+$, possiamo concludere che $F^A \subseteq F^+$. ■

- $F^+ \subseteq F^A$

  Presa una dipendenza funzionale $X \longrightarrow Y \in F^+$, dato che questa appartiene a F^+ sappiamo che è rispettata da una qualsiasi istanza legale di R. Per dimostrare che questa appartiene a F^A useremo una particolare istanza r di R, e cercheremo di dimostrare che questa è legale:

  r

X^+				$R - X^+$			
1	1	...	1	1	1	...	1
1	1	...	1	0	0	...	0

  Dimostrando che questa istanza è legale dimostreremo anche che se abbiamo $X \longrightarrow Y \in F^+$ allora avremo anche $X \longrightarrow Y \in F^A$, ossia $F^+ \subseteq F^A$.
  Sia $V \longrightarrow W$ una dipendenza funzionale in F. Abbiamo due casi:

  - $t_1[V] \neq t_2[V]$

    In questo caso la dipendenza è soddisfatta, dato che queste sono le uniche due tuple in r.

  - $t_1[V] = t_2[V]$

    In questo caso abbiamo $V \subseteq X^+$, dato che le due tuple sono uguali solo per gli attributi di X^+. In questo caso, per il Lemma di Chiusura, abbiamo che $X \longrightarrow Y \in F^A$, e dunque, per l'assioma di transitività, abbiamo che $X \longrightarrow W \in F^A$. Sempre per il lemma troviamo che $W \subseteq X^+$, dunque le due tuple t_1 e t_2 sono uguali anche sui valori di W. Dunque $V \longrightarrow W$ è soddisfatta.

  Poiché abbiamo soddisfatto $V \longrightarrow W$, che è una qualsiasi dipendenza funzionale in F, troviamo che r è un'istanza legale di R.
  Ora che abbiamo trovato che r è un'istanza legale di R troviamo che questa soddisfa la dipendenza funzionale $X \longrightarrow Y$, dato che questa appartiene a F^+ (ricordiamo che ogni dipendenza funzionale in F^+ è, per definizione, soddisfatta da qualsiasi istanza legale di R). Poiché $X \subseteq X^+$ (dato che $X \longrightarrow X$ è sempre soddisfatta), troviamo che $t_1[X] = t_2[X]$ e quindi, poiché r soddisfa $X \longrightarrow Y$, anche $t_1[Y] = t_2[Y]$ è verificata. Ciò significa che $Y \subseteq X^+$ e, per il lemma di chiusura, troviamo proprio che $X \longrightarrow Y \in F^A$. ■

Proposizione 5.5 – Corollario

Preso uno schema R, due sottoinsiemi X e Y di R e un insieme di dipendenze funzionali F, $\boldsymbol{X \longrightarrow Y \in F^+ \iff Y \subseteq X^+}$.

Questo corollario deriva proprio dal lemma di chiusura, che ci dice che $X \longrightarrow Y \in F^A \iff Y \subseteq X^+$, e dal teorema appena dimostrato che ci dice che $F^A = F^+$.

Proposizione 5.6 – Verificare se K sia una chiave o meno

Per verificare che K sia una chiave di uno schema R preso un insieme di dipendenze funzionali basta dimostrare che $\boldsymbol{R \subseteq K^+}$ e dimostrare che nessun sottoinsieme di K verifichi questa proprietà.

5.6 Calcolare la chiusura di un insieme di attributi

Per verificare dunque se $X \longrightarrow Y$ appartenga a F^+ e per verificare se K sia una chiave bisogna saper calcolare la chiusura di un insieme di attributi. Studiamo quindi un algoritmo per eseguire questo calcolo velocemente.

Per arrivare a questo algoritmo studiamo prima un esempio. Ipotizziamo di avere uno schema $R = ABCDEH$, e di avere un insieme di dipendenze funzionali $F = \{AB \longrightarrow CD, EH \longrightarrow D, D \longrightarrow H\}$. Calcoliamo ora alcune chisure:

- $A^+ = A$ (nessun'altro attributo in F è determinato SOLO da A)
- $D^+ = DH$
- $(AB)^+ = ABCDH$ (H perché AB determina CD e D determina H, dunque per transitività AB determina H)
- $(ABE)^+ = ABCDHE$ (dunque ABE è chiave, dato che nessun sottoinsieme di ABE è chiave)

Vediamo ora però l'algorito vero e proprio che calcola velocemnte la chiusura di un qualsiasi insieme di attributi:

```
ChiusuraInsieme(R,F,X): // R schema, F insieme di dipendenze funzionali, X sottoinsieme di R
    Z = X
    S = {A in R | esiste Y -> V in F dove A in V e Y in Z}
    // Aggiungo un attributo A se fa parte di una dipendenza funzionale dove a sinistra c'è un valore Y che è contenuto in Z e a destra c'è un V tale che A è contenuto in V
    while S non è sottoinsieme di Z:
        Z = Z unione S // Z aumenta con Z
        S = {A in R se esiste Y -> V in F t.c. A in V e Y in Z} // Aggiungo nuovi valori
    // Esco dal while quando S in un giro del while non cambia più valore, ossia sono finiti gli attributi che dipendono da X
    return Z
```

N.B. Questo algoritmo è un algoritmo *greedy*: ciò significa che inizia calcolando una soluzione non coretta al 100% ma che, aggiungendo sempre più informazioni, ci permette di arrivare alla soluzione desiderata.

Proviamo a eseguire questo algoritmo con $X = ABE$:

- Prima del while: $Z = \{ABE\}, S = \{CD\}$
- 1° iterazione: $Z = \{ABECD\}, S = \{CDH\}$
- 2° iterazione: $Z = \{ABECDH\}, S = \{CDH\}$
- Fine while: ritorniamo $Z = \{ABECDH\}$, come ci aspettavamo

Dimostriamo ora in modo formale che questo algoritmo che il valore ritornato sia sempre X^+.

Dimostrazione

Indichiamo con Z_0 il valore iniziale di Z e con Z_i il valore di Z alla i-esima iterazione del while, e la stessa cosa facciamo con S_0 e S_i. Dunque avremo $Z_0 = X, Z_1 = Z_0 \cup S_0, \ldots, Z_f = Z_{f-1} \cup S_{f-1}$. Z_f è il valore finale di Z. Questo sicuramente esiste dato che il while continua fino a quando aggiungiamo a Z un attributi di R, ed essendo questi attributi in numero finito prima o poi il while si concluderà sicuramente. Per dimostrare quindi che l'algoritmo ci restituisca X^+ non ci basta che dimostrare che $Z_f = X^+$. Per farlo dividiamo la dimostrazione in due parti: nella prima dimostreremo che $Z_f \subseteq X^+$, mentre nella seconda dimostreremo che $X^+ \subseteq Z_f$.

- $Z_f \subseteq X^+$
  Dimostriamo questa prima parte per induzione.
  - Caso base: $i = 0$
    $Z_0 = X$, e dato che $X \subseteq X^+$ è sempre vero, troviamo che $Z_0 \subseteq X^+$.
  - Ipotesi induttiva
    Assumiamo vero che $Z_i \subseteq X^+$.
  - Passo induttivo
    Dobbiamo quindi dimostrare che $Z_{i+1} \subseteq X^+$ sia vero. Dato che $\forall A \in Z_{i+1}$ è vero che $A \in Z_i \cup S_i$, dobbiamo analizzare 2 casi:
    1° Caso: $A \in Z_i$
    Per ipotesi induttiva sappiammo che $Z_i \subseteq X^+$, dunque troviamo proprio che $A \subseteq X^+$
    2° Caso: $A \in S_i$
    Questo significa che $\exists Y \longrightarrow V \in F$ tale che $A \subseteq V$ e $Y \subseteq Z_i$. Dato che $Z_i \subseteq X^+$ per ipotsi induttiva, troviamo che $Y \subseteq X^+$, e per il lemma di chiusura troviamo che $X \longrightarrow Y \in F^A$. A questo punto, per l'assioma di transitività, troviamo che $X \longrightarrow V \in F^A$, e riapplicando il lemma di chiusura troviamo che $V \subseteq X^+$, e dato che $A \subseteq V$, troviamo proprio che $A \subseteq X^+$.
    Dato che in entrambi i casi troviamo che $A \subseteq X^+$, possiamo concludere che $Z_{i+1} \subseteq X^+$.
    ■
- $X^+ \subseteq Z_f$
  Vogliamo quindi dimostrare che $A \in X^+ \implies A \in Z_f$. Dato che $A \in X^+$, per il lemma

di chisura sappiamo che $X \longrightarrow A \in F^+$. Dunque sappiamo che questa dipendenza è soddisfatta da ogni istanza legale. Riprendiamo a questo punto la stessa istanza r già vista nella dimostrazione del teorema $F^+ = F^A$, e dimostriamo che questa istanza è legale.

r

Z_f				$R - Z_f$			
1	1	...	1	1	1	...	1
1	1	...	1	0	0	...	0

Prendiamo ora una dipendenza funzionale $Y \longrightarrow V \in F$. Guardando r notiamo che $t_1[Y] = t_2[Y] \implies Y \subseteq Z_f$. Guardando ora la definizione di S, notiamo che qui ci sono tutti i signoli attributi A di V (dato che $Y \subseteq Z_f$). Dunque $\forall A \in V \;\; A \subseteq S_f$, ossia $V \subseteq S_f$, e dato che $S_f \subseteq Z_f$ (perché questo è l'ultimo valore di S, dunque stiamo uscendo dal while e dunque la condizione $S \not\subseteq Z$ non è più soddisfatta), troviamo che $V \subseteq Z_f$, ossia $t_1[V] = t_2[V]$, dunque troviamo che la dipendenza $Y \longrightarrow V$ è rispettata in r. Dato che questo vale per ogni dipendenza in F, concludiamo che r è legale.
A questo punto, dato che $X = Z_0 \subseteq Z_f$, troviamo che $t_1[X] = t_2[X]$, e dato che $X \longrightarrow A$ (l'abbiamo visto all'inizio il perché questo è vero), troviamo che $t_1[X] = t_2[X] \implies t_1[A] = t_2[A]$, e affinché questo sia vero è necessario che $A \in Z_f$. ■

5.7 Trovare le chiavi di uno schema

Ora ripendiamo di nuovo lo schema $R = ABCDEH$, con un nuovo insieme di dipendenze funzionali $F = \{AB \longrightarrow CD, C \longrightarrow E, AB \longrightarrow E, ABC \longrightarrow D\}$, proviamo a calcolare $(ABH)^+$ usando il nostro algoritmo:

- Prima del while: $Z_0 = \{ABH\}, S_0 = \{CDE\}$
- 1° iterazione: $Z_1 = \{ABHCDE\}, S_1 = \{CDE\}$
- Fine while: ritorniamo $Z = \{ABHCDE\}$

A questo punto ci domandiamo: ABH è chiave? Sicuramente $(ABH)^+ = R$ (dunque ABH è superchiave), ma esiste un sottoinsieme di ABH che è con questa stessa proprietà? Per verificarlo dovremmo teoricamente fare $2^3 - 2 = 6$ verifiche (AB, AH, BH, A, B, H), ossia un numero esponenziale di verifiche, ma non tutti questi controlli sono veramente necessari. Infatti, in questo caso possiamo notare due cose importanti:

- H sicruamente deve essere nella chiave, dato che non è mai stato determinato, dunque ci basta controllare tutti i sottoinsiemi con H all'interno.
- Non essendo né BH né AH delle chiavi, possiamo già concludere che nemmeno i loro rispettivi sottoinsiemi lo siano, evitandoci così degli altri controlli.

Dunque ABH concludiamo che **è una chiave**. Ma è l'unica? In questo caso si, per un motivo molto semplice: non è solo H a non essere determinato, ma nemmeno AB lo sono, dunque

sono attributi che devono per forza di cose trovarsi nella chiave, e dato che già ABH è chiave, concludiamo che questa **è unica**.

Prendiamo ora uno schema $R = ABCDEGH$ e un insieme di dipendenze funzionali $F = \{AB \longrightarrow D, G \longrightarrow A, G \longrightarrow B, H \longrightarrow E, H \longrightarrow G, D \longrightarrow H\}$. Cerchiamo *tutte* le 4 chiavi in questo schema.

Potremmo andare per froza bruta, e verificare ogni possibile combinazione di attributi e trovare le uniche che possano funzionare come chiavi, ma questo è un approccio molto costoso e inefficace. Generalmente si parte quindi da un insieme di attributi che determinano qualcosa in F e ne si calcola la chiusura. In questo caso, inoltre, notiamo che l'attributo C non è mai determinata, dunque sappiamo già con certezza che C *appartiene a ogni chiave*, e allo stesso modo notiamo invece che E *non sarà mai in nessuna chiave*, dato che è determinato ma non determina nulla. Calcoliamo dunque la chiusura $(ABC)^+$, che vale proprio R. Dunque sicuramente ABC è superchiave, ma dobbiamo ora verificare che le chiusure $(AC)^+$ e $(BC)^+$ non valgano R. Dato che $(AC)^+ = AC$ e $(BC)^+ = BC$, troviamo che $\boldsymbol{ABC}$ è una delle 4 chiavi che cercavamo.
Ora, dato che G determina sia A che B, è molto probabile che CG sia un'altra chiave. Infatti, $(CG)^+ = ABCDEHG$, e dato che $C^+ = C$, troviamo proprio che $\boldsymbol{CG}$ è chiave.
A questo punto, dato che H determina G, è molto probabile che anche CH sia una chiave. Infatti, $(CH)^+ = ABCDEHG$, e dato ceh C da sola abbiamo già visto che non può essere chiave, troviamo che $\boldsymbol{CH}$ è chiave.
Infine, dato che H è determinato da D, troviamo che anche CD è molto probabilmente una chiave. Infatti, $(CD)^+ = ABCDEHG$, e dato che C non è una chiave troviamo che $\boldsymbol{CD}$ è una delle quattro chiavi che volevamo.
Concludiamo quindi che $\boldsymbol{ABC}, \boldsymbol{CG}, \boldsymbol{CH}$ e $\boldsymbol{CD}$ sono le quattro chiavi di questo schema R.

Proposizione 5.7 – Test unicità delle chiave

Preso uno schema R e un insieme di dipendenze funzionali F, dopo aver calcolato $\boldsymbol{X = \bigcap_{V \to W \in F}\big(R - (W - V)\big)}$, $\boldsymbol{X^+ = R} \implies \boldsymbol{X}$ è l'**unica chiave** di R.

Applichiamo questo test sullo schema $R = ABCDE$, con insieme di dipendenze funzionali $F = \{AB \longrightarrow C, AC \longrightarrow B, D \longrightarrow E\}$. Troviamo che $X = R - C \cap R - D \cap R - E = AD$. Ora, dato che $(AD)^+ = ADE \neq R$, troviamo che AD non è l'unica chiave. Questo significa che non esiste nessuna chiave unica? No. Infatti, $X \neq R$ significa solo che X non è l'unica chiave di R, non che non esista alcuna chiave unica in R.